

Basic statistics with R and the UBStats package

R. Piccarreta

July 2024

Contents

1	Introduction to R and RStudio	3
1.1	Basic syntax: commands, expressions and assignments	3
1.2	R “null” values: NULL, NA, NaN, and Inf	5
1.3	R objects	5
1.3.1	Vectors	5
1.3.2	Factors	7
1.3.3	Matrices	8
1.3.4	Dataframes	8
1.3.5	Lists	12
1.4	Operators, functions, and packages	14
1.4.1	Basic operators	14
1.4.2	Introduction to functions	15
1.4.3	R Packages	16
2	Package UBStats: installation and data	18
3	Descriptive statistics: distributions	20
3.1	Introduction to the functions	20
3.2	Univariate analysis	21
3.2.1	Frequency tables	21
3.2.2	Plots of univariate distributions	23
3.3	Bivariate analysis	27
3.3.1	Cross-tabs: Joint and conditional distributions	27
3.3.2	Plots of joint and conditional distributions	29
4	Descriptive statistics: Summaries	34
4.1	Summaries of a single variable	34
4.1.1	Conditional summaries of a single variable	35
4.1.2	Graphical display of conditional central tendency measures	36
4.2	Linear association: covariance and correlation	38
5	Working with random numbers in R	39
5.1	Normal r.v.	39
5.2	Student’s t r.v.	40
5.3	Chi-square and F r.v.	40
6	Inference	42
6.1	Inference on the mean	43
6.1.1	Confidence intervals for the mean	43
6.1.2	Test of hypotheses on the mean	44
6.2	Inference on the proportion	45

6.2.1	Confidence intervals for the proportion	45
6.2.2	Test of hypotheses on the proportion	46
6.3	Inference on the difference between means	47
6.3.1	Confidence intervals for independent samples	48
6.3.2	Hypotheses testing for independent samples	52
6.3.3	Test on the equality of variance for independent samples	54
6.3.4	Confidence intervals for paired samples	54
6.3.5	Hypotheses testing for paired samples	56
6.4	Inference on the difference between proportions	57
6.4.1	Confidence intervals for the difference between proportions	57
6.4.2	Hypotheses testing on the difference between proportions	59
6.5	Goodness-of-fit and independence tests	61
6.5.1	Goodness-of-fit (fully specified probabilities)	61
6.5.2	Test of independence	62
6.6	Test on the correlation coefficient	63
7	The linear regression model	64
7.1	Estimating a regression model	64
7.1.1	Simple linear regression	64
7.1.2	Multiple linear regression	65
7.1.3	A closer look at the regression output	66
7.2	Predictions	68
7.3	Analysis of residuals and of influential observations	70
7.3.1	Other regression diagnostics	73

This material has been prepared and is intended only for students attending the course “Statistics”, cod. 30001, at Bocconi University. It is forbidden to circulate or to quote this report without authorization.

The document offers an overview of the most important tools to analyze data using R, but it cannot be considered as an exhaustive introduction to the software and it is specifically focused on the concepts introduced in the course.

For the sake of completeness, some tools are presented even if not used during the course; this specific material is framed in red in the text.

1 Introduction to R and RStudio

R (<https://cran.r-project.org/>) is an integrated suite of software facilities for data manipulation, calculation and graphical display, working on a wide variety of UNIX platforms, Windows and MacOS. It can be installed by downloading the binaries for base distribution from the “Comprehensive R Archive Network” (CRAN) (<https://cran.r-project.org/bin/windows/base/> or <https://cran.r-project.org/bin/macosx/base/>).

To interact with R we use RStudio (<https://rstudio.com/>), a set of integrated tools designed to help you be more productive with R. It includes a console, a syntax-highlighting editor supporting direct code execution, and a variety of tools for plotting, viewing history, and managing your workspace. To download RStudio, go to <https://rstudio.com/products/rstudio/download/> and download RStudio Desktop

RStudio is organized into four windows (or panels), represented in Figure 1 below.

Console (bottom-left): This is the execution pane, used by R to interact with the user. It displays the submitted code, gives information about its elaboration – possibly signaling errors – and, if required, displays the produced output. It is possible to work interactively and to enter commands directly on the Console panel but it is generally preferable to write and collect commands in an R script (also to keep track of the required elaborations).

Source (top-left). This is where **R scripts** can be created and edited. To open a new **R script** (or code) select **New File|R Script** from the menu **File**.

The code written in the Source panel will be executed only when it is sent to the Console panel (see below) to be run. To run the code (or a portion of it) it is possible to select the portion of code to be submitted, and to click on the *Run* button on the top right of the panel. It is also possible to use the hot-key *Control+Enter* (*Command+Return* on Mac). The results of the elaborations can be displayed directly on the Console (or in the Plot window), can be stored in an object, or written on disk. Scripts can be saved using the **Save** or **Save as** options from the menu **File**.

Environment/History (top-right). The *Environment* tab displays the names of the objects created throughout the current session, together with some basic information (e.g., the dimensions of the objects and the type of the objects). The *History* tab lists the R commands executed during the R session.

Files/Plots/Packages/Help (bottom-right): This is a multipurpose pane showing the contents of directories on your computer (*File*), the obtained plots (*Plots*), a list of the available and active contributed packages (*Packages*; see sec. 1.4.3), and the (required) help pages for R functions and packages (*Help*).

1.1 Basic syntax: commands, expressions and assignments

R is an expression language with a very simple syntax. Elementary commands consist of either **expressions** or **assignments**, typically separated by a newline. Comments can be put almost anywhere, starting with a hashmark (`#`): everything to the end of the line is a comment.

Expressions (given as a command in a script or on the console) are evaluated by R, executed (if valid), and printed, but the result is not saved:

```
# Expressions given as commands (printed and lost)
3+5*3.5 # a number
> [1] 20.5
# Expression calling a function given as commands (printed and lost)
log(3+5*3.5) # a number
> [1] 3.020425
# Expression based on an operator (printed and lost)
3>0 # a logic value (TRUE/FALSE)
> [1] TRUE
```

Expressions typically include **operators** – here for example the arithmetic operators `+` and `*` (sec. 1.4.1 for a list of the most relevant operators) – and/or **functions**. Section 1.4.2 will introduce functions more formally. At this stage, we count on the intuitive understanding that a function is called by specifying its name (for example `log`) followed (at least) by parentheses `()` typically including the function’s **arguments** separated by commas. The arguments identify the object/s the function should be applied to, and the possible options controlling how the function is applied.

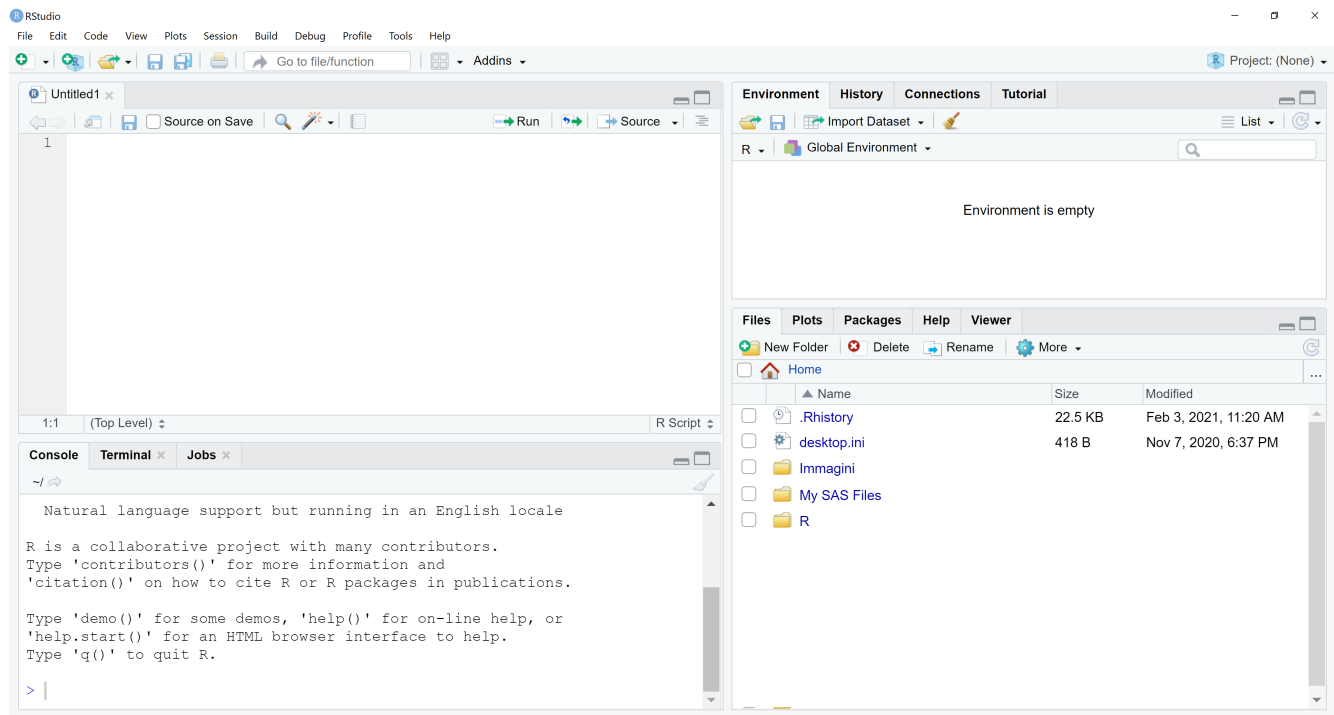


Figure 1: RStudio panels

An **assignment** also evaluates an expression, but in addition it passes the value to an R object. The assignment operators are '`<-`' or '`=`' (we will typically use the former). For example, the commands `x <- 3*6`, `x = 3*6`, but also `3*6 -> x` create an object named `x` whose content is 18. It is also possible to define objects based on previously created objects.

```
# Use function c() (concatenate) to create a NUMERIC VECTOR
x <- c(1,5,8,-1,0,-2,10)
# Use function c() to create a CHARACTER VECTOR
s = c("F","M","F","F","M","F","M","F","M","F")
# Use operator '>' to create a LOGIC VECTOR indicating which elements in x are positive
l <- (x>0)
# Create a new numeric vector based on x
y <- x+2
```

Important:

- The **names** assigned to objects can contain all the alphanumeric symbols (A-Z, a-z, 0-9), but *no spaces* are allowed. Symbols '`.`' and '`_`', are also admitted, but a name can only start with a letter or with '`.`' (but the second character can't be a digit).
- When an object is assigned the **same name** as an existing object, R **overwrites** the latter with **no warning**. It is advisable to assign names reflecting the objects' content, avoiding *typical* names (such as `x` or `y`; they are used in this document when we do not care about overwriting).
- R is **case sensitive**: `x` and `X` will point to different objects.

Objects created with assignments are not automatically printed. They can be printed by running their name or using the `print()` function:

```
x
> [1] 1 5 8 -1 0 -2 10
print(s)
> [1] "F" "M" "F" "F" "M" "F" "M" "F" "M" "F"
print(l)
> [1] TRUE TRUE TRUE FALSE FALSE FALSE TRUE
```

1.2 R “null” values: NULL, NA, NaN, and Inf

R language supports some particular “null” values that can be assigned, can result from certain operations, or can be present in imported data. For example:

```
x <- c(1,5,8,-1,0,-2,10)
log(x)
> [1] 0.000000 1.609438 2.079442      NaN      -Inf      NaN 2.302585
```

NaN (Not A Number) is typically produced by numerical computation, for example `log(-2)` or `sqrt(-1)` (logarithm or square root of a negative number), or `(0/0)`

Inf (resp. **-Inf**) stands for infinity (resp. negative infinity), and it results from storing an excessively large number or by numerical computation, for example division by 0 (or `log(0)` as shown above). Note that **Inf** is a number, even if too big to be represented.

In addition, **NA** (Not Available) is a logical constant indicating a *missing* value. In many cases NA are present in datasets corresponding to cases for whom a given information is not available, but they can also be the output of functions.

Finally, **NULL** is used in R to indicate an empty set, and is often returned by expressions and functions whose value is undefined.

Notably, by default in R operations on NA and NaN return respectively NA and NaN, because the specification of the operation is considered as incomplete, so that the result cannot be known.

1.3 R objects

R allows defining different types of data structures, that differ for their characteristics and possibly for the specific functions that can be applied to them (*object-oriented style of programming*: some functions return different results depending on the structure of the input objects). Focusing on the objects relevant in our course, it is worth mentioning:

- **Vectors**: ordered collection of elements of the same **type** (numeric, character, logical)
- **Matrices**: sets of elements of the same **type** (numeric, character, logical) arranged in a 2-dimensional **table**.
- **Factors**: vectors used to specify a discrete classification of the components of other vectors, typically to assign user-defined labels to the values of another vector.
- **Dataframes**: a dataframe is a collection of columns with different characteristics (vectors of any type – numeric, character, or logical – and/or factors).
- **Lists**: collection of objects of any type, typically used to organize the possibly articulated output of some functions.

In the following, we focus on vectors and factors (the constituents of dataframes), and on dataframes; matrices (only marginally used in the course) and lists will be only briefly introduced, and will be discussed more into details with reference to specific applications.

1.3.1 Vectors

The simplest structures in R, **vectors**, consist of an ordered collection of elements of the same type (numeric, character or logical). As already seen, a vector can be created using the function `c()`, that can take an arbitrary number of arguments and returns a vector concatenating its arguments (possibly vectors themselves) end to end. Another useful function is `rep()`, that creates a vector of elements repeated for a specific number of times:

```
x <- c(1,5,8,0,1,10,3)
print(x)
> [1] 1 5 8 0 1 10 3
s1 <- rep("North",4)
s2 <- rep("South",3)
s1
> [1] "North" "North" "North" "North"
s2
> [1] "South" "South" "South"
s <- c(s1, s2) # same as s <- c(rep("North",4), rep("South",3))
s
> [1] "North" "North" "North" "North" "South" "South" "South"
```

Note that if elements of different types are combined, R forces the vector to a unique type:

```
c(1,3,11,"F","M") # combination of numeric and character types
> [1] "1" "3" "11" "F" "M"
```

1.3.1.1 Length and names of a vector. The function `length()` returns the length of an object; for vectors, it is the number of their elements. The function `names()` allows to assign names to a vector's elements or to access names previously defined:

```
x <- c(1,5,8,0,1,10,3)
names(x)<-c("Berlin","Paris","Amsterdam","Stockholm","Lisbon","Madrid","Rome")
length(x)
> [1] 7
x
> Berlin Paris Amsterdam Stockholm Lisbon Madrid Rome
> 1 5 8 0 1 10 3
```

It is also possible to assign names when a vector is created:

```
x<-c("Berlin"=1,"Paris"=5,"Amsterdam"=8,"Stockholm"=0,"Lisbon"=1,"Madrid"=10,"Rome"=3)
```

1.3.1.2 Indexing and subsetting vectors. Subsets of the elements of a vector may be selected by appending to the name of the vector the operator `[ ]`, whose argument is a vector whose elements are the positions of the elements to select, or their names (if assigned), or the condition/s that they must satisfy:

```
x[3] # select the 3rd element
> Amsterdam
> 8
x.sub1 <- x[c(1,4,6)] # select elements in positions 1,4,6
x.sub2 <- x[c("Berlin","Stockholm","Madrid")] # select elements using their names
x.sub3 <- x[-c(1,4,5,7)] # remove elements in positions 1,4,5,7

x.sub1 # same as x.sub2
> Berlin Stockholm Madrid
> 1 0 10
x.sub3
> Paris Amsterdam Madrid
> 5 8 10
```

The elements of a vector can be selected also based on conditions defined on the vector itself or on another vector:

```
x.sub4 <- x[x>=5] # select elements >= 5
x.sub4
> Paris Amsterdam Madrid
> 5 8 10

s
> [1] "North" "North" "North" "North" "South" "South" "South"
s.n <- s=="North" # identify elements satisfying a condition
s.n
> [1] TRUE TRUE TRUE TRUE FALSE FALSE FALSE
# select based on a condition defined on another vector
x.sub5 <- x[s=="North"]
x.sub6 <- x[s.n]
x.sub5 # same as x.sub6
> Berlin Paris Amsterdam Stockholm
> 1 5 8 0
```

It is also possible to selectively modify elements of a vector:

```
# selectively modifying:
x.new <- x
x.new[x<5] <- 0 # assign 0 to elements < 5
x.new
> Berlin Paris Amsterdam Stockholm Lisbon Madrid Rome
> 0 5 8 0 0 10 0
```

1.3.2 Factors

Factors are vectors used to assign to the values of vectors of any type labels (*levels*), possibly specified by the analyst, thus allowing to easily recode the values of an input vector. They are created applying the function `factor()` to a vector:

```
char.1 <- c("F","M","F","F","M","F","M","F","M","F")
factor.1 <- factor(char.1)
factor.1
> [1] F M F F M F M F M F
> Levels: F M
```

The function `levels()` allows accessing or modifying the levels of a factor:

```
levels(factor.1) <- c("Female", "Male") # recoding
factor.1
> [1] Female Male Female Female Male Female Male Female Male Female
> Levels: Female Male
```

It is possible to explicitly assign labels when defining the factor, and also to change the order of the levels using the arguments `levels` and `labels` when creating the factor.

```
factor.1 <- factor(char.1, levels = c("M","F"), labels=c("Male","Female"))

char.2 <- c(1,2,1,1,2,1,2,1,2,1) # numeric vector whose values indicate the gender
factor.2 <- factor(char.2, levels = c(1,2), labels = c("Female", "Male"))

# same as:
factor.2 <- factor(char.2, levels = c(1,2))
levels(factor.2) <- c("Female", "Male")
factor.2
> [1] Female Male Female Female Male Female Male Female Male Female
> Levels: Female Male
```

This is particularly useful to control the order of levels, for example when the strings in the input vector have an ordering and the alphabetical order is not adequate:

```
# reorganize the order of the levels
char.3 <- c("Low","Medium","High","Medium","High","VeryLow","High","Medium","Low")
factor(char.3) # levels are in alphabetical order!
> [1] Low Medium High Medium High VeryLow High Medium Low
> Levels: High Low Medium VeryLow

# create a factor by re-ordering the values:
factor.3 <- factor(char.3, levels = c("VeryLow","Low","Medium","High","VeryHigh"))
factor.3
> [1] Low Medium High Medium High VeryLow High Medium Low
> Levels: VeryLow Low Medium High VeryHigh
```

Finally, it is possible to create a factor by assigning the same level to different values:

```
factor.4 <- factor.3
levels(factor.4) <- c("Low","Low","Medium","High","High")
```

```
# same as
factor.4<-factor(char.3, levels = c("VeryLow", "Low", "Medium", "High", "VeryHigh"),
                 labels = c("Low", "Low", "Medium", "High", "High"))
factor.4
> [1] Low      Medium High      Medium High      Low      High      Medium Low
> Levels: Low Medium High
```

1.3.3 Matrices

A matrix can be regarded as a generalization of a vector. Indeed, it is a vector arranged in a 2-dimensional table or as a row-wise or column-wise concatenation of vectors. A matrix can be created using the function `matrix()`, with arguments `data` (the vector to be arranged into a matrix), `nrow` and `ncol` (the number of rows and columns of the matrix), and `byrow`, a logical value specifying whether the matrix should be filled by column (`byrow = FALSE`, default) or by row (`byrow = TRUE`).

```
num.v <- c(1,3,4.1,5,6.1,8,9,10.11,30,5.3)
mat.num.1 <- matrix(num.v, nrow = 2, ncol = 5) # matrix filled by column
mat.num.1
>      [,1] [,2] [,3] [,4] [,5]
> [1,]    1  4.1  6.1  9.00 30.0
> [2,]    3  5.0  8.0 10.11  5.3
mat.num.2<-matrix(num.v, nrow = 2, ncol = 5, byrow = TRUE) # matrix filled by row
mat.num.2
>      [,1] [,2] [,3] [,4] [,5]
> [1,]    1    3  4.10    5  6.1
> [2,]    8    9 10.11   30  5.3
```

Matrices can also be created using the functions `cbind()` or `rbind()` to bind together vectors (or matrices) column-wise or row-wise respectively:

```
num.2A <- c(1,3,4.1,5,6.1)
num.2B <- c(8,9,10.11,30,5.3)
mat.num.2 <- rbind(num.2A, num.2B) # row-binding
mat.num.2
>      [,1] [,2] [,3] [,4] [,5]
> num.2A    1    3  4.10    5  6.1
> num.2B    8    9 10.11   30  5.3

mat.num.3 <- matrix(c(1:3, 21:23, 1,120,240, 5,9,12), nrow = 3, ncol = 4)
mat.num.3 <- cbind(1:3, 21:23, c(1,120,240), c(5,9,12)) # column-binding
mat.num.3
>      [,1] [,2] [,3] [,4]
> [1,]    1   21    1    5
> [2,]    2   22  120    9
> [3,]    3   23  240   12
```

As for vectors, there are functions to access a matrix' dimensions, to assign names to its rows and columns, and to select elements, rows or columns. Such functions, working both for matrices and for dataframes, will be presented in the next section.

1.3.4 Dataframes

A dataframe may for many purposes be regarded as a matrix with columns possibly of **different** types and characteristics. Its columns can be vectors of any mode as well as factors.

It is possible to create a dataframe directly in R. Nonetheless, dataframes are typically defined by importing datasets from other sources or by loading previously created R dataframes.

1.3.4.1 Importing dataframes. To import a dataset from an **external source** (e.g. .csv or .xlsx) one should select **Import dataset** in the menu **File**, selecting the proper source. For example, to import a dataset saved as a .csv (comma separated values), the correct path is **File->Import Dataset->From Text (base)** (see Figure 2).

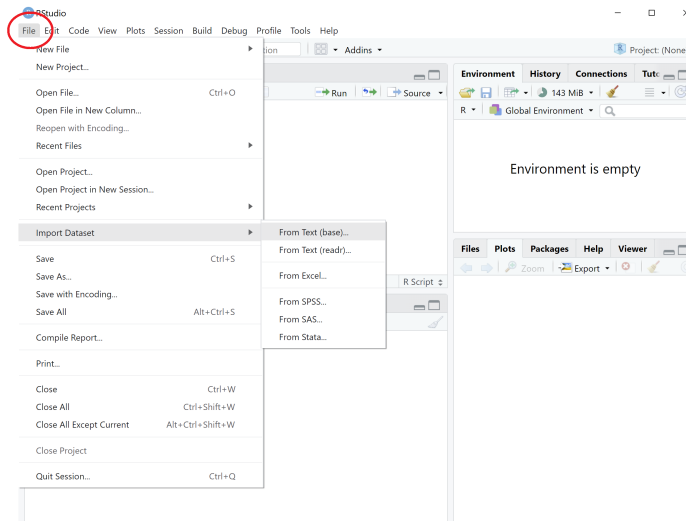


Figure 2: Import dataframe from external source

RStudio will open a window offering a preview of the dataset and allowing to select some import options. For example, Figure 3 reports the preview of the dataset **Loyalty.csv**.

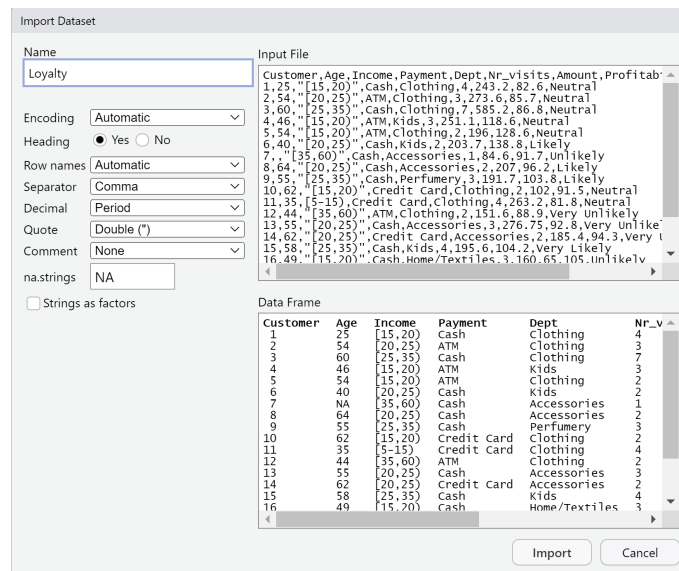


Figure 3: Import window: preview of dataset Loyalty.csv

By clicking on **Import** the dataframe **Loyalty** is created.

The main features of the dataframe can be explored in different ways. The structure of the dataframe can be visualized using the function `str()` (that can be used to inspect the structure of any R object). Also, the first rows of the dataframe can be printed using the function `head()`. Another very useful function available in R to preliminarily explore a dataframe is `summary()`, which offers some information about the type and the main summaries of the variables.

```
str(Loyalty) # visualise the structure of the dataframe
```

```
> 'data.frame': 125 obs. of 9 variables:
> $ Customer      : int  1 2 3 4 5 6 7 8 9 10 ...
> $ Age           : int  25 54 60 46 54 40 NA 64 55 62 ...
```

```
> $ Income      : chr  "[15,20)" "[20,25)" "[25,35)" "[15,20)" ...
> $ Payment     : chr  "Cash" "ATM" "Cash" "ATM" ...
> $ Dept        : chr  "Clothing" "Clothing" "Clothing" "Kids" ...
> $ Nr_visits   : int   4 3 7 3 2 2 1 2 3 2 ...
> $ Amount      : num   243 274 585 251 196 ...
> $ Profitability: num   82.6 85.7 86.8 118.6 128.6 ...
> $ Recommend   : chr  "Neutral" "Neutral" "Neutral" "Neutral" ...
```

```
head(Loyalty) # print the first rows of the dataframe
```

```
> Customer Age Income Payment Dept Nr_visits Amount Profitability Recommend
>      1  25 [15,20)  Cash Clothing      4  243.2      82.6  Neutral
>      2  54 [20,25)  ATM  Clothing      3  273.6      85.7  Neutral
>      3  60 [25,35)  Cash Clothing      7  585.2      86.8  Neutral
>      4  46 [15,20)  ATM   Kids       3  251.1     118.6  Neutral
>      5  54 [15,20)  ATM Clothing      2  196.0     128.6  Neutral
>      6  40 [20,25)  Cash   Kids       2  203.7     138.8   Likely
```

```
summary(Loyalty) # explore the summaries of the variables
```

```
>      Customer      Age      Income      Payment
> Min.      : 1  Min.      :25.00  Length:125  Length:125
> 1st Qu.: 32  1st Qu.:40.75  Class :character  Class :character
> Median : 63  Median :51.50  Mode  :character  Mode  :character
> Mean    : 63  Mean     :50.75
> 3rd Qu.: 94  3rd Qu.:61.00
> Max.    :125  Max.     :79.00
>      NA's      :1
>      Dept      Nr_visits      Amount      Profitability
> Length:125  Min.      :1.000  Min.      : 60.45  Min.      : 81.8
> Class :character  1st Qu.:2.000  1st Qu.:154.50  1st Qu.:139.1
> Mode  :character  Median :2.000  Median :200.70  Median :178.8
>      Mean      :2.664  Mean      :229.26  Mean      :233.6
>      3rd Qu.:3.000  3rd Qu.:281.25  3rd Qu.:280.4
>      Max.      :7.000  Max.      :585.20  Max.      :847.0
>      Recommend
> Length:125
> Class :character
> Mode  :character
```

A dataframe can be saved on disk using the function `save()`, by specifying the dataframe to save and the file to create (path and name of the saved dataframe):

```
save(Loyalty,file="C:/myfolder/subfolder/Loyalty.RData")
```

The same syntax can be used to save more objects, by listing their names separated by commas. Note that the separator between the folders' names is forward slash (/), **irrespective** of the operating system.

To **load** a dataframe previously created with R one should click **Open File** in the menu **File** and should select the R data to open; R will load the dataframe in the environment after confirmation.

1.3.4.2 Creating dataframes. Dataframes can also be directly created in R using the function `data.frame()`, whose arguments are the vectors or factors constituting the dataframe. For example, consider information collected on 10 individuals:

```
income <- c(40, 55, 60, 112, 34, 89, NA, 98, 121, 73)
gender <- factor(c("f", "f", "f", "m", "m", "f", "m", "f", "f", "m"))
sat <- factor(c("low", "medium", "high", "high", "low", "high", "medium", "high", "low", "low"),
             levels=c("low","medium","high")) # factor with ordered levels
```

```
# create the dataframe:
dat <- data.frame(income, gender, sat)
str(dat)
> 'data.frame': 10 obs. of 3 variables:
> $ income: num 40 55 60 112 34 89 NA 98 121 73
> $ gender: Factor w/ 2 levels "f","m": 1 1 1 2 2 1 2 1 1 2
> $ sat : Factor w/ 3 levels "low","medium",...: 1 2 3 3 1 3 2 3 1 1
```

1.3.4.3 Dimensions, names of columns and rows in a dataframe. The number of rows and columns in a dataframe can be accessed using the functions `dim()`, or `nrow()` and `ncol()`:

```
dim(Loyalty) # dimensions of the dataframe
> [1] 125 9
nrow(Loyalty)
> [1] 125
ncol(Loyalty)
> [1] 9
```

The functions `rownames()` and `colnames()` allow accessing or modifying a dataframe's rows and columns names.

```
# visualize colnames and rownames
colnames(Loyalty)
> [1] "Customer" "Age" "Income" "Payment"
> [5] "Dept" "Nr_visits" "Amount" "Profitability"
> [9] "Recommend"
head(rownames(Loyalty)) # first rownames
> [1] "1" "2" "3" "4" "5" "6"

# modify colnames - assign new vector of columns names
colnames(dat) <- c("Income", "Gender", "Satisfaction")
```

The previous functions can also be used for matrices, but for matrices rows and columns names are not assigned by default, and should be assigned. Moreover, **for dataframes** it is also possible to assign columns names directly at creation:

```
# assign names at creation
dat<-data.frame(Income=income, Gender=gender, Satisfaction=sat)
```

1.3.4.4 Indexing and subsetting dataframes. Elements in a dataframe, can be accessed (or extracted) using the same operator introduced for vectors, `[ ]`, indicating within parentheses the indices (or the names) of the rows and of the columns to be extracted *separated by a comma* (no index should be specified if all the rows are all the columns should be extracted).

```
Loyalty[3,4] # extracting one element
> [1] "Cash"
Loyalty[3,] # extracting the 3rd row (all the columns)
> Customer Age Income Payment Dept Nr_visits Amount Profitability
> 3 3 60 [25,35) Cash Clothing 7 585.2 86.8
> Recommend
> 3 Neutral
Loyalty[,2] # extracting the 2nd column (all the rows)
> [1] 25 54 60 46 54 40 NA 64 55 62 35 44 55 62 58 49 59 70 43 25 27 48 48 65 61
> [26] 56 53 58 67 60 34 35 29 29 30 30 31 54 52 50 47 36 61 68 66 64 67 68 69 44
> [51] 45 46 65 74 52 54 40 43 47 46 67 66 46 42 38 39 34 55 59 66 69 72 75 74 79
> [76] 60 63 56 46 57 52 51 54 63 47 37 35 38 64 62 28 55 58 60 61 61 50 50 63 60
> [101] 41 49 39 33 58 53 37 37 39 41 43 46 76 50 33 32 40 50 51 53 55 48 43 27 28
```

```
# extracting the first 2 rows of the column Payment
Loyalty[1:2,"Payment"]
> [1] "Cash" "ATM"

# extracting the first 5 rows and some columns
Loyalty[1:5,c("Customer","Nr_visits","Amount")]
>   Customer Nr_visits Amount
> 1         1         4  243.2
> 2         2         3  273.6
> 3         3         7  585.2
> 4         4         3  251.1
> 5         5         2  196.0
```

The very same syntax can also be used to index and subset matrices. In addition, for **dataframes** a single column/variable can be selected using a simpler expression, by appending to the name of the dataframe the symbol **\$** followed by the name of the variable. This simplifies the syntax used to modify the values in a variable and/or to add (or erase) columns to a dataframe:

```
## Operator $
Loyalty$Nr_visits           # column/variable Nr_visits
> [1] 4 3 7 3 2 2 1 2 3 2 4 2 3 2 4 3 2 2 7 3 2 2 2 4 3 3 3 3 2 1 4 2 2 6 1 2
> [38] 2 3 3 2 4 4 4 2 2 4 2 3 6 1 2 3 1 2 2 3 4 3 1 2 2 2 4 2 2 4 1 2 3 6 2 2 2
> [75] 1 4 3 4 2 3 2 1 2 2 3 3 3 3 2 3 3 3 6 1 2 1 3 1 3 3 2 1 1 3 2 2 1 3 2 6 3
> [112] 2 4 1 4 2 3 6 3 1 2 1 2 2 2

# creating new variables
Loyalty$Amount_V <- Loyalty$Amount/Loyalty$Nr_visits
Loyalty$Recommend.F <- factor(Loyalty$Recommend,
                              levels = c("Very Unlikely","Unlikely","Neutral",
                                           "Likely","Very Likely"))

# remove a variable
Loyalty$Toerase <- 0 # all values are 0
Loyalty$Toerase <- NULL # erase the column from the dataframe
```

Additional Material

1.3.5 Lists

Lists are very important in R, because they are often used to store and organize the output of functions. We will illustrate how to work with lists with specific reference to the output of the function `lm()` that will be used later to apply regression analysis. Nonetheless, we briefly show below how to create a list and how to access its elements.

A list is created in a way similar to dataframes with the function `list()`, and consists of an ordered collection of objects – its *components* – that can be objects of any kind. For example, a list could consist of a numeric vector, a single logical value, a matrix, a dataframe, but also a function and possibly a list itself:

```
list1<-list(dat, "data on income, gender, satisfaction", 5:8)
list2<-list(list1, "Self created", "24-07-2023")
```

The function `str()` can be used to visualize the structure of a list.

```
str(list1) # visualise the structure of the list
> List of 3
> $ : 'data.frame': 10 obs. of 3 variables:
> ..$ Income      : num [1:10] 40 55 60 112 34 89 NA 98 121 73
> ..$ Gender      : Factor w/ 2 levels "f","m": 1 1 1 2 2 1 2 1 1 2
> ..$ Satisfaction: Factor w/ 3 levels "low","medium",...: 1 2 3 3 1 3 2 3 1 1
> $ : chr "data on income, gender, satisfaction"
> $ : int [1:4] 5 6 7 8
```

Note that as for dataframes the operator `$` can be used to access the components of a list.

1.3.5.1 Dimensions, names of elements in a list. The **length** of a list is the number of its (top level) components, and can be accessed using the function `length()`.

```
length(list1)
> [1] 3
length(list2)
> [1] 3
```

The components of the lists are always numbered, and can be named using the function `names()`. As for dataframes, the names of the elements in the list can be specified directly in the function `list()` used to define the object:

```
names(list1)<-c("InputData", "Content", "Indices") # assign names to an existing list

# create a list and simultaneously assign names
list2<-list(Data=list1, Source="Self.Created", Date="24-07-2023")
```

1.3.5.2 Indexing and subsetting lists. The **single** elements of a list can be accessed or extracted using the operator `[[ ]]`, whose argument is the position or of the element in the list or its name. For example `list1[[1]]`, `list1[["InputData"]]` or (similarly to what seen for dataframes) `list1$InputData` extract the first *object*, named `InputData` in `list1`.

```
#extract the 3rd component of the list
list2[[3]] # same as list2[["Date"]] or list2$Date
> [1] "24-07-2023"
```

As for other objects, the operator `[ ]` can be used to extract a *sub-list* including the components specified in a vector whose elements are the positions of the components to select or their names:

```
#extract the 2nd and the 3rd component of the list
list2[c(2,3)] # same as list2[c("Source","Date")]
> $Source
> [1] "Self.Created"
>
> $Date
> [1] "24-07-2023"
```

Sequential indexing is particularly important for lists, that can be organized into very complex structures. For example in:

```
list2$Data$InputData[,c(2,1,3)] # same as list2[[1]][[1]][,c(2,1,3)]
```

`list2$Data` selects the first object in the list, which is a list itself (`list1`), `list2$Data$InputData` selects the first object in that list, which is a dataframe, and `list2$Data$InputData[,c(2,1,3)]` selects the 2nd, 1st and 3rd columns of such dataframe; thus, the required object is the original dataframe `dat` with a different arrangement of the columns.

1.4 Operators, functions, and packages

In this section we provide some basic information on operators, functions and packages.

1.4.1 Basic operators

There are three main categories of operators in R to manipulate data: arithmetic, relational (comparison), and logical (in addition, note that ‘<’ and ‘=’ are **assignment** operators). Below is a list of (a selection of) operators with their description and usage.

Arithmetic operators act on numbers and return **numbers**:

Operator	Description	Input	Usage	Result
+	Addition	$x < -11$; $y < -3$	$x + y$	14
-	Subtraction		$x - y$	8
*	Multiplication		$x * y$	33
/	Division		x / y	3.6666667
^	Raise to Power		x^y	1331

Comparison operators verify whether the relation between two operands holds or not, and return the **logical** values TRUE or FALSE.

Operator	Description	Input	Usage	Result
<	Lesser than	$x < -11$; $y < -3$	$x < y$	FALSE
>	Greater than		$x > y$	TRUE
<=	Lesser than or equal		$x <= y$	FALSE
>=	Greater than or equal		$x >= y$	TRUE
==	Equal		$x == y$	FALSE
		$c < -\text{"Yes"}; d < -\text{"yes"}$	$c == d$	FALSE
!=	Different		$x != y$	TRUE
		$c < -\text{"Yes"}; d < -\text{"yes"}$	$c != d$	TRUE

Logical operators work on operands whose value indicate whether a condition is true or false. For example, if $x < -11$ and $y < -3$ the objects $(x > 2)$, $(y > 5)$ and $(y > 1)$ describe conditions: the first and the third conditions are satisfied (TRUE), whilst the second is not (FALSE).

Operator	Description	Input	Usage	Result
!	Logical NOT	$x < -11$; $y < -3$	$!(x > 2)$	FALSE
			$!(y > 5)$	TRUE
&	Logical AND		$(x > 2) \& (y > 5)$	FALSE
	Logical OR		$(x > 2) (y > 5)$	TRUE
			$(x > 2) (y > 1)$	TRUE

All these operators can be applied to vectors, and are performed element by element. For example, for numeric vectors:

```
x <- c(1.234, 7.21, 5, 4.1, 10.678)
y <- c(1, 8, 3, 6, 2)
x + y                # arithmetic operator on vectors
> [1]  2.234 15.210  8.000 10.100 12.678
(x > 4)              # comparison operator on vectors
> [1] FALSE  TRUE  TRUE  TRUE  TRUE
(x == y)
> [1] FALSE FALSE FALSE FALSE FALSE
```

```
(x > 3) & (y < 5)  # logical & comparison operators on vectors
> [1] FALSE FALSE TRUE FALSE TRUE
(x > 3) | (y < 5)
> [1] TRUE TRUE TRUE TRUE TRUE
```

and for character vectors:

```
x <- c("Good", "fair", "Bad", "Bad", "Fair")
y <- c("F", "M", "F", "M", "M")

x == "Fair"          # comparison operator on vector
> [1] FALSE FALSE FALSE FALSE TRUE
(x == "Bad") & (y == "F")  # logical operators on vectors
> [1] FALSE FALSE TRUE FALSE FALSE
(x == "Fair") | (y == "M") # logical operators on vectors
> [1] FALSE TRUE FALSE TRUE TRUE
(x == "Fair" | x == "fair") | (y == "M") # parentheses used to specify the order of operations
> [1] FALSE TRUE FALSE TRUE TRUE
```

1.4.2 Introduction to functions

An assignment in R typically involves a call to a **function**, an (ordered) collection of statements that can be applied to R objects to obtain a certain result.

Functions are identified by their **name** (e.g. `log` calculates the logarithm of its argument). Typically functions in R depend on **arguments**, listed within parentheses after the function's name, separated by commas. The arguments identify the object/s the function should be applied to, and the possible options controlling how the function is applied. An argument is a placeholder: when a function is invoked, specific values are passed to the arguments. The complete list of arguments of a function can be retrieved using the function `args()`. For example, the function `log`:

```
args(log)
> function (x, base = exp(1))
> NULL
```

calculates the logarithm of `x` with base `base`. Note that a **default** value is assigned to `base`, `base=exp(1)`. Thus, while a value for `x` must be specified, when the function is called without any value specified for `base` the default value `exp(1)` will be assigned to it.

Observe instead that the call:

```
log(base = 10)
> Error in eval(expr, envir, enclos): argument "x" is missing, with no default
```

returns an error because no default value is assigned to `x`.

The arguments of a function can be listed within parentheses in different ways:

- **Named form:** arguments provided indicating for each the respective name may be listed in any order in a call. This is the approach that we recommend to beginners.
- **Un-named form:** arguments provided without indicating their name must be listed in the order reported by `args()`.
- A **mix** of the two forms: initial argument(s) specified in an unnamed, yet positional, form and the named arguments (in any order) after the positional ones; it is also possible to provide some arguments with their names and others in their correct position.

Below are equivalent examples:

```
log(x = 10, base = 2) # named form
log(10, 2)            # un-named form, in correct order
log(base = 2, x = 10) # named form with reversed order
log(10, base = 2)     # only the first argument un-named (in its correct position)
log(base = 2, 10)     # named form for 'base', followed by 'x'
```

A special type of argument is the **dots** argument (`...`), also known as *ellipsis*, which allows the function to take an undefined and arbitrary number of arguments or to pass arguments on to other functions. For example:

```
args(print)
> function (x, ...)
> NULL
```

The function has a first argument `x` which indicates the object to be printed, and a dots argument allowing the user to specify a number of arguments, designed for specific types of inputs.

1.4.2.1 Getting help on functions. It is possible to get information on the syntax and usage of functions (or operators) using the R on-line help.

Typing for example `?log`, or `help(log)` or `help("log")` opens the corresponding help page in the **Help** tab of the **Files/Plots/Packages/Help** panel in RStudio (the search tab in such page can also be used to get help on a function).

The help page for a function displays a **Description** of the function, its **Usage** (syntax), and its **Arguments**. A section **Details/Notes** reports relevant details and possible exceptions. The list of the object/s returned by the function is reported in the **Value** section. Finally, the section **See Also** contains links to help pages close or similar to the current one, and in **Examples** some applications are shown, very useful to understand the typical syntax and the possible alternative usages of the function.

1.4.3 R Packages

The R system includes a huge number of mathematical, statistical and general-purposes functions (distributed with every binary distribution of R, and saved on the hard disk when R is installed). Nonetheless, one of the greatest advantages of R is the rich collection of **R packages** (or **R libraries**) developed by users in the R community to add new functionalities to R. Some packages implement specialized statistical methods, others give access to data or hardware, and others are designed to complement textbooks.

Such packages must be installed (only once) on the hard disk, and must be accessed (loaded) at the beginning of each R session.

Installing packages. In RStudio packages can be easily installed or updated by clicking on the button *Install* in the **Packages** tab of the **Files/Plots/Packages/Help** panel, or by selecting **Tools->Install Packages** in the main menu (see Figure 4).

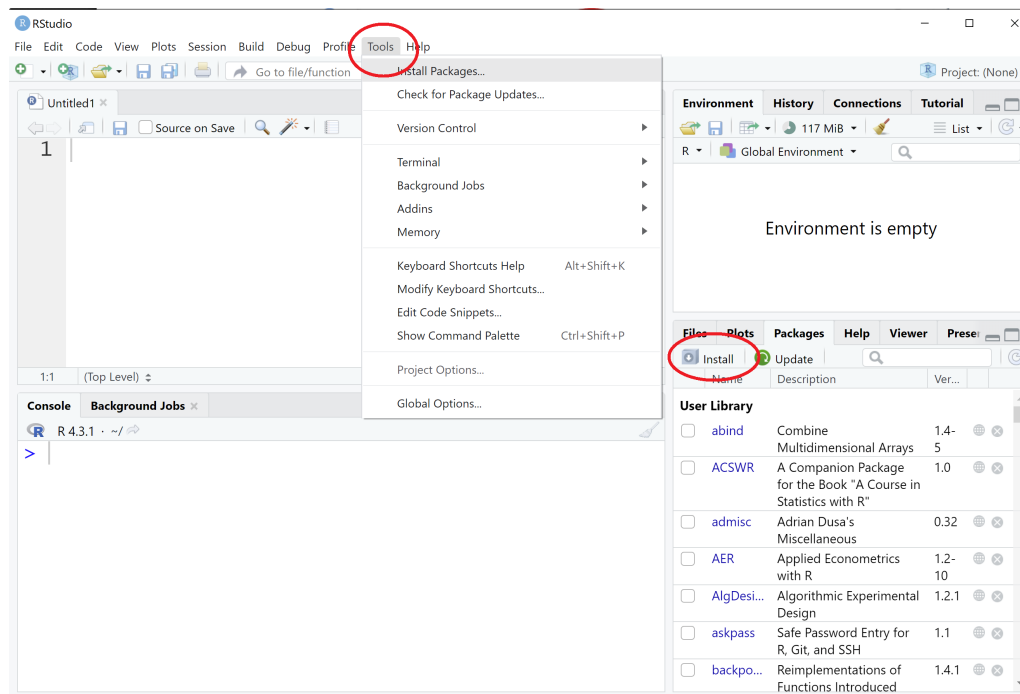


Figure 4: Installation of R packages

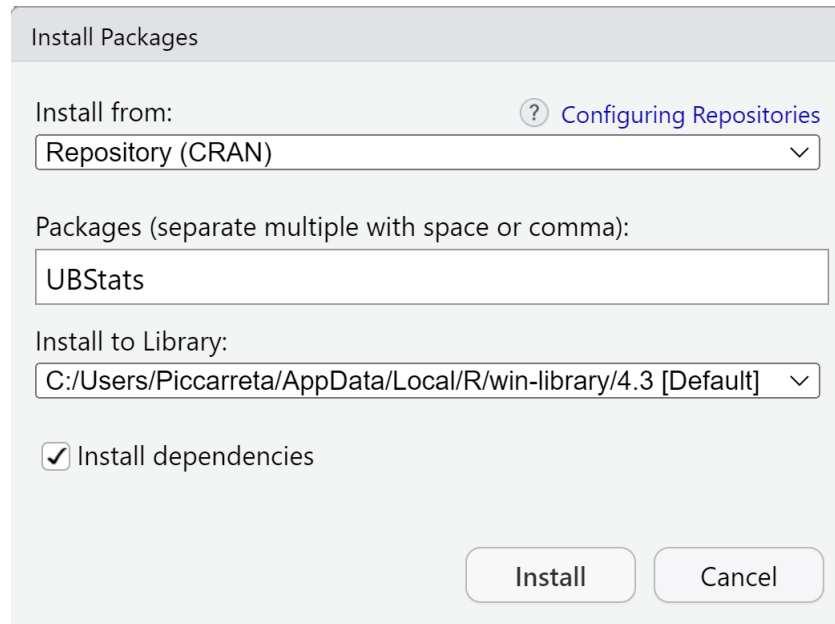


Figure 5: Installation window

In both cases, the installation window reported in Figure 5 is opened. In the **Packages** box, type the name of the package you want to install: here we install `UBStats`. Some packages depend on other packages; To fetch a package and all those it depends on (in any way) that are not already installed, be sure that the corresponding option `Install dependencies` in the installation window is flagged. A package can also be installed by simply running:

```
install.packages("UBStats",dependencies = TRUE)
```

Note that it suffices to install a package only once (or every time you reinstall R on your computer). The list of packages installed on your hard disk is displayed in the tab *Packages* in the **File/Plots/Packages/Help** panel.

Loading packages. The content (e.g. functions) of a package is available only after it is **loaded** during a session. A particular package can be loaded using the function `library()` (see the next section) or by *flagging* the package in the list of **Packages** in panel **Files/Plots/Packages/Help**.

All packages have **help pages** for each of their functions: by default, in a session help pages are available only for functions of **loaded** packages.

2 Package UBStats: installation and data

In this section we introduce the package **UBStats**, developed for the course to simplify basic statistical analysis. Even if a number of functions are available in R at this aim, the package includes a set of functions sharing a similar syntax, and allows dealing with some cases not covered by the basic R functions.

The package must be **installed** as described in the previous section.

```
install.packages("UBStats",dependencies = TRUE)
```

As mentioned, the installation procedure must be run only **once**. Instead, the package must be loaded at the beginning of **each** R session, by flagging **UBStats** in the list of **Packages** in panel **Files/Plots/Packages/Help** or by running:

```
library(UBStats)
```

The package **UBStats** includes, besides functions, two dataframes, named `MktDATA.Orig` and `MktDATA`.

`MktDATA.Orig` contains data collected through a survey on a sample of customers of a company operating in the retail food sector. The company sells products from 3 major categories (referred as A, B, C) Customers can order and buy products in the company physical stores, or on the company's website (they can also place orders on the website and pick them up in one store). Customers' activity in the last two years (observation period) is available to the company; additional information is retrieved through questionnaires or fidelity cards. During the observation period different marketing strategies were adopted to improve customers' fidelisation, and 5 marketing campaigns were launched; a last campaign was launched at the end of the observation period. Below is a list of the variables in the dataframe (levels of the variables listed in alphabetical order):

- **CustId** (num): identification label
- **Gender** (chr): gender (F, M)
- **Age** (num): age (in years)
- **Education** (chr): level of education (College, Graduate, HighSchool, Post-Grad)
- **Marital_Status** (chr): marital status (Divorced, Married, Single, Together, Widow)
- **Children** (num): number of children in the household
- **Kids** (num): number of kids aged less than 12 in the household
- **Income** (chr): customer's income (measured in classes)
- **Baseline** (num): index (from 0 to 1) assigned by the marketing dept indicating how promising the customer was judged at the beginning of the observation period
- **LikeMost** (chr): Most frequently bought category in the last two years (P.A, P.B, P.C)
- **TotVal** (num): amount spent in the last 2 years
- **NPickUp_Purch** (num): number of purchases made on the website and picked up in physical store
- **NWeb_Purch** (num): number of purchases made on company's website and delivered at home
- **NStore_Purch** (num): number of purchases made in a physical store
- **NDeals** (num): number of products purchases with discount
- **CustClass** (chr): customer's classification (assigned by the marketing dept) based on past profitability (Bronze, Gold, Platinum, Silver)
- **PastCampaigns** (num): number of offers accepted by the customer in the last 2 years' marketing campaigns
- **LastCampaign** (num): binary variable (0/1) indicating whether (1) or not (0) the customer accepted the offer in the campaign launched at the end of the observation period.
- **WouldSuggest** (chr): variable signalling whether (Yes) or not (No) the customer declared they would suggest the company's products to friends and family

To make the dataframe available and to visualize its structure (not printed):

```
data(MktDATA.Orig, package = "UBStats")
str(MktDATA.Orig)
```

In the next chapters we will rely on the dataframe `MktDATA`, which has been obtained by adding some variables to `MktDATA.Orig`. Below we illustrate the procedure followed to create `MktDATA` from `MktDATA.Orig`, in order to recall or introduce some basic operations on data. Variables were added to a copy of the dataframe:

```
NewData <- MktDATA.Orig # create a copy of the dataframe
```

First of all, the character variables `Marital_Status`, `Education` and `CustClass` were transformed into factors in order to

suitably reorganize their values (because the standard – alphabetical – order of their values is not satisfactory or adequate).

```
# Build factors to reorganize the variable's levels:
NewData$Marital_Status<-factor(MktDATA.Orig$Marital_Status,
                               levels = c("Single","Together","Married","Divorced","Widow"))
# reorder ordinal variables' levels:
NewData$Education <- factor(NewData$Education,
                             levels = c("HighSchool","College","Graduate","Post-Grad"))
NewData$CustClass <- factor(NewData$CustClass,
                             levels = c("Bronze","Silver","Gold","Platinum"))
```

The same was done for the variable Income, measured in classes. Indeed, tabulating the variable using the function `table()`:

```
table(NewData$Income)
>
> [100000,120000]    [2000,20000)    [20000,30000)    [30000,40000)    [40000,50000)
>                41                45                132                256                374
> [50000,60000)    [60000,70000)    [70000,80000)    [80000,100000)
>                320                310                325                399
```

it is clear that R does not recognize the numerical ordering of its levels. We therefore create a factor to re-order the variable's levels (even if this is not strictly necessary when the functions in package UBStats are used, see the next chapter), as well as a new, simplified, classification by collapsing some classes:

```
NewData$Income.F<-factor(NewData$Income,
                          levels = c("[2000,20000)", "[20000,30000)", "[30000,40000)",
                                       "[40000,50000)", "[50000,60000)", "[60000,70000)",
                                       "[70000,80000)", "[80000,100000)", "[100000,120000]"))
NewData$Income.S <- factor(NewData$Income,
                            levels = c("[2000,20000)", "[20000,30000)", "[30000,40000)",
                                         "[40000,50000)", "[50000,60000)", "[60000,70000)",
                                         "[70000,80000)", "[80000,100000)", "[100000,120000]"),
                            labels=c("[2000,40000)", "[2000,40000)", "[2000,40000)",
                                       "[40000,60000)", "[40000,60000)", "[60000,80000)",
                                       "[60000,80000)", "[80000,120000]", "[80000,120000]"))
```

Some additional numerical variables were also added to the dataframe: the total number of purchases in the last two years, `TotPurch`, the average value of purchases, `AOV`, and the share of products bought using promotions, `DealShare`:

```
NewData$TotPurch <- NewData$NPickUp_Purch + NewData$NWeb_Purch +
  NewData$NStore_Purch
NewData$AOV <- NewData$TotVal/NewData$TotPurch
NewData$DealShare <- NewData$NDeals/NewData$TotPurch
```

Finally, we created two logical variables signalling respectively whether the share of products purchased with discount is higher than or equal to 0.5, `Deals.ge50`, and whether the customer responded at least of one of the 5 campaigns launched *during* the observation period, `RespCampaign`:

```
NewData$Deals.ge50 <- NewData$DealShare >= 0.5
NewData$RespCampaign <- NewData$PastCampaigns > 0
```

This dataframe, renamed as `MktDATA`, is available in package UBStats and can be made available running:

```
data(MktDATA, package = "UBStats")
```

3 Descriptive statistics: distributions

In this chapter we illustrate how to organize and describe the empirical distribution of one or more variables using **tables** and **plots**. There are functions available in R at this aim. For example the function `table()` returns the frequency table of one (`table(x)`) or two (`table(x,y)`) variables. As for plots, again functions are available and in particular the package `ggplot2` is a powerful package to draw graphics. Nonetheless, we will refer to the functions available in package `UBStats`, developed for the course in order to simplify the syntax. Examples are based on the dataframe `MktDATA` available in the package. Thus, to start the package and the dataframe have to be loaded:

```
library(UBStats)
data("MktDATA")
```

3.1 Introduction to the functions

To analyze univariate and bivariate distributions using **tables**, the following functions are available in `UBStats`:

- `distr.table.x(x, <other args>, breaks, interval=FALSE, data)`
- `distr.table.xy(x, y, <other args>, breaks.x, breaks.y, interval.x=FALSE, interval.y=FALSE, data)`

The most relevant arguments of the functions are:

- **x**: identifies the variable to be analysed. *x* can be the name of a vector or a factor in the workspace *or* the name of one of the columns in the dataframe specified in *data*. The same holds for *y* when two variables are studied jointly.
- **data**: an optional data frame containing the variable to be analysed.
- **breaks**: can be used to classify the values taken by a **numeric** input variable into **intervals**. If *breaks* is an **integer**, the *x* range will be divided into intervals of equal width. If *breaks* is a **vector** of *increasing* values, it defines the endpoints of the required intervals (closed on the left and open on the right, except for the last interval, which is closed on the right). To cover the *entire range* of *x*, the minimum and the maximum values taken by *x* must be included between the first and the last break. However, it is possible to specify a set of breaks covering only a *portion* of the *x* range. When two variables are analyzed jointly, **breaks.x** and **breaks.y** can be used to classify *x* and/or *y* into intervals.
- **interval**: logical value indicating whether *x* is a variable **measured in classes**. If *interval* = `TRUE`, the functions try to identify the endpoints of the intervals; if the detected intervals are not consistent (e.g. overlapping intervals, or intervals with upper endpoint higher than the lower one), the variable is analysed as it is – if possible – even if results are not necessarily meaningful. When two variables are analyzed jointly, **interval.x** and **interval.y** can be used to indicate whether *x* or *y* are measured in classes.

The functions have other arguments, that we will typically leave to their default values:

- **total**: indicates whether the sum of the requested frequencies should be added to the tables (*total* = `TRUE`, default) or not.
- **f.digits**, **p.digits**, and **d.digits**: specify the number of decimals used to round proportions (default: *f.digits* = 2), percentages (default: *p.digits* = 0), or densities (only for univariate tables, default: *d.digits* = 5). Importantly, if the chosen rounding formats some non-zero values as zero, the number of decimals is increased so that all values have at least one significant digit, unless the argument *force.digits* (see below) is set to `TRUE`.
- **force.digits**: indicates whether proportions, percentages and densities should be forcedly rounded to the number of decimals specified in *f.digits*, *p.digits*, and *d.digits* even if non-zero values are rounded to zero (defaults to `FALSE`).
- **use.scientific**: indicates whether numbers in tables should be displayed using scientific notation (`TRUE`) or not (`FALSE`, default) typically used by R to represent very large or very small numbers (e.g. $\sim 1e+05$ or $2e-05$).
- **adj.breaks**: indicates whether the endpoints of the intervals of a numerical variable (*x* and/or *y*) *when classified into intervals* should be displayed avoiding scientific notation (`TRUE`, default).

To analyze univariate and bivariate distributions using **plots**, the following functions are available in `UBStats`:

- `distr.plot.x(x, <other args>, breaks, interval=FALSE, data)`
- `distr.plot.xy(x, y, <other args>, breaks.x, breaks.y, interval.x=FALSE, interval.y=FALSE, data)`

The functions **share** some of the arguments listed above for functions `distr.table.x()` and `distr.table.xy(x)`; in addition, it is worth mentioning some arguments that can be used to control coloring in the plots:

- **bw**: indicates whether a plot should be colored in scale of greys (*bw* = `TRUE`) or using a standard palette (*bw* = `FALSE`, default)

- **color**: optional string vector specifying colors to use in a plot (e.g. `color = "red"` or `color = c("red", "orange", "yellow", "green", "darkgreen")`) rather than a standard palette (NULL, default).

Importantly, note that the argument `use.scientific` in this case can be used to specify whether numbers on the plot's *axes* should be displayed using scientific notation (TRUE) or not (FALSE, default).

3.2 Univariate analysis

This section illustrates functions to tabulate and to plot the frequency distribution of one variable.

3.2.1 Frequency tables

The function `distr.table.x()` builds the frequency table of a vector or a factor: it lists the values taken by the variable, arranged in *standard order* (logical, alphabetical or numerical order for vectors or data classified into intervals, order of levels for factors) together with their **frequency** in data, namely **counts** (absolute frequencies), **proportions** (relative frequencies) or **percentages**, possibly **cumulated**. For variables classified into intervals it is possible to consider also the **densities**.

In the function (more details on the help page of the package):

- **x**: name of the vector (or factor) to be analysed *or* name of one column in the dataframe specified in `data`.
- **freq**: frequencies to be displayed (more options are possible). Allowed options are "counts", "percentages", "proportions", "densities" (only for variables classified into intervals), and "cumulative" (possibly abbreviated, e.g. "prop", "dens"). If no frequency is specified, counts and proportions are displayed by default; if only cumulative is specified, counts and proportions will be displayed, with their corresponding cumulative frequencies.
- **breaks**: can be used to classify a *numerical* variable `x` into *intervals* (as illustrated in sec. 3.1).
- **interval**: indicates (as illustrated in sec. 3.1) whether `x` is a variable measured in intervals (`interval=TRUE`).

For example, below is the table obtained for the factor Education in dataframe MktDATA using defaults.

```
# table for a factor / same syntax for qualitative variable
distr.table.x(Education, data = MktDATA)
> Education Count Prop
> HighSchool 213 0.10
> College 415 0.19
> Graduate 1124 0.51
> Post-Grad 470 0.21
> TOTAL 2222 1.00
# same as:
# distr.table.x(MktDATA$Education)
```

The cumulative frequencies can be obtained for all the types of frequencies specified in `freq`. For the discrete variable Children, different types of frequencies are obtained and cumulated:

```
# note that the names of the requested frequencies can be abbreviated
distr.table.x(Children, data = MktDATA, freq = c("count", "prop", "cum"), f.digits = 4)
> Children Count Prop Cum.Count Cum.Prop
> 0 631 0.2837 631 0.2837
> 1 1121 0.5040 1752 0.7878
> 2 419 0.1884 2171 0.9762
> 3 53 0.0238 2224 1.0000
> TOTAL 2224 1.0000
```

3.2.1.1 Numerical variables classified into intervals The values taken by a numerical variable can be classified into intervals by specifying **breaks** indicating the endpoints of the desired intervals.

Below are examples for the numerical continuous variable AOV, classified respectively into intervals with the same width and into intervals with different widths.

```
# 5 intervals with the same width
distr.table.x(AOV, breaks = 6, freq = c("Count", "Perc", "Cum"), p.digits = 2, data = MktDATA)
> AOV Count Percent Cum.Count Cum.Percent
> [2,31) 1280 57.55 1280 57.55
```

```
> [31,60) 458 20.59 1738 78.15
> [60,89) 347 15.60 2085 93.75
> [89,118) 113 5.08 2198 98.83
> [118,147) 22 0.99 2220 99.82
> [147,176] 4 0.18 2224 100.00
> TOTAL 2224 100.00

# intervals with different width / also densities required
distr.table.x(AOV, breaks = c(0,20,30,50,100,180),
              freq = c("Count","Perc","Cum","Densities"), p.digits = 2, data = MktDATA)
>      AOV Count Percent Density Cum.Count Cum.Percent
> [0,20) 881 39.61 0.01981 881 39.61
> [20,30) 363 16.32 0.01632 1244 55.94
> [30,50) 440 19.78 0.00989 1684 75.72
> [50,100) 439 19.74 0.00395 2123 95.46
> [100,180] 101 4.54 0.00057 2224 100.00
> TOTAL 2224 100.00
```

As mentioned above, the table can also be built limiting attention to intervals not covering the entire range of values:

```
distr.table.x(AOV, breaks = c(0,20,40,80,100,150),
              freq = c("Count","Perc","Cum","Dens"), p.digits = 2, data = MktDATA)
> Warning:
> Some values of 'x' not counted!
> -> Selected 'breaks' might not span 'x's range
> -> Be sure this is actually what you want
>
>      AOV Count Percent Density Cum.Count Cum.Percent
> [0,20) 881 39.68 0.01984 881 39.68
> [20,40) 703 31.67 0.01583 1584 71.35
> [40,80) 489 22.03 0.00551 2073 93.38
> [80,100) 50 2.25 0.00113 2123 95.63
> [100,150] 97 4.37 0.00087 2220 100.00
> TOTAL 2220 100.00
```

In this case the function prints a warning. [Note: in the following this warning message will not be displayed to save space]

3.2.1.2 Numerical data measured in classes The values taken by a numerical variable measured in classes are ordered categories which nonetheless are not recognized as numerical by R. To properly arrange the intervals, one could build a proper factor, as seen before. Nonetheless, this can be directly accommodated by the function, by specifying that *x* is measured in intervals, setting *interval=TRUE*:

```
distr.table.x(Income.S, interval = TRUE, freq = c("Prop","Perc","Cum","Dens"), data = MktDATA)
>
> Intervals endpoints detected and used for 'Income.S'
>
>      Income.S Prop Percent Density Cum.Prop Cum.Percent
> [2000,40000) 0.20 20 0.000005 0.20 20
> [40000,60000) 0.32 32 0.000016 0.51 51
> [60000,80000) 0.29 29 0.000014 0.80 80
> [80000,120000] 0.20 20 0.000005 1.00 100
> TOTAL 1.00 100
```

Observe the message signalling that consistent endpoints were detected [Note: in the following this message will be not be displayed to save space]

3.2.2 Plots of univariate distributions

The function `distr.plot.x()` creates different plots for frequency tables.

In the function (more details on the help page of the package):

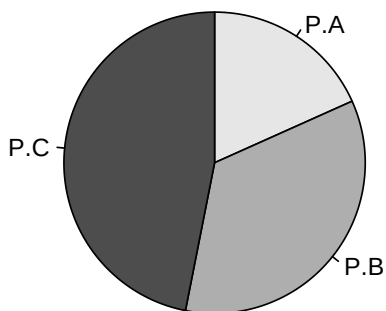
- **x**: name of the vector (or factor) to be analysed *or* name of one column in the dataframe specified in `data`.
- **freq**: **one** type of frequencies to use in the plot. Allowed options are "counts" (default), "percentages", "proportions", and "densities" (for histograms and density plots), possibly abbreviated (e.g. "prop", "dens").
- **plot.type**: **one** of the following plots "pie", "bars", "spike", "histogram", "density", "boxplot", and "cumulative" (possibly abbreviated, e.g. "hist", "cum").
- **ord.freq**: for pie and bars plots, the levels of `x` can be displayed in standard order (`ord.freq="none"`, default) or by increasing or decreasing order of their frequencies (`ord.freq="increasing"` and `ord.freq="decreasing"` respectively).
- **breaks**: can be used to classify a *numerical* variable `x` into *intervals* (as illustrated in sec. 3.1).
- **interval**: indicates (as illustrated in sec. 3.1) whether `x` is a variable measured in intervals (`interval=TRUE`).

3.2.2.1 Pie, bar, and spike charts. Pie, bar and spike charts display the frequency (as specified in argument `freq`) table of a variable of **any** type, whatever the number of distinct values it takes.

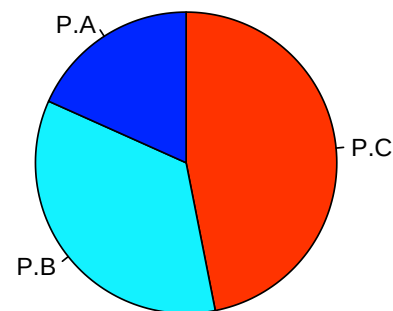
Below are the **pie** charts for the character variable `LikeMost`, built both in black/white (on the left) and (on the right) by coloring the slices and ordering the variable's levels based on their frequency (decreasing).

```
distr.plot.x(x = LikeMost, plot.type = "pie", bw = TRUE, data = MktDATA)
distr.plot.x(x = LikeMost, plot.type = "pie", ord.freq = "dec", data = MktDATA)
```

Pie chart: LikeMost



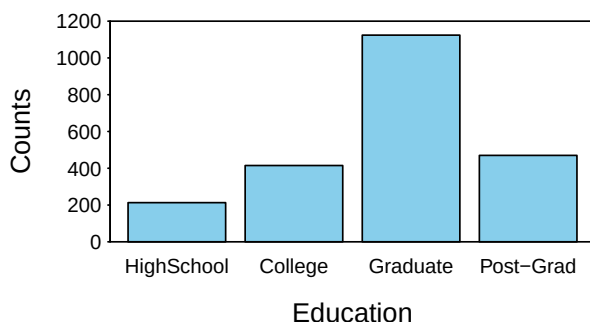
Pie chart: LikeMost



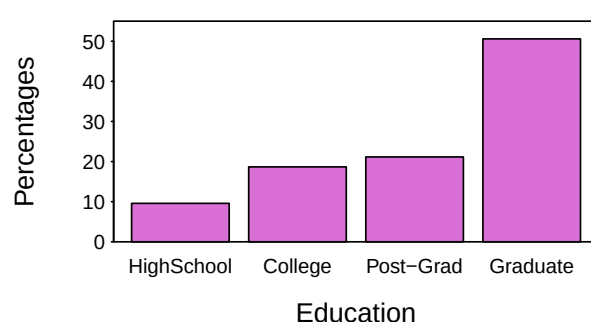
Below are two **bars** plots for the factor `Education`. One plot (on the left) reports the counts (default) of the factor's levels displayed in standard order; the other (on the right) reports the percentages of the levels arranged in an increasing order.

```
distr.plot.x(x = Education, plot.type = "bars", data = MktDATA)
distr.plot.x(x = Education, plot.type = "bars", freq = "percentage", ord.freq = "inc",
  color = "orchid", data = MktDATA)
```

Bar plot: Education

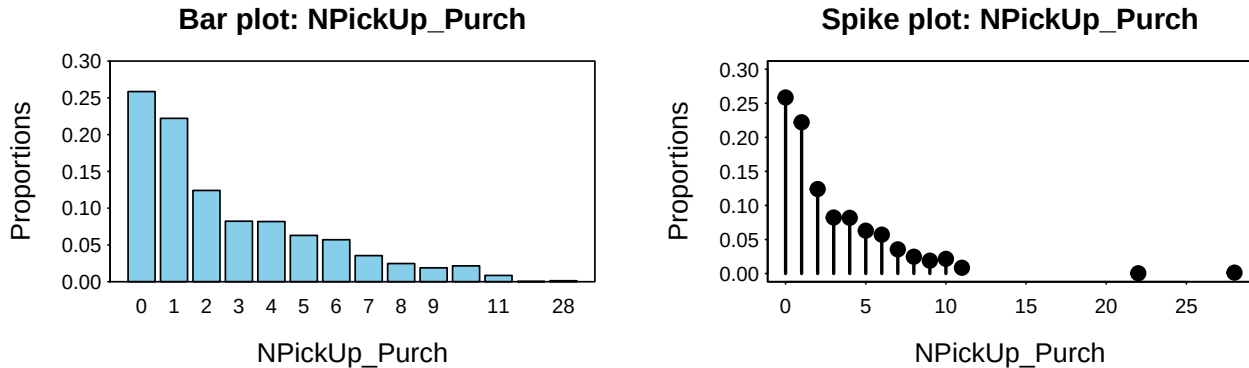


Bar plot: Education



A **spike** plot can also be used to plot a frequency table. Below a bar and a spike plot are built for the numerical discrete variable NPickUp_Purch.

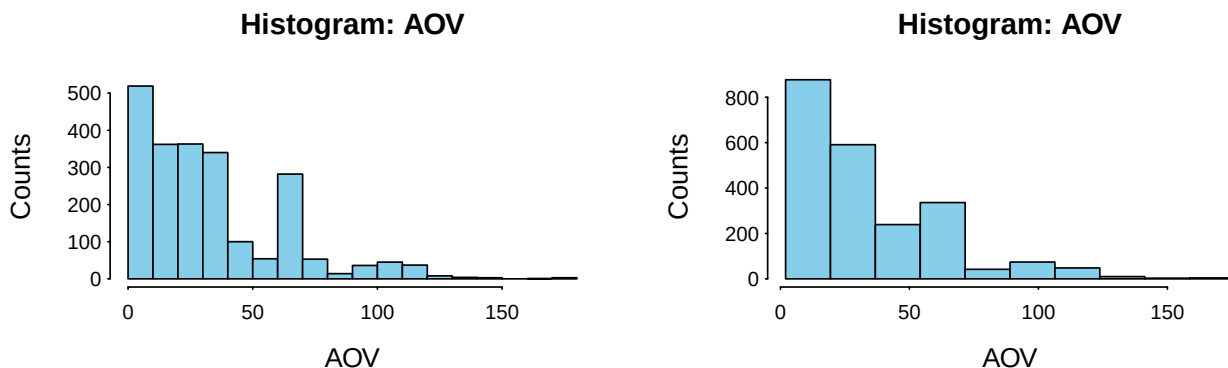
```
distr.plot.x(x = NPickUp_Purch, plot.type = "bars", freq = "prop", data = MktDATA)
distr.plot.x(x = NPickUp_Purch, plot.type = "spike", freq = "prop", data = MktDATA)
```

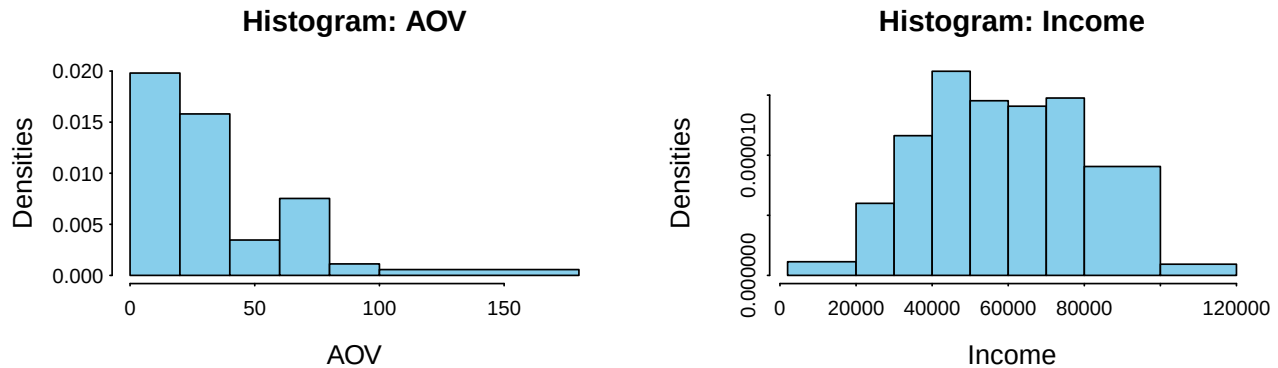


3.2.2.2 Histograms. Histograms display the distribution of numerical data classified into intervals. When the input is **numeric** and breaks are not provided, the histogram is built using a routine in R. Otherwise, the indicated breaks are used – possibly covering only a portion of the range – only if they are consistent (i.e. increasing). Histograms can also be built for variables **measured in classes** – provided that the argument `interval` is set to `TRUE` – only if the detected endpoints of the classes are consistent. When the classes have the same width, the histogram can report different types of frequencies (counts, percentages, proportions), as well as densities. Instead, when classes have different widths, densities are reported, irrespective of the type of frequency indicated in `freq`.

The two plots below report histograms for the numerical continuous variable AOV built without specifying breaks (top-left), using 10 classes with the same width (top-right), and specifying intervals (bottom-left). In the last plot (bottom-right) the histogram for the variable Income, measured in classes, is displayed.

```
# Histogram - no breaks selected
distr.plot.x(x = AOV, plot.type = "histogram", data = MktDATA)
# Histogram - 10 intervals with the same width
distr.plot.x(x = AOV, plot.type = "histogram", breaks = 10, data = MktDATA)
# Histogram - with breaks
distr.plot.x(AOV, plot.type = "histogram", breaks = c(0,20,40,60,80,100,180), data = MktDATA)
distr.plot.x(Income, plot.type = "histogram", interval = TRUE, data = MktDATA)
```





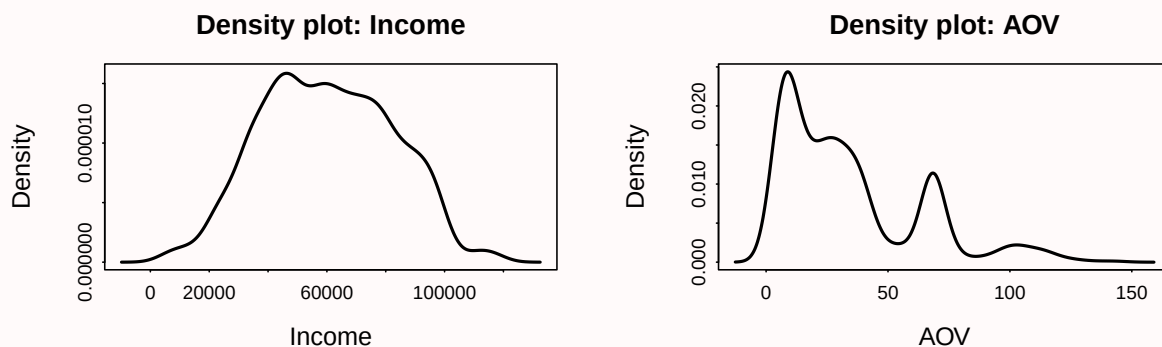
Observe that in the plots only some labels are reported on the horizontal axis. To visualize all the labels it is possible to *zoom* the plot (button **Zoom** in the **Plots** window of the bottom-right multi-scope panel) or to enlarge manually the **Plots** window. Observe further that even if `freq` is left to its default value (counts), when – due to decimals and rounding – the widths of the interval differ even at a lower extent, densities are reported in the plots, since intervals have different lengths.

Note that if **one variable measured in classes has inconsistent endpoints its histogram will not be obtained**.

Additional Material

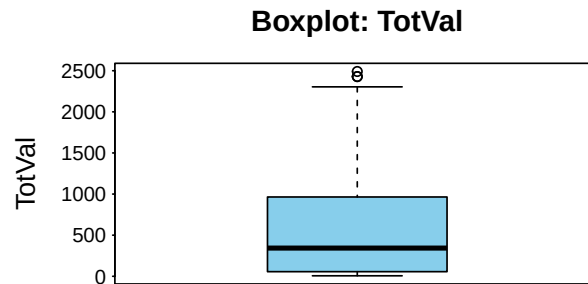
3.2.2.3 Density Plots. Even if not explicitly treated in the course, we describe how to obtain density plots for numerical data. These plots are built by properly smoothing a histogram based on a very fine classification into intervals. Thus, for this type of plot, the possibly specified breaks are **ignored**, and are used only to define the range of the considered values. For variables measured in classes, the plot is an approximation, based on data simulated by assuming uniform distribution of values within the classes. In this case, densities are reported on the vertical axis irrespective of the value specified for `freq`.

```
# A variable measured in classes
distr.plot.x(Income, plot.type = "density", interval = TRUE, data = MktDATA)
# A numerical variable, with breaks specified / ignored
# but used to limit the range of considered values
distr.plot.x(x = AOV, plot.type = "density",
             breaks = c(0,20,40,60,80,100,150), data = MktDATA)
```



3.2.2.4 Boxplots. Boxplots can be obtained **only** for numerical variables:

```
distr.plot.x(x = TotVal, plot.type = "boxplot", data = MktDATA)
```

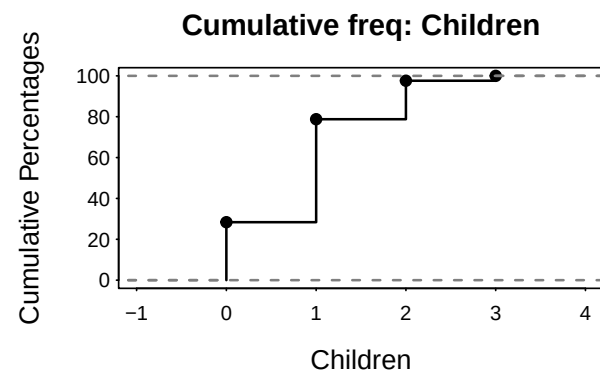
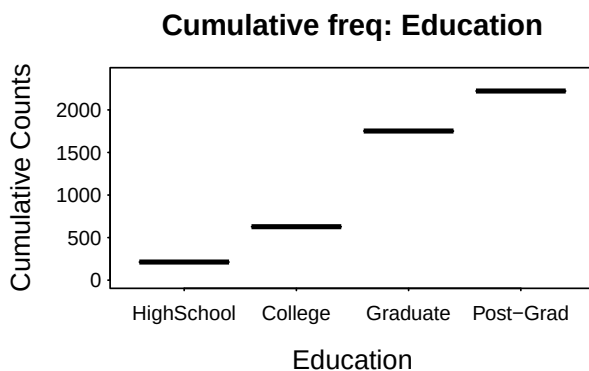


Note: an **error message** is returned when x is a character vector or factor, and also when breaks are specified or when variables measured in classes (`interval=TRUE`) are considered.

3.2.2.5 Plots of cumulative distributions. Cumulative distributions can be obtained, and therefore plotted, both for ordinal and for numerical data. Therefore, the function `distr.plot.x()` builds cumulative plots irrespective of the type of variable (logical, character or numerical vector, factor). The function builds a **stair steps** plot, unless x is a numerical variable classified in intervals using breaks or a variable measured in classes (this should be specified by setting `interval = TRUE`). Steps corresponding to the observed values have heights proportional to the values' frequencies (note that if `freq = densities` the function will return an error, because only frequencies can be cumulated).

Below plots of cumulative counts and percentages respectively are reported for the factor Education and for the discrete numerical variable Children.

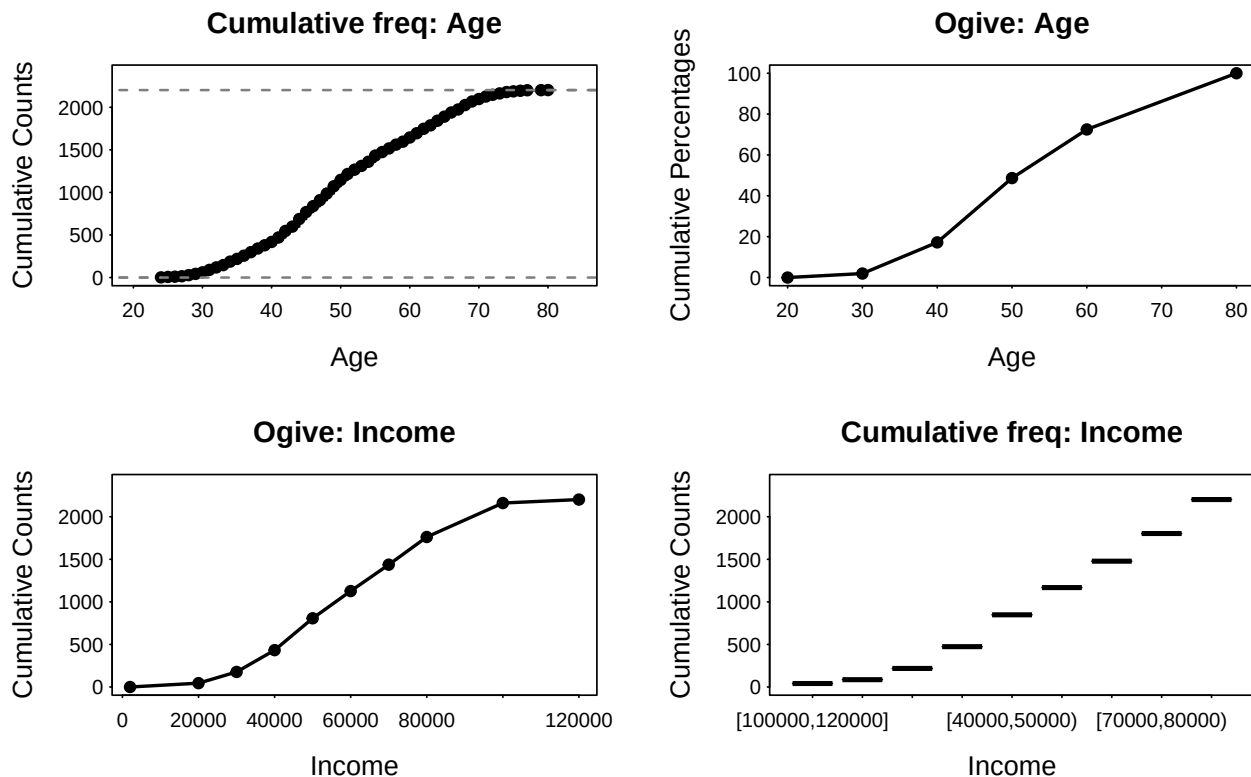
```
distr.plot.x(x = Education, plot.type = "cum", data = MktDATA)
distr.plot.x(x = Children, plot.type = "cum", freq = "percent", data = MktDATA)
```



Note the difference between the plots depending on whether the input is a character variable/factor (whose levels have no numerical meaning) or a numeric variable.

For numeric variables taking many values, the cumulative plot can be built based on raw data. If *breaks* are specified, an **ogive** is displayed, connecting the frequencies cumulated at the intervals' endpoints. For variables **measured in classes** (when `interval = TRUE` is specified), only the ogive can be built, because no information is available on the specific values observed within each class. Note that if `interval = TRUE` is not specified, the function will treat the variable as a character/factor and will return a stair steps plot.

```
# A continuous variable with no specified breaks
distr.plot.x(Age,plot.type="cum", data = MktDATA)
# A continuous variable with specified breaks
distr.plot.x(Age,plot.type="cum",freq="perc",breaks=c(20,30,40,50,60,80), data = MktDATA)
# A variable measured in classes (interval=T)
distr.plot.x(Income,plot.type="cum",interval=TRUE, data = MktDATA)
# A variable measured in classes / no interval indicated
distr.plot.x(Income,plot.type="cum", data = MktDATA)
```



3.3 Bivariate analysis

This section illustrates how to tabulate and plot the joint and conditional distributions of two variables.

3.3.1 Cross-tabs: Joint and conditional distributions

The function `distr.table.xy()` builds contingency tables of the joint or conditional frequencies for each combination of the values taken by two variables.

In the function (more details on the help page of the package):

- **x,y:** names of the vectors (or factors) to be analysed **or** names of the columns in the dataframe specified in `data`; it is possible to use a mixed specification (e.g. one vector and one column in `data`). Note that in the table `x` is displayed on the *rows* and `y` on the *columns*.
- **freq:** frequencies to be displayed (more options are possible). Allowed options are "counts" (default), "percentages", and "proportions" (possibly abbreviated (e.g. "prop", "perc")).
- **freq.type:** type of frequencies to be displayed (more options are possible). Allowed options are "joint" frequencies (default), "`x|y`" (or "column") – the distributions of `x` conditioned to `y`, and "`y|x`" (or "row") – the distributions of `y` conditioned to `x`.
- **breaks.x, breaks.y:** can be used to classify variables `x` and/or `y` – if *numerical* – into *intervals* (as illustrated in sec. 3.1).
- **interval.x, interval.y:** indicate (as illustrated in sec. 3.1) whether `x` and/or `y` are measured in intervals (e.g., `interval.x = TRUE`).

Some examples are reported below.

```
# Character vectors, factors, and discrete numeric vectors
distr.table.xy(x= LikeMost, y = Children, data = MktDATA) # default: joint counts
> Joint counts
>      Children
> LikeMost   0   1   2   3 TOTAL
>   P.A    122 222  55   8  407
>   P.B    280 360 114  20  774
>   P.C    229 539 250  25 1043
>   TOTAL   631 1121 419  53 2224

# Conditional distribution (proportions) of x|y (same if freq.type = "col")
distr.table.xy(x = LikeMost, y = Education, freq = "Prop", freq.type = "x|y", data = MktDATA)
> x|y: Proportions
>      Education
> LikeMost HighSchool College Graduate Post-Grad
>   P.A      0.28    0.22    0.17    0.14
>   P.B      0.31    0.34    0.39    0.26
>   P.C      0.40    0.44    0.44    0.60
>   TOTAL      1.00    1.00    1.00    1.00

# Conditional distribution (%) of y/x (columns-conditionals)
distr.table.xy(x = CustClass, y = Children, freq = "perc", freq.type = "row", data = MktDATA)
> Row Percentages
>      Children
> CustClass   0   1   2   3 TOTAL
>   Bronze    13  58  26   3  100
>   Silver    44  49   6   1  100
>   Gold     68  29   2   1  100
>   Platinum  83  15   1   1  100

# A numerical variable classified into intervals and a factor
distr.table.xy(x = CustClass, y = TotPurch, breaks.y = c(0,5,10,15,20,35),
               freq = c("Prop"), freq.type = "y|x", data = MktDATA)
> y|x: Proportions
>      TotPurch
> CustClass [0,5) [5,10) [10,15) [15,20) [20,35] TOTAL
>   Bronze  0.223 0.395  0.189  0.141  0.052 1.000
>   Silver  0.000 0.000  0.167  0.278  0.556 1.000
>   Gold    0.000 0.004  0.065  0.371  0.560 1.000
>   Platinum 0.000 0.000  0.095  0.435  0.470 1.000

# Two numerical variables classified into intervals
distr.table.xy(x = Income.S, y = TotPurch, interval.x = TRUE, breaks.y = c(0,5,10,15,20,35),
               freq = c("Counts"), freq.type = "row", data = MktDATA)
> Row Counts
>      TotPurch
> Income.S  [0,5) [5,10) [10,15) [15,20) [20,35] TOTAL
> [2000,40000) 167  251   11    3    1  433
> [40000,60000) 151  290  167   62   24  694
> [60000,80000) 10   44  144  214  223  635
> [80000,120000] 2    1   43  177  217  440
```

3.3.2 Plots of joint and conditional distributions

The function `distr.plot.xy()` creates different plots for bivariate distributions. Following the standard conventions used in plots, the variable `x` is reported on the horizontal axis.

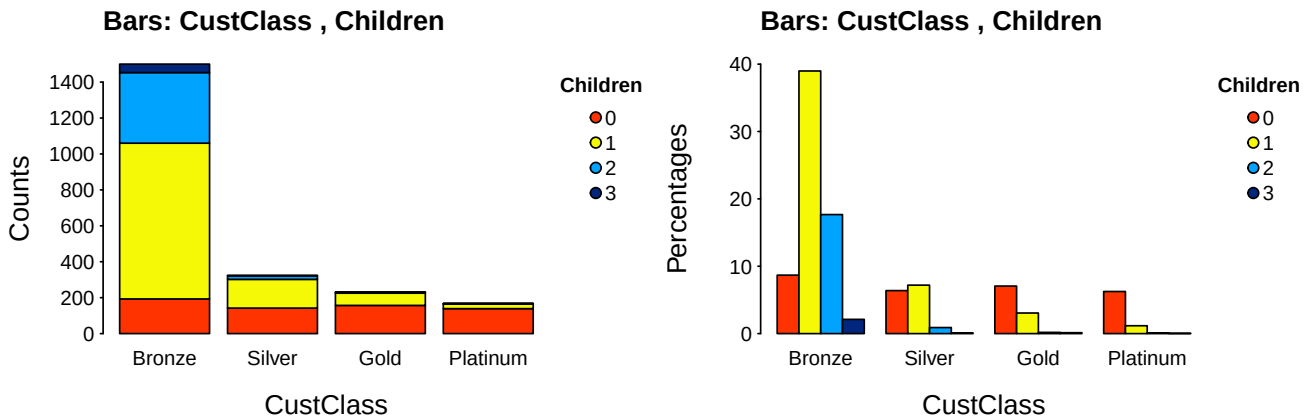
In the function (more details on the help page of the package):

- `x,y`: names of the vectors (or factors) to analyse **or** names of the columns in the dataframe specified in `data`; a mixed specification is possible (e.g, one vector and one column in `data`). Note: `x` is displayed on the *horizontal* and `y` on the *vertical* axes.
- `freq`: frequencies to be displayed (only **one** option is possible). Allowed options are "counts" (default), "percentages", "proportions" (possibly abbreviated, e.g. "prop", "perc").
- `freq.type`: type of frequencies to be displayed (only **one** option is possible). Allowed types are "joint" (default) for joint frequencies and "`x|y`" and "`y|x`" for conditional frequencies.
- `plot.type`: **one** of the following plots: "bars", "scatter", "boxplot".
- `bar.type`: indicates whether in a bar plot, stacked (`bar.type="stacked"`, default) or side-by-side (`bar.type="beside"`) bars should be displayed.
- `breaks.x, breaks.y`: allow classifying `x` and/or `y` into intervals (as illustrated in sec. 3.1).
- `interval.x, interval.y`: indicate whether `x` and/or `y` are variables measured in intervals (as illustrated in sec. 3.1).
- `fitline`: indicates whether the line of best fit (also called trendline or regression line) should be added to a **scatterplot** (`fitline = TRUE`) or not (`fitline = FALSE`, default).
- `legend`: indicates whether a legend should be displayed (`legend = TRUE`, default) or not (`legend = FALSE`).

Note: further arguments specific for **scatterplots** are illustrated in the dedicated section.

3.3.2.1 Bivariate bar plots. Below, the cross-tab of the variables `CustClass` (a factor) and `Children` (discrete numeric) are displayed using respectively stacked bars representing counts and side-by-side bars representing percentages.

```
distr.plot.xy(x = CustClass, y = Children, freq = "Counts", freq.type = "joint",
  plot.type = "bars", data = MktDATA)
distr.plot.xy(x = CustClass, y = Children, freq = "Percentages", freq.type = "joint",
  plot.type = "bars", bar.type = "beside", data = MktDATA)
```

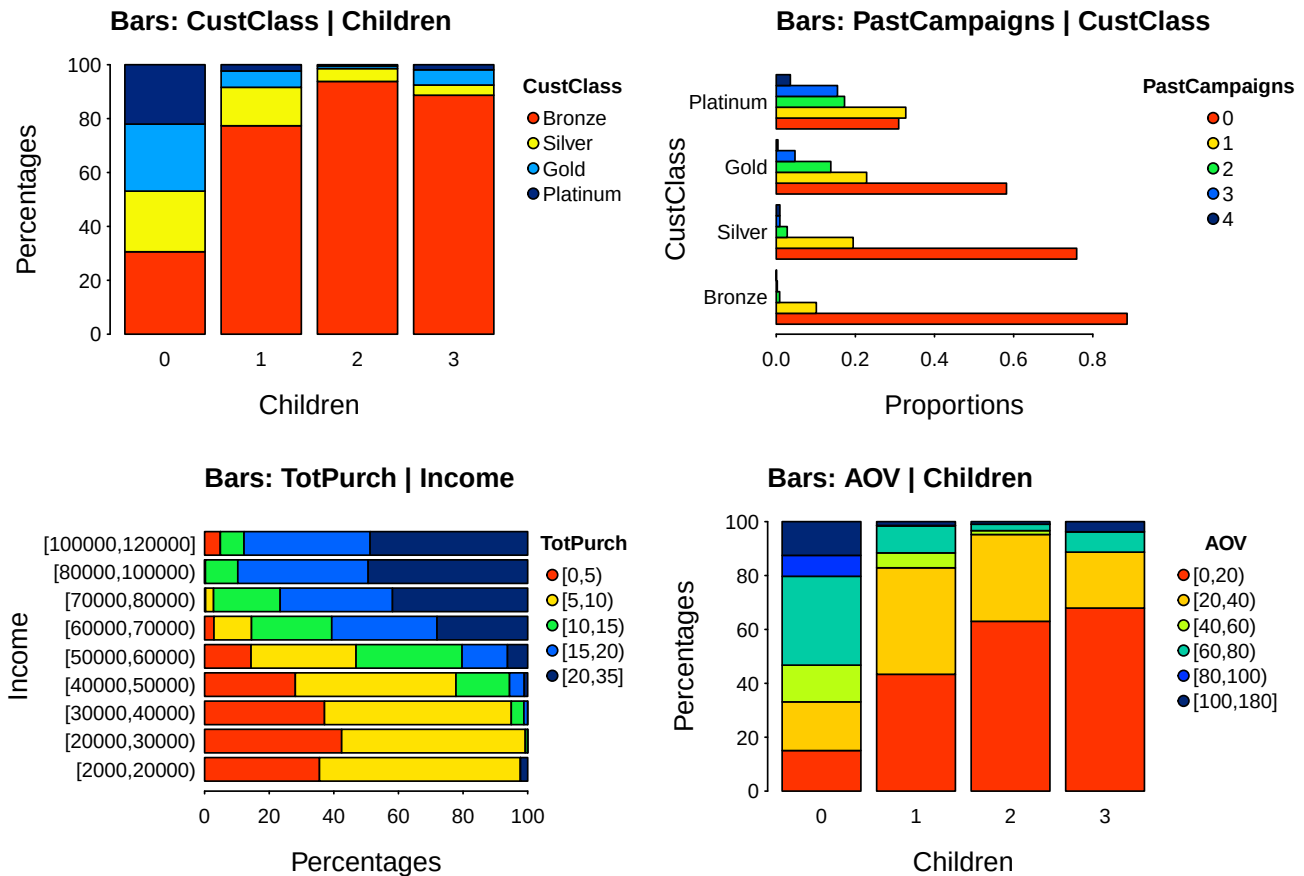


Note that in the tables (see previous section) and in the plots of the cross-tables, the arrangements of the variables and of their values differ. Indeed, in the table `x`'s values are placed on the rows and `y`'s values on the columns. In addition, the `y`'s values were arranged on columns in an increasing order, whereas in the plot the categories are displayed from bottom to top.

When conditional distributions are requested, only proportions or percentages can be displayed. Note that since `x` is on the horizontal axis, when the distributions of `x|y` are requested, the bars conditioned to the values of `y` are displayed horizontally.

```
distr.plot.xy(x = Children, y = CustClass, freq = "percent", freq.type = "y|x",
  plot.type = "bars", data = MktDATA)
distr.plot.xy(x = PastCampaigns, y = CustClass, freq = "prop", freq.type = "x|y",
  plot.type = "bars", bar.type = "beside", data = MktDATA)
```

```
distr.plot.xy(x = TotPurch, y = Income, freq = "perc", freq.type = "x|y", plot.type = "bars",
  breaks.x = c(0,5,10,15,20,35), interval.y = TRUE, data = MktDATA)
distr.plot.xy(x = Children, y = AOV, freq = "percent", freq.type = "y|x", plot.type = "bars",
  breaks.y = c(0,20,40,60,80,100,180), data = MktDATA)
```

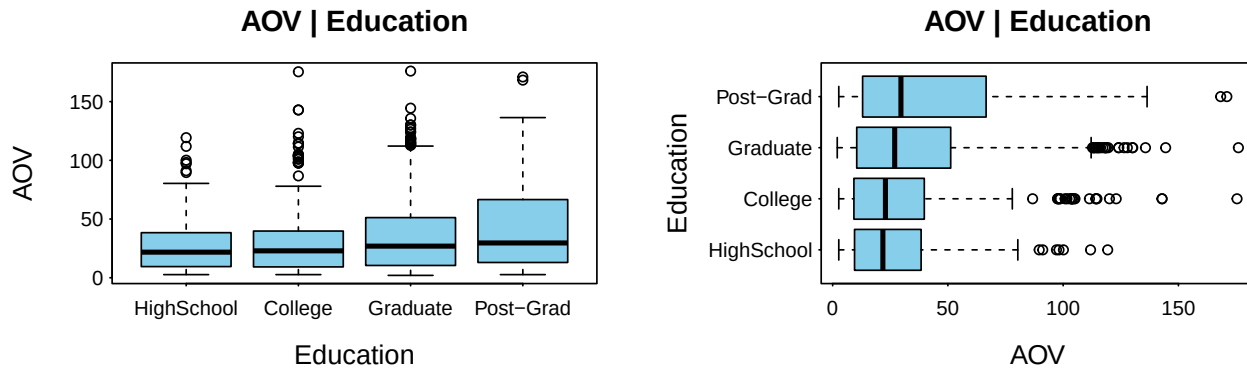


Note: bar plots can be built also when one or both the variables are numeric. Nonetheless, the function `distr.plot.xy` returns an error when at least one of the numeric variables takes **more than 20 values**. There are instead no restrictions on the number of values/levels taken by character variables or factors.

3.3.2.2 Side-by-side boxplots. Side-by-side boxplots allow to compare the distribution of a numeric variable (typically but not necessarily continuous) **conditioned** to the levels of a variable taking a limited number of values.

In these plots, `x` is placed on the horizontal axis and `y` variable on the vertical one. In addition, if one of the variables is categorical or is a factor, the boxplots are built conditioned to that variable **irrespective** of the chosen `freq.type`. For example, below are the plots for the numeric variable AOV and for factor Education: the boxplots are obtained conditioned to the factor even when the distributions of Education conditioned to AOV are requested. The difference between the two plots lies in the arrangement (vertical or horizontal) of the boxplots, depending on whether Education is the `x`- or the `y`-variable.

```
distr.plot.xy(x = Education, y = AOV, plot.type = "boxplot", freq.type = "x|y", data = MktDATA)
distr.plot.xy(x = AOV, y = Education, plot.type = "boxplot", freq.type = "y|x", data = MktDATA)
```



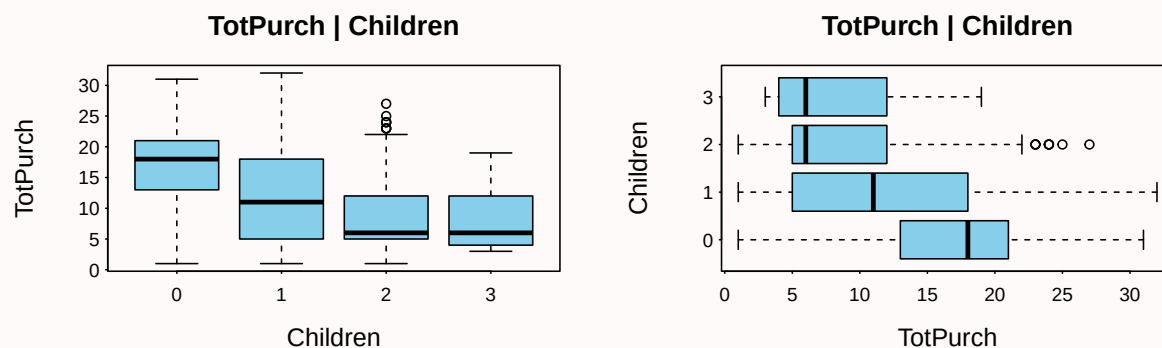
When none of the variables is numeric the function returns an error. Note that, as seen before, boxplots cannot be built for numeric variables classified into intervals.

Additional Material

When side-by-side boxplots are requested for **two numerical variables**, the type of frequency indicated in `freq.type` is relevant, because it is not possible to deduce from the variables' characteristics what is the desired/correct conditioning variable. If `freq.type` is not specified, the function `distr.plot.xy` builds the boxplots conditioned to `x` (it is advisable to clarify which conditional distribution one is interested in, by setting the proper `freq.type`).

```
# plots conditioned to the x variable because no freq.type is specified
distr.plot.xy(x = Children, y = TotPurch, plot.type = "boxplot", data = MktDATA)
# same as:
# distr.plot.xy(x = Children, y = TotPurch, plot.type = "boxplot",
#               freq.type = "y/x", data = MktDATA)

# plots conditioned to the y variable
distr.plot.xy(x = TotPurch, y = Children, plot.type = "boxplot",
              freq.type = "x/y", data = MktDATA)
```

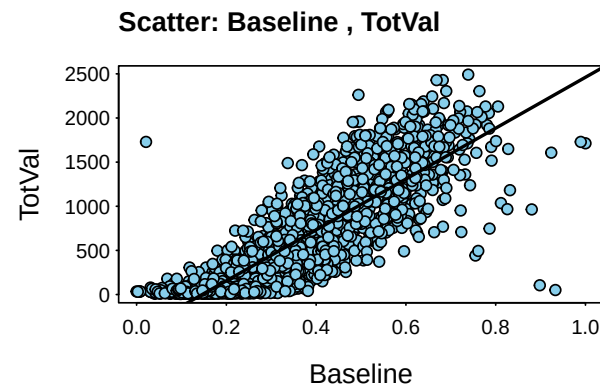
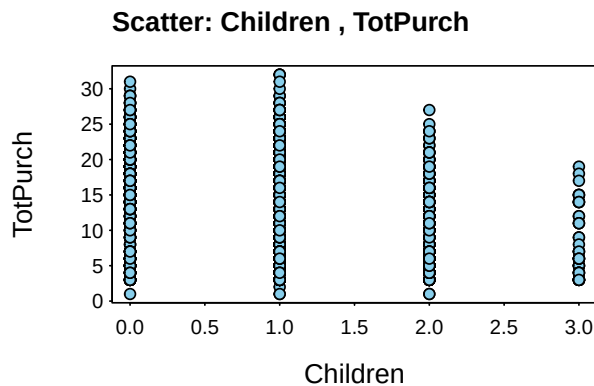


3.3.2.3 Scatterplots. Scatterplots are typically used to display the joint distribution of two numeric variables (see the next subsection for scatterplots built for variables of different types). In the plots a dot is reported corresponding to each observed combination of values: the arguments `freq` and `freq.type` are not considered in this case.

The argument `var.c`, specific for these plots, allows coloring scatterplots based on a third variable of any type; the arguments `breaks.c` and `interval.c=TRUE`, can be used to specify if a numeric coloring variable should be classified into intervals and if it is measured in intervals.

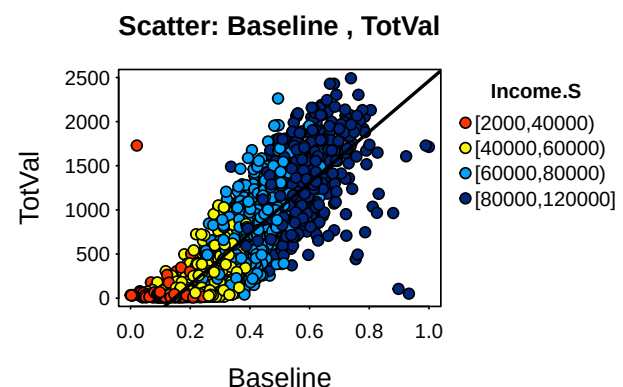
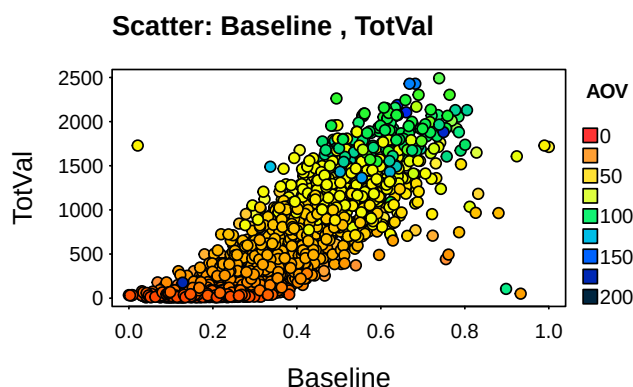
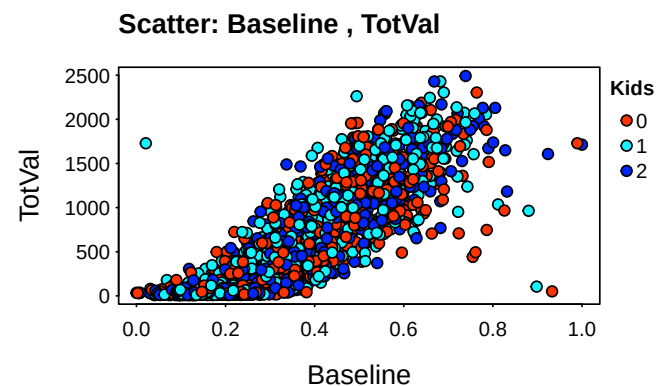
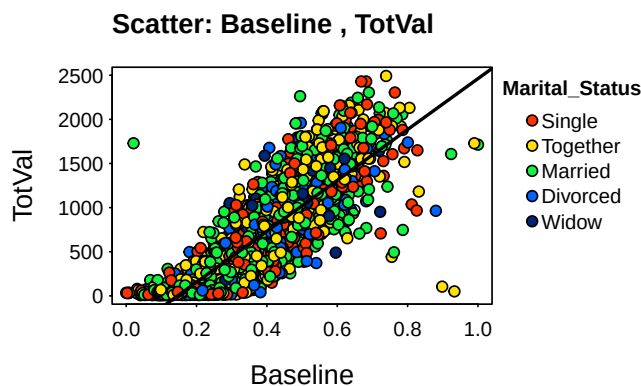
Below are the scatterplots for two pairs of numeric variables, with a **fitline** (also called trendline) added to the right-plot.


```
distr.plot.xy(x = Children, y = TotPurch, plot.type = "scatter", data = MktDATA)
distr.plot.xy(x = Baseline, y = TotVal, plot.type = "scatter", fitline = TRUE, data = MktDATA)
```



Below are plots colored based on a factor, a discrete numeric variable, a numeric continuous variable, and a variable measured in classes:

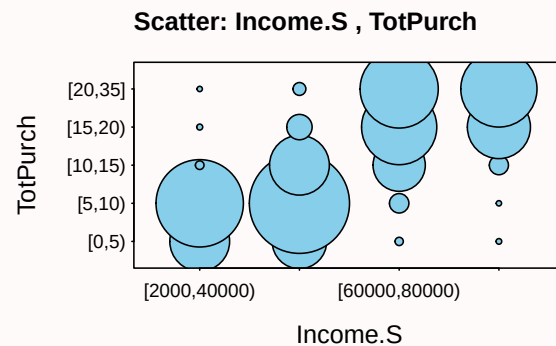
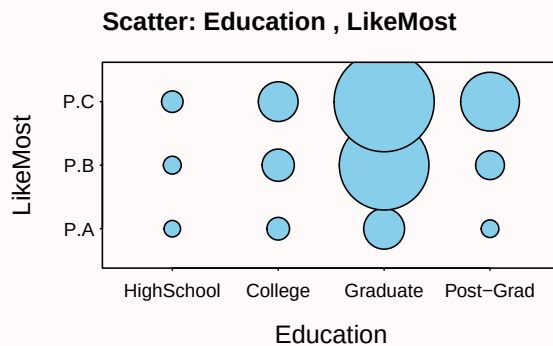
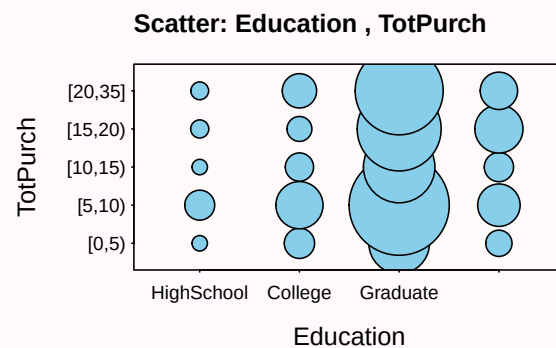
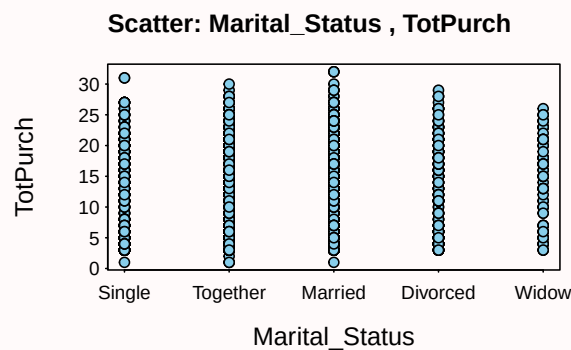
```
distr.plot.xy(x = Baseline, y = TotVal, plot.type = "scatter", var.c = Marital_Status,
              fitline=TRUE, data = MktDATA)
distr.plot.xy(x = Baseline, y = TotVal, plot.type = "scatter", var.c = Kids, data = MktDATA)
distr.plot.xy(x = Baseline, y = TotVal, plot.type = "scatter", var.c = AOV, data = MktDATA)
distr.plot.xy(x = Baseline, y = TotVal, plot.type = "scatter", var.c = Income.S,
              interval.c = TRUE, fitline = TRUE, data = MktDATA)
```



Additional Material

Scatterplots for non-numeric data. Scatterplots can be built also when **one** or **both** the variables are **not** numeric. In the first case, a standard scatterplot is built; when both the variables are categorical or factors – a **bubble** plot is built, with the size of the dots proportional to the joint frequencies of the observed pairs of values. Note that in this case `var.c` would be ineffective because each bubble represents more cases; same holds for `fitline` because at least one variable is not numeric.

```
# one variable only is numerical
distr.plot.xy(x = Marital_Status, y = TotPurch, plot.type = "scatter", data = MktDATA)
# none of the variables is numerical
distr.plot.xy(x = Education, y = TotPurch, breaks.y = c(0,5,10,15,20,35),
              plot.type = "scatter", data = MktDATA)
# two characters/factors
distr.plot.xy(x = Education, y = LikeMost, plot.type = "scatter", data = MktDATA)
# classified variables (not properly numerical)
distr.plot.xy(x = Income.S, y = TotPurch, interval.x = TRUE, breaks.y = c(0,5,10,15,20,35),
              plot.type = "scatter", data = MktDATA)
```



4 Descriptive statistics: Summaries

In this section we first describe how to summarize data on one single variable using central or non central tendency measures and measures of dispersion. Next we focus on measures to assess the extent of linear association between two variables.

4.1 Summaries of a single variable

Basic summaries can be obtained with simple standard functions in R. For example, the functions `mean`, `sd`, `var`, `quantiles` return respectively the mean, the standard deviation, the variance, and the quantiles of a single variable, as shown below. In the case of missing values in data, these functions return a missing value (NA), unless the option `na.rm = TRUE` is used to exclude missing values from computations.

```
mean(MktDATA$Age) # returns NA since some cases have missing values
> [1] NA
mean(MktDATA$Age, na.rm = TRUE)
> [1] 51.0772
sd(MktDATA$Age, na.rm = TRUE)
> [1] 11.71164
var(MktDATA$Age, na.rm = TRUE)
> [1] 137.1626
# by default quantile() returns the five numbers summary
quantile(MktDATA$Age, na.rm = TRUE)
> 0% 25% 50% 75% 100%
> 24 43 50 61 80
# other quantiles can be obtained using argument probs:
quantile(MktDATA$Age, probs = c(0.05,0.1,0.25,0.5,0.75,0.9,0.99), na.rm = TRUE)
> 5% 10% 25% 50% 75% 90% 99%
> 32.00 36.00 43.00 50.00 61.00 68.00 74.99
```

Also the sample proportion can be easily obtained, by preliminarily defining a binary or logical vector whose values indicate whether a certain condition is satisfied. For example:

```
sum(MktDATA$AOV > 50) # counts: clients with AOV > 50
mean(MktDATA$AOV > 50) # proportion

mean(MktDATA$Age >= 40) # proportion of clients aged 40 or more: missing value!!
mean(MktDATA$Age >= 40, na.rm = TRUE)

# same procedure directly applied to binary or logical vectors:
mean(MktDATA$LastCampaign) # binary variable: proportion
mean(MktDATA$RespCampaign) # logical variable: proportion
# the variance:
var(MktDATA$RespCampaign) # p(1-p), p=proportion
```

The function `distr.summary.x()` allows to obtain a collection of summaries, possibly *conditioned* to the levels of one or two variables; it automatically excludes missing values from computations. The function has the following arguments (more details on the help page of the package):

- **x**: name of the vector to be analysed **or** name of one column in the dataframe specified in `data`;
- **stats**: one or more summaries. A number of options are available. The following options return a set of statistics: "summary" (returns min, q1, median, mean, q3, max, sd, var), "central" (tendency measures), "dispersion", "fivenumbers" (five number summary), "quartiles", "quintiles", "deciles", "percentiles". It is possible to require a specific set of statistics: "q1", "q2", "q3", "mean", "median", "mode" (returns the mode, the number of modes and the proportion of cases with modal value), "min", "max", "sd", "var", "cv" (coefficient of variation), "range", "IQR", or "p1", "p2", ..., "p100" (specific percentiles)
- **by1, by2**: optional vectors or factors (typically taking few values/levels) used to build conditional summaries, that can be defined same way as **x**.
- **breaks.by1, breaks.by2**: allow classifying **by1** and/or **by2** into intervals (as illustrated in sec. 3.1).

- `interval.by1`, `interval.by2`: indicate whether `by1` and/or `by2` are variables measured in intervals (as illustrated in sec. 3.1).
- `adj.breaks`: indicates whether the endpoints of intervals of `by1` or `by2`, when classified into intervals, should be displayed avoiding scientific notation (TRUE, default).
- `digits`, `f.digits`: number of decimals used to round statistics (default = 2 decimals) and frequencies (default = 4 decimals). Importantly, if the chosen rounding formats some non-zero values as zero, the number of decimals is increased so that values have at least one significant digit, unless the argument `force.digits` (see below) is set to TRUE.
- `force.digits`: indicates whether summaries should be forcedly rounded to the number of decimals specified in `digits` and `f.digits` even if non-zero values are rounded to zero (defaults to FALSE).
- `use.scientific`: indicates whether summaries should be displayed using the scientific notation (TRUE) or not (FALSE, default) typically used by R to represent very large or very small numbers (e.g. $\sim 1e+05$ or $2e-05$).

Below are some examples.

Default summaries: default statistics obtained for a numerical and for a logical variable:

```
distr.summary.x(x = AOV, data = MktDATA) # for the proportion
> Summary measures for AOV
> Summary measures
>   n n.a min  q1 median mean  q3 max  sd  var
> 2224  0  2 10.25 26.37 34.24 44.96 176 28.95 838.37
distr.summary.x(x = MktDATA$AOV > 50) # for binary variable
> Summary measures for MktDATA$AOV > 50
> Summary measures
>   n n.a min  q1 median mean q3 max  sd  var
> 2224  0  0  0  0  0 0.24  0  1 0.43 0.18
```

Collections of summaries: more detailed information on central tendency, dispersion, five numbers summary:

```
distr.summary.x(x = AOV, stats = c("central", "dispersion", "fivenumbers"), data = MktDATA)
> Summary measures for AOV
> Central tendency measures
>   n n.a mode n.modes mode% median mean
> 2224  0   8     1 0.013 26.37 34.24
> Measures of dispersion
>   n n.a range IQR range  sd  var  cv
> 2224  0  174  34.71 28.95 838.37 0.85
> Five number summary
>   n n.a min  q1 median  q3 max
> 2224  0  2 10.25 26.37 44.96 176
```

Note that for the mode also the proportion of cases characterized by the modal value is reported.

Selected statistics: collection of specific summary measures

```
distr.summary.x(x = AOV, stats = c("mean", "sd", "cv", "fivenumbers", "p10", "p90"), data = MktDATA)
> Summary measures for AOV
> Five number summary
>   n n.a min  q1 median  q3 max
> 2224  0  2 10.25 26.37 44.96 176
> Requested statistics
>   n n.a mean  sd  cv p10 p90
> 2224  0 34.24 28.95 0.85 6.33 69.7
```

4.1.1 Conditional summaries of a single variable

The function `distr.summary.x()` can also be used to obtain summaries conditioned to the levels of one variable or to the combination of levels of two variables, specified using the arguments `by1` and `by2`. Below some examples are reported.

Specific statistics (percentiles) for a numeric variable conditioned to one variable:

```
distr.summary.x(x = TotVal, stats = c("p5", "p10", "p25", "p50", "p75", "p90", "p95"),
  by1 = Gender, digits = 1, data = MktDATA)
> Summary measures for TotVal | Gender
> Requested statistics
> Gender      n n.a p5 p10 p25  p50   p75  p90   p95
>      F 1046   0 16  19  36  65.0 256.8  544  832.0
>      M 1176   0 39  87 393 893.5 1299.2 1676 1884.5
```

Summaries (five numbers) conditioned to one variable:

```
distr.summary.x(x = AOV, stats = "fivenumbers", by1 = Gender, data = MktDATA)
> Summary measures for AOV | Gender
> Five number summary
> Gender      n n.a min    q1 median    q3    max
>      F 1046   0   2  7.33 11.50 23.35 168.31
>      M 1176   0   3 27.91 40.46 68.44 176.00
```

Summaries (five numbers) conditioned to two variables:

```
distr.summary.x(x = AOV, stats = "fivenumbers", by1 = Gender, by2 = Kids, data = MktDATA)
> Summary measures for AOV | Gender, Kids
> Five number summary
> Gender Kids    n n.a min    q1 median    q3    max
>      F   0 346   0 2.67 12.60 24.41 32.48 168.31
>      M   0 934   0 4.67 33.96 51.26 69.11 176.00
>      F   1 668   0 2.67  6.50  9.50 16.86  68.80
>      M   1 226   0 3.00 10.17 22.45 34.15 130.00
>      F   2  32   0 2.00  5.00  7.39 11.03  24.75
>      M   2  16   0 3.33 12.01 20.19 29.07  34.89
```

4.1.2 Graphical display of conditional central tendency measures

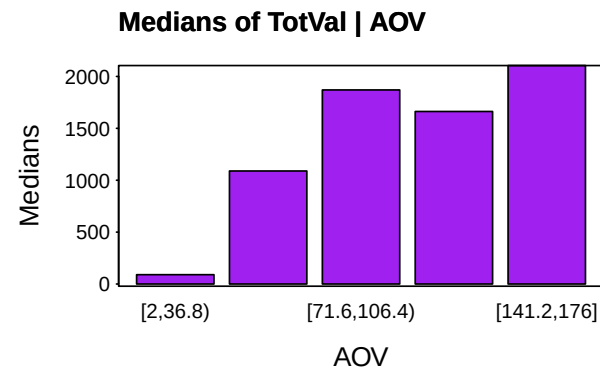
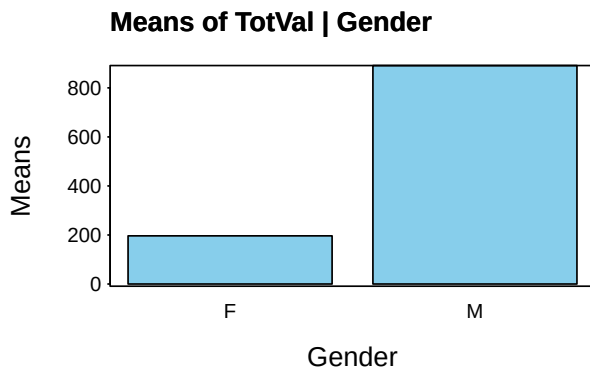
The function `summaries.plot.x()` allows plotting central and non-central conditional tendency measures (namely, means, medians, and quantiles) for a single numeric variable conditioned to one or two variables. The function has the following arguments (more details on the help page of the package):

- **x**: name of the vector to be analysed **or** name of one column in the dataframe specified in `data`;
- **stats**: specifies the tendency measure/s to display in the plot. The available options are "mean", "median", and specific sets of quantiles, namely "quartiles", "quintiles", "deciles", and "percentiles" (a further option, "ci.mean", allowing to plot the means and their confidence intervals, will be discussed later).
- **plot.type**: specifies the type of plot used to compare the requested measures conditioned to the levels of a variable `by1`, possibly broken down by the levels of a second variable, `by2`, if specified. The most useful available options are "bars" (only for "mean" or "median", when a single layer `by1` is specified) and "lines" (this is the unique option available for quantiles).
- **by1, by2**: vectors or factors (typically taking few values/levels) used to build conditional summaries, that can be defined same way as `x`; note that at least one layer must be specified.
- **breaks.by1, breaks.by2**: allow classifying `by1` and/or `by2` into intervals (as seen in sec. 3.1).
- **interval.by1, interval.by2**: indicate whether `by1` and/or `by2` are variables measured in intervals (as seen in sec. 3.1).
- **adj.breaks**: indicates whether the endpoints of intervals of `by1` or `by2`, when classified into intervals, should be displayed avoiding scientific notation (TRUE, default).
- **bw, color**: specify whether the plot has to be colored using a scale of greys (`bw = TRUE`) or a user-defined palette (e.g., `color = c("gold", "magenta")`) rather than a standard palette (see sec. 3.1).
- **legend**: indicates whether a legend should be displayed (`legend = TRUE`, default) or not (`legend = FALSE`).

Also in this case, the argument `use.scientific` can be used to specify whether numbers on the plot's *axes* should be displayed using scientific notation (TRUE) or not (FALSE, default).

Below are the bar plots for the means and the medians of the numerical variable `TotVal` conditioned respectively to the categorical variable `Gender` and to the numerical variable `AOV` classified into intervals.

```
# Means (bars) by a character
summaries.plot.x(x = TotVal, stats = "mean", by1 = Gender, data = MktDATA)
>   Gender    n n.a   mean
> 1     F 1046   0 196.4598
> 2     M 1176   0 891.1267
# Medians (bars) by a variable classified into intervals,
# with user-defined color
summaries.plot.x(x = TotVal, stats = "median", by1 = AOV, breaks.by1 = 5,
                 color = "purple", data = MktDATA)
>   AOV      n n.a median
> 1  [2,36.8) 1468   0   90.0
> 2  [36.8,71.6) 575   0 1089.0
> 3  [71.6,106.4) 116   0 1870.0
> 4 [106.4,141.2)  58   0 1662.5
> 5 [141.2,176]   7   0 2105.0
```

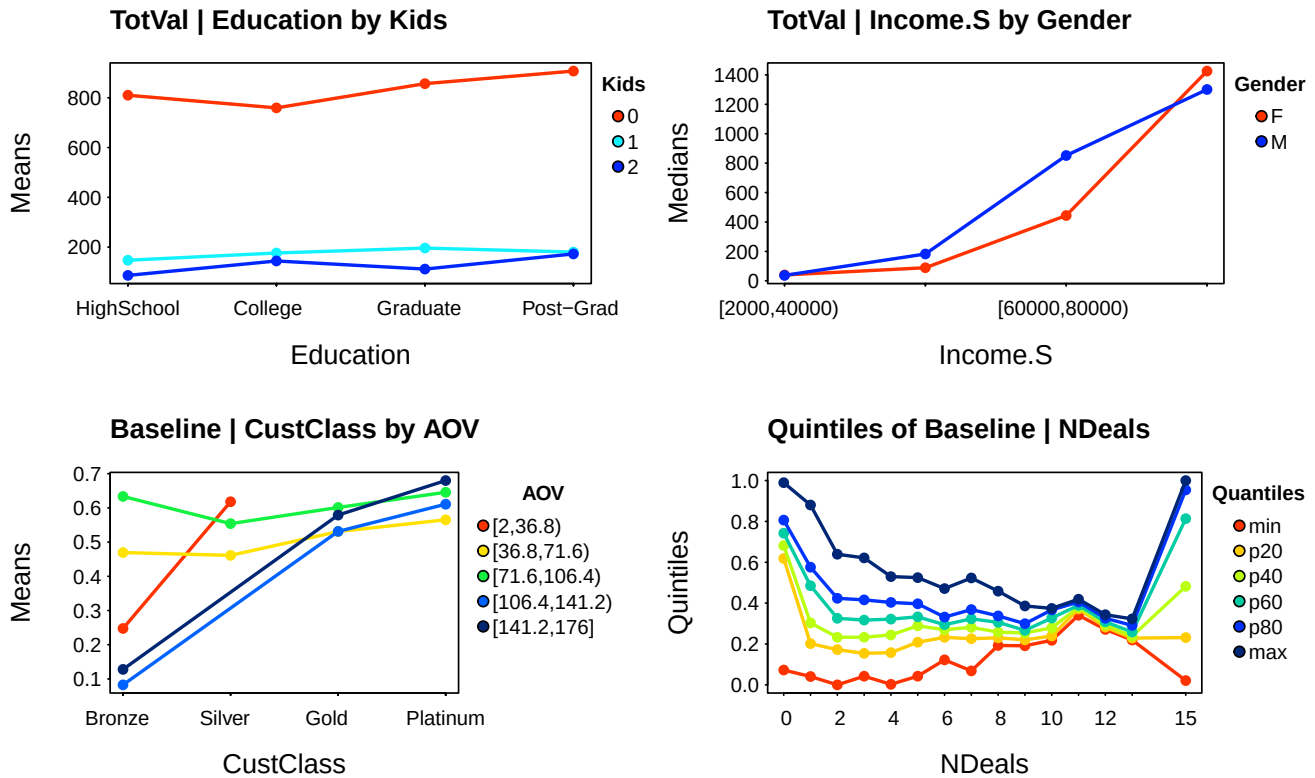


Note that a table with the requested statistics is also printed out [In the following such tables will be not be displayed to save space].

When the means or the medians have to be calculated conditioned to two variables, only lines plots can be used. Same holds when plots for conditioned quantiles are requested; in this case, only one layer is allowed.

```
# Means and medians by two variables
summaries.plot.x(x = TotVal, stats = "mean", by1 = Education, by2 = Kids, data = MktDATA)
summaries.plot.x(x = TotVal, stats = "median", by1 = Income.S, by2 = Gender,
                 interval.by1 = TRUE, data = MktDATA)
summaries.plot.x(x = Baseline, stats = "mean", by1 = CustClass, by2 = AOV,
                 breaks.by2 = 5, data = MktDATA)

# Quantiles by a single variable
summaries.plot.x(x = Baseline, stats = "quantiles", plot.type = "lines",
                 by1 = NDeals, data = MktDATA)
```



4.2 Linear association: covariance and correlation

To calculate the covariance and the correlation between two variables it is possible to refer to **basic functions in R**, namely `cov()` and `cor()`, using the following arguments:

- **x** and **y**: two input **numeric** vectors; note that typically R basic function only admit vectors as input, and it is not possible to refer to the columns-names of a dataframe.
- **use**: specifies how to deal with missing values. The default value is `use = "everything"`, that returns a missing value if one of the data is a missing value. When only two variables are considered, as in the present case, cases with missing values can be removed from the computations specifying `use = "complete.obs"` (possibly abbreviated, e.g. `use = "complete"`) [Other options are possible, but relate to the case when the functions are applied to multiple variables].

Below covariance and correlations are obtained for two variables having missing values:

```
cov(MktDATA$Baseline, MktDATA$TotVal) # missing value!
> [1] NA
cov(MktDATA$Baseline, MktDATA$TotVal, use = "complete")
> [1] 84.48346
cor(MktDATA$Baseline, MktDATA$TotVal, use = "complete")
> [1] 0.8557355
```

Note that the functions return an error if one or both the input vectors are not numeric:

```
cov(MktDATA$Income, MktDATA$TotVal)
> Error in cov(MktDATA$Income, MktDATA$TotVal): is.numeric(x) || is.logical(x) is not TRUE
```

5 Working with random numbers in R

We provide a brief illustration of the R functions to calculate probabilities and percentiles for random variables distributed according to some basic probability/density distributions, namely the normal (or Gaussian) distribution, the Student's t distribution, the chi-square distribution, and the Fisher's F distribution.

The **probabilities** associated with a r.v. can be calculated using functions named `pd.name(q, <args>)`, where *d.name* is the name of the distribution (`norm`, `t`, `chisq`, `f`), and *<args>* are arguments specifying the distribution's parameters. The argument *q* is the (numeric) **value** at which the cumulative probability has to be calculated. With the illustrated syntax, the functions return $P(X \leq q)$, that is the probability **cumulated at** *q* (or, in other words, the area under the curve left of *q*).

Using a similar logic, the **quantiles** of a r.v. can be calculated using functions named `qd.name(p, <args>)`, where *d.name* is the name of the distribution (`norm`, `t`, `chisq`, `f`), and *<args>* are arguments specifying the distribution's parameters. The argument *p* is a **probability**, indicating the order of the required percentile; the functions return the *p*-th percentile, that is the value at which the **cumulated probability** is *p* (thus, the area under the curve left of the value is *p*).

As a further notice, consider that the mentioned functions depend on additional arguments that will not be considered here (in order to keep the syntax as simple as possible).

5.1 Normal r.v.

The **probabilities** associated with a normal random variable can be calculated using the function `pnorm(q, mean, sd)`, where:

- *q* is the (numeric) **value** corresponding to which the probability $P(X \leq q)$ has to be calculated
- **mean, sd**: specify the mean and the standard deviation of the distribution; if not specified the standard normal distribution is used, with `mean=0` and `sd=1`.

Below are some examples:

```
# probability that a normal r.v. with mean 3 and sd 2 is <=4:
pnorm(4, mean = 3, sd = 2) # same as pnorm(q = 4, mean = 3, sd = 2) or pnorm(4,3,2)
> [1] 0.6914625
# probability that a normal r.v. with mean 3 and sd 2 is >1:
1-pnorm(1, mean = 3, sd = 2) # same as 1-pnorm(q = 1, mean = 3, sd = 2) or 1-pnorm(1,3,2)
> [1] 0.8413447

# simplified call for standard normal (mean=0, sd=1):
pnorm(1.96) #
> [1] 0.9750021
1-pnorm(1.96) # P(Z>1.96)=P(Z< -1.96)
> [1] 0.0249979
```

Using this function it is clearly possible also to calculate probabilities related to the sample mean under the assumption of normal distribution or when the number of cases is high enough to assume normal distribution, by properly selecting the values specified in `mean` and `sd`.

The **quantiles** of a normal random variable can be calculated using the function `qnorm(p, mean, sd)`, where:

- *p* is an input **probability**, indicating the order of the required percentile
- **mean, sd**: specify the mean and the standard deviation of the distribution; if not specified the standard normal distribution is used, with `mean=0` and `sd=1`.

Below are some examples:

```
# 90-th percentile of a normal r.v. with mean 3 and sd 2:
qnorm(0.9, mean = 3, sd = 2) # same as qnorm(p = 0.9, mean = 3, sd = 2) or qnorm(0.9,3,2)
> [1] 5.563103

# 10-th percentile of normal r.v. with mean 3 and sd 2:
qnorm(0.1, mean = 3, sd = 2) # same as qnorm(p = 0.1, mean = 3, sd = 2) or qnorm(0.1, 3, 2)
> [1] 0.4368969
```



```
# simplified call for standard normal (mean = 0, sd = 1):
qnorm(0.975) # percentile 0.975
> [1] 1.959964
qnorm(1 - 0.975) # percentile 0.025
> [1] -1.959964
-qnorm(0.975) # this holds because std normal is symmetric around 0
> [1] -1.959964
```

5.2 Student's t r.v.

The **probabilities** associated with a Student's t distribution can be calculated using the function `pt(q, df)`, where:

- **q** is the (numeric) **value** corresponding to which the probability $P(X \leq q)$ has to be calculated
- **df**: degrees of freedom of the distribution.

Below are some examples:

```
# probability that a student's t with 15 df is <=3:
pt(3, df = 15) # same as pt(q = 3, df = 15) or pt(3,15)
> [1] 0.9955136

# probability that a student's t with 15 df is >3:
1-pt(3, df = 15) # same as 1 - pt(q = 3, df = 15) or 1 - pt(3,15)
> [1] 0.004486369
```

The **quantiles** of a Student's t distribution can be calculated using the function `qt(p, df)`, where:

- **p** is an input **probability**, indicating the order of the required percentile
- **df**: degrees of freedom of the distribution.

Below are some examples:

```
# 90-th percentile of a student's t with 15 df:
qt(0.9, df = 15) # same as qt(p = 0.9, df = 15) or qt(0.9,15)
> [1] 1.340606

# 10-th percentile of a student's t with 15 df:
qt(0.1, df = 15) # same as qt(p = 0.1, df = 15) or qt(0.1,15)
> [1] -1.340606

# or, since the student's t distribution is symmetric around 0:
-qt(0.9,15)
> [1] -1.340606
```

5.3 Chi-square and F r.v.

The **probabilities** associated with the chi-square and the F distributions can be calculated using respectively the functions `pchisq(q, df)` and `pf(q, df1, df2)`. The functions differ from those described before only with respect to their characterizing parameters:

```
# probability that a chi-square r.v. with 5 df is <=3:
pchisq(q = 3, df = 5)
> [1] 0.3000142

# probability that a chi-square r.v. with 5 df is >3:
1 - pchisq(3,5) # unnamed form
> [1] 0.6999858

# probability that an F r.v. with 1 and 5 df is <=3:
pf(q = 3, df1 = 1, df2 = 5)
```



```
> [1] 0.8561892
# probability that an F r.v. with 1 and 5 df is >3:
1 - pf(3,1,5) # unnamed form
> [1] 0.1438108
```

The **quantiles** of the two distributions can be calculated using the functions `qchisq(p, df)` and `qf(p, df1, df2)`. The functions differ from those described before only with respect to their characterizing parameters:

```
# 90-th percentile of chi-square r.v. with 5 df:
qchisq(p = 0.9, df = 5) # same as qchisq(0.9,5)
> [1] 9.236357

# 10-th percentile of chi-square r.v. with 5 df:
qchisq(0.1, 5) # same as qchisq(p = 0.1, df = 5)
> [1] 1.610308

# 95-th percentile of an F r.v. with 1 and 5 df:
qf(p = 0.95, df1 = 1, df2 = 5) # same as qf(0.95,1,5)
> [1] 6.607891

# 5-th percentile of an F r.v. with 1 and 5 df:
qf(0.05, 1, 5) # same as qf(p = 0.05, df1 = 1, df2 = 5)
> [1] 0.004344768
```

6 Inference

Confidence intervals and tests for a number of parameters can be easily obtained based on sample statistics (point estimates) and their standard errors, and on the percentiles of a proper distribution.

For example, the 95% confidence interval for the mean of a population normally distributed with variance equal to 25 based on a sample of size 135 whose mean is 12.44 can be calculated as:

```
n <- 135
xbar <- 12.44
sigma2 <- 25
ME <- qnorm(0.975)*sqrt(sigma2/n) # margin of error
ci_mean <- c((xbar - ME), (xbar + ME))

# same as:
sigma <- 5
ME <- qnorm(0.975)*sigma/sqrt(n) # margin of error
ci_mean <- c((xbar - ME), (xbar + ME))

ci_mean
> [1] 11.59657 13.28343
```

If the variance is unknown, the confidence interval can be obtained based on the sample variance, say 30, and on the percentile of the normal distribution (central limit theorem) or of the Student's t distribution (under the assumption of normality):

```
xbar <- 12.44
s2 <- 30
n <- 135
ME.Norm <- qnorm(0.975)*sqrt(s2/n) # margin of error
ci_mean.Norm <- c((xbar - ME.Norm), (xbar + ME.Norm))
ME.T <- qt(0.975, df = (n-1))*sqrt(s2/n) # margin of error
ci_mean.T <- c((xbar - ME.T), (xbar + ME.T))
ci_mean.Norm
> [1] 11.51606 13.36394
ci_mean.T
> [1] 11.50764 13.37236
```

The same holds for the proportion; the confidence interval based on a sample sized 450 with a sample proportion equal to 0.14 (63 successes over 450) can be obtained as:

```
phat <- 0.14 # or phat<-63/450
n <- 450
ME.Norm <- qnorm(0.975)*sqrt(phat*(1 - phat)/n) # margin of error
ci_prop.Norm <- c((phat - ME.Norm), (phat + ME.Norm))
ci_prop.Norm
> [1] 0.1079406 0.1720594
```

Similar procedures can be used to determine rejection regions or p-values for tests on mean or proportions. Same holds for the difference between means and proportions; in these cases, the standard errors to use in confidence intervals or hypothesis testing must be properly calculated depending on the assumptions.

Different functions are available in R for inference based on **raw data**. Here we refer to the functions available in package **UBStats**, in order to focus on a reduced number of functions, developed using the same logic and sharing a similar syntax, and allowing to deal with specific cases not covered by the basic R functions (e.g. inference on the mean in the case of known variance).

6.1 Inference on the mean

The functions `CI.mean()` and `TEST.mean()` allow obtaining confidence intervals for the mean of a population and to test hypotheses on it. In both the functions:

- **x**: name of a **numeric** vector of data **or** name of one column in the dataframe specified in `data`.
- **sigma**: **optional** argument specifying the population standard deviation, when **known**.
- **digits**: specifies the number of decimals used to round statistics (default = 2 decimals). If the chosen rounding formats some non-zero values as zero, the number of decimals is increased so that all values have at least one significant digit, unless the argument `force.digits` (see below) is set to `TRUE`.
- **force.digits**: indicates whether reported values should be rounded to the number of decimals specified in `digits` even if non-zero values are rounded to zero (defaults to `FALSE`).
- **use.scientific**: indicates whether numbers in tables should be displayed using scientific notation (`TRUE`) or not (`FALSE`, default) to represent very large or very small numbers (e.g. $\sim 1e+05$ or $2e-05$).

In addition:

- in function `CI.mean` the argument `conf.level` can be used to specify the confidence level (default is 0.95).
- in function `TEST.mean` the argument `mu0` specifies the null hypothesis (default is 0), and the argument `alternative` specifies the direction of the alternative hypothesis, which can be `"two.sided"` (population mean differs from `mu0`, default), or `"less"` (population mean is lower than `mu0`), or `"greater"` (population mean is higher than `mu0`).

6.1.1 Confidence intervals for the mean

The function `CI.mean` reports the number of cases used to build the interval, `n`, the sample mean, `xbar`, the population standard deviation, `sigma_X`, when known, and the sample standard deviation, `s_X`, otherwise, the standard error of the sample mean, `SE`, or its estimate, `se`, and the limits of the confidence interval, `Lower` and `Upper`.

Below are some examples to build intervals under different assumptions on the variance.

6.1.1.1 Known variance.

```
# CI for the mean with KNOWN variance with default options
CI.mean(x = AOV, sigma = 30, data = MktDATA)
> Confidence interval for the mean
> Confidence level: 0.95
> Variance: known
>      n  xbar sigma_X   SE Lower Upper
> 2224 34.24      30 0.64 32.99 35.49

# CI for the mean with KNOWN variance, with confidence level 0.9 and 3 digits
CI.mean(x = AOV, sigma = 30, conf.level = 0.9, digits = 3, data = MktDATA)
> Confidence interval for the mean
> Confidence level: 0.9
> Variance: known
>      n  xbar sigma_X   SE Lower Upper
> 2224 34.241      30 0.636 33.194 35.287
```

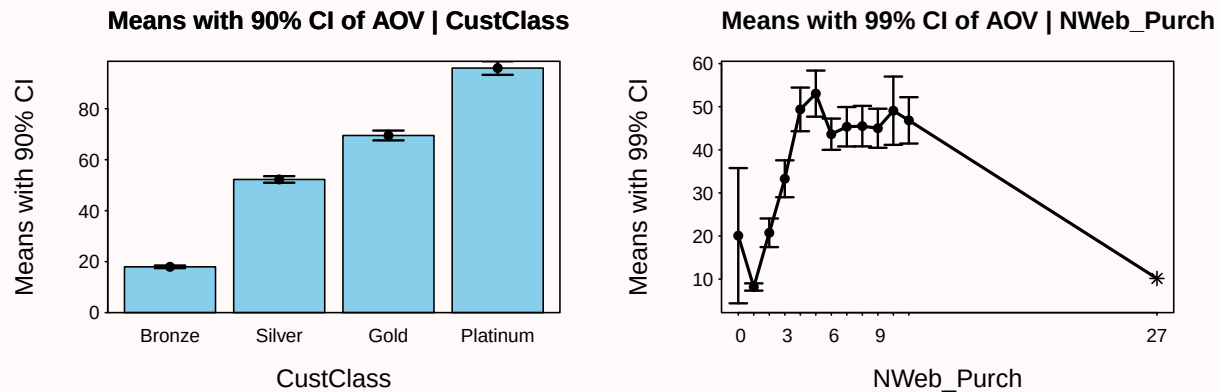
6.1.1.2 Unknown variance. In this case two intervals are built, based respectively on the normal approximation and on the Student's *t* distribution.

```
# CI for the mean with UNKNOWN variance, confidence level 0.99
CI.mean(x = AOV, conf.level = 0.99, data = MktDATA)
> Confidence interval for the mean
> Confidence level: 0.99
> Variance: unknown
>      n  xbar   s_X   se Lower Upper
> Normal.Approx 2224 34.24 28.95 0.61 32.66 35.82
> Student-t      2224 34.24 28.95 0.61 32.66 35.82
```

Additional Material

6.1.1.3 Plots of means with their confidence levels The function `summaries.plot.x()` introduced in Section 4.1.2 allows to plot the means of a numerical variable conditioned to the values of a (typically discrete) variable, together with their confidence intervals, obtained under the assumption of unknown variance (using the Student's *t* distribution). Below two examples are reported, using respectively bars and lines plots.

```
summaries.plot.x(x = AOV, stats = "ci.mean", conf.level = 0.90,
                 plot.type = "bars", by1 = CustClass, data = MktDATA)
summaries.plot.x(x = AOV, stats = "ci.mean", conf.level = 0.90,
                 plot.type = "lines", by1 = NWeb_Purch, data = MktDATA)
```



Actually, the function also displays a table with means and confidence intervals, not reported here to save space.

6.1.2 Test of hypotheses on the mean

The function `TEST.mean` reports the information used to test the null hypothesis versus the specified alternative (number of cases, *n*, sample mean, *xbar*, population standard deviation, *sigma_X* when known or sample standard deviation, *s_X*, standard error of the sample mean, *SE*, or its estimate, *se*), the value of the *test statistic* (*stat*) and its *p-value*.

6.1.2.1 Known variance.

```
# bilateral test
TEST.mean(x = NStore_Purch, sigma = 9, mu0 = 5, alternative = "two.sided", data = MktDATA)
> Test hypotheses on mean
> Variance: known
> Null hypothesis          H0:  $\mu = 5$ 
> Alternative hypothesis H1:  $\mu \neq 5$ 
>   n xbar sigma_X   SE stat p-value
> 2224 5.82      9 0.19  4.3 <0.0001
```

```
# unilateral test
TEST.mean(x = NStore_Purch, sigma = 9, mu0 = 5, alternative = "greater", data = MktDATA)
> Test hypotheses on mean
> Variance: known
> Null hypothesis          H0:  $\mu = 5$  or  $\mu \leq 5$ 
> Alternative hypothesis H1:  $\mu > 5$ 
>   n xbar sigma_X   SE stat p-value
> 2224 5.82      9 0.19  4.3 <0.0001
```

6.1.2.2 Unknown variance. In this case the hypotheses are tested based respectively using both the normal approximation and the Student's *t* distribution.

```
# unilateral test
TEST.mean(x = TotVal, mu0 = 600, alternative = "less", data = MktDATA)
> Test hypotheses on mean
> Variance: unknown
> Null hypothesis      H0:  $\mu = 600$  or  $\mu \geq 600$ 
> Alternative hypothesis H1:  $\mu < 600$ 
>
>      n  xbar  s_X  se  stat p-value
> Normal.Approx 2224 563.64 576.4 12.22 -2.97 0.001
> Student-t     2224 563.64 576.4 12.22 -2.97 0.001
```

6.2 Inference on the proportion

The functions `CI.prop()` and `TEST.prop()` allow obtaining confidence intervals for the proportion of (cases coded as) successes in a population and to test hypotheses on it. In both the functions:

- **x**: name of a vector or factor or name of one column in the dataframe specified in `data`.
- **success**: if `x` is a factor, a character vector, or a numeric non-binary vector, **success** **must** be used to indicate the category/value corresponding to success. The argument can be omitted if `x` is a binary numeric vector (taking values 0 or 1 only; in this case **success** is assumed to be 1) or a logical vector (in these cases **success** is assumed to be TRUE).
- **digits**: specifies the number of decimals used to round statistics (default = 2 decimals). If the chosen rounding formats some non-zero values as zero, the number of decimals is increased so that all values have at least one significant digit, unless the argument `force.digits` (see below) is set to TRUE.
- **force.digits, use.scientific**: indicates whether reported values should be forcedly rounded to the number of decimals specified in `digits` even if non-zero values are rounded to zero (default = FALSE), and/or whether too large or too small numbers should be displayed using scientific notation (default = FALSE).

Also in this case, in function `CI.prop` the argument `conf.level` specifies the confidence level (default is 0.95), and in function `TEST.prop` the argument `p0` specifies the null hypothesis (default is 0.5), and the argument `alternative` specifies the direction of the alternative hypothesis, which can be "two.sided" (population proportion differs from `p0`, default), or "less" (population proportion is lower than `p0`), or "greater" (population proportion is higher than `p0`).

6.2.1 Confidence intervals for the proportion

The function `CI.prop` reports the number of cases, **n**, the sample proportion, **phat**, the sample standard deviation, **s_X**, the estimated standard error of the sample proportion, **se**, and the limits of the confidence interval, **Lower** and **Upper**.

Below we build confidence intervals in the case when the input vector is **not binary or logical**, and the value coded as success **must** be indicated.

```
# Character: a specific value
CI.prop(x = WouldSuggest, success = "Yes", data = MktDATA)
> Confidence interval for the proportion
> Confidence level: 0.95
>      n phat  s_X  se Lower Upper
> 2224 0.68 0.47 0.01 0.66 0.7

# Factor: a specific level
CI.prop(Education, success = "Post-Grad", conf.level = 0.9, digits = 4, data = MktDATA)
> Confidence interval for the proportion
> Confidence level: 0.9
>
> Warning: 2 obs with NA on 'x' removed
>      n  phat  s_X  se Lower Upper
> 2222 0.2115 0.4084 0.0087 0.1973 0.2258

# Numeric vector: a specific value
CI.prop(Children, success = 2, conf.level = 0.99, data = MktDATA)
> Confidence interval for the proportion
> Confidence level: 0.99
```

```
>      n phat  s_X   se Lower Upper
> 2224 0.19 0.39 0.01  0.17  0.21
```

For the **binary** variable LastCampaign (the respondents joined (1) or not (0) the last marketing campaign), and for the **logical** variable Deals.ge50 (the share of products purchases with discount is higher than or equal to 0.5 –TRUE/FALSE) argument success can be *omitted*, provided that the success event is 1 in the first case and TRUE in the second.

```
# CI for the proportion based on a binary variable (taking values 0/1)
CI.prop(x = LastCampaign, conf.level = 0.9, digits = 3, data = MktDATA)
> Confidence interval for the proportion
> Confidence level: 0.9
>      n phat  s_X   se Lower Upper
> 2224 0.15 0.357 0.008 0.138 0.163
# CI.prop(x=LastCampaign, success=1, conf.level=0.9, digits=3, data=MktDATA) # equivalent

# CI for the proportion based on a logical variable (taking values TRUE/FALSE)
CI.prop(x = Deals.ge50, conf.level = 0.99, digits = 3, data = MktDATA)
> Confidence interval for the proportion
> Confidence level: 0.99
>      n phat  s_X   se Lower Upper
> 2224 0.137 0.344 0.007 0.118 0.155
# CI.prop(x=Deals.ge50, success=TRUE, conf.level=0.99, digits=3, data=MktDATA) # equivalent
```

Confidence interval for the proportion of cases satisfying a condition can be built based on a logical indicator of success. For example, to build the interval for the proportion of clients who purchased more than 20 times:

```
# Build a (logical) vector indicating whether the condition is satisfied
HighPurch<-MktDATA$TotPurch>20
CI.prop(x = HighPurch, conf.level = 0.99, digits = 3, data = MktDATA)
> Confidence interval for the proportion
> Confidence level: 0.99
>      n phat  s_X   se Lower Upper
> 2224 0.172 0.377 0.008 0.151 0.192
```

To build the interval for the proportion of gold or platinum clients:

```
# Build a (logical) vector indicating whether a condition is satisfied
IsTop<-MktDATA$CustClass=="Gold" | MktDATA$CustClass=="Platinum"
CI.prop(x = IsTop, conf.level = 0.9, digits = 3, data = MktDATA)
> Confidence interval for the proportion
> Confidence level: 0.9
>      n phat  s_X   se Lower Upper
> 2224 0.18 0.384 0.008 0.166 0.193
```

6.2.2 Test of hypotheses on the proportion

The function TEST.prop reports the same information displayed by CI.prop plus the value of the *test statistic* (stat) and its *p-value*. The considerations about the features of input vectors and the coding of success seen for confidence intervals hold also for tests on proportions. Below are some examples, based on data of different type.

```
# Bilateral test (character or factor: specific value/level)
TEST.prop(x = WouldSuggest, success = "Yes", p0 = 0.7, data = MktDATA)
> Test hypotheses on proportion
> Null hypothesis          H0: p = 0.7
> Alternative hypothesis H1: p ≠ 0.7
>      n phat  s_X   se stat p-value
> 2224 0.68 0.46 0.01 -2.26    0.02
```

```

# Unilateral test (binary variable - takes values 0/1)
TEST.prop(x = LastCampaign, p0 = 0.2, alternative = "less", digits = 3, data = MktDATA)
> Test hypotheses on proportion
> Null hypothesis      H0: p = 0.2 or p ≥ 0.2
> Alternative hypothesis H1: p < 0.2
>   n phat s_X   se  stat p-value
> 2224 0.15 0.4 0.008 -5.874 <0.0001
# same as
# TEST.prop(x=LastCampaign, success=1, p0=0.2, alternative="less", digits=3, data=MktDATA)

# Unilateral test (logical variable, taking values TRUE/FALSE)
TEST.prop(x = Deals.ge50, p0 = 0.13, alternative = "greater", digits = 3, data = MktDATA)
> Test hypotheses on proportion
> Null hypothesis      H0: p = 0.13 or p ≤ 0.13
> Alternative hypothesis H1: p > 0.13
>   n phat s_X   se  stat p-value
> 2224 0.137 0.336 0.007 0.938 0.174
# same as
# TEST.prop(x=Deals.ge50, success=TRUE, p0=0.13, alternative="greater", digits=3, data=MktDATA)

# Build a (logical) vector indicating whether a condition is satisfied
IsTop<-MktDATA$CustClass=="Gold" | MktDATA$CustClass=="Platinum"
TEST.prop(x = IsTop, p0 = 0.2, alternative = "less", data = MktDATA)
> Test hypotheses on proportion
> Null hypothesis      H0: p = 0.2 or p ≥ 0.2
> Alternative hypothesis H1: p < 0.2
>   n phat s_X   se  stat p-value
> 2224 0.18 0.4 0.01 -2.37 0.01

```

6.3 Inference on the difference between means

The functions `CI.diffmean()` and `TEST.diffmean()` allow building confidence intervals for the difference between the means of two populations and to test hypotheses on it. In both the functions:

- **type**: type of samples, "independent" or "paired".
- **x, y, by**: arguments used to identify the two samples whose means have to be compared. **x** must be specified, and can be combined with **y** or with **by** (only **one** between **y** and **by** can be specified). **x**, **y**, and **by** can be the names of vectors, or the names of columns in the dataframe specified in **data** (it is possible to use a mixed specification, e.g. one vector and one column in data).
 - **x, y**: should be specified when data on the two sub-samples are stored in two distinct vectors.
 - **x, by**: should be specified when data on the two sub-samples should be obtained by *splitting* **x** into two groups based on the **two values** of the **by** vector; in this case, **x** and **by** must have the same length, **x** must be numeric, and **by** can take only two values. Note that this option is available only for **independent** samples, to avoid errors in matching cases.
- The functions allow dealing also with the (very rare case) when the populations' standard deviations are known. In particular:
 - **sigma.x** and **sigma.y** are (optional) arguments to specify the known standard deviations for two **independent samples** identified by **x** and **y**. If only one of the two arguments is specified, the known standard deviations are assumed to be equal.
 - **sigma.by** is an (optional) argument to specify the known standard deviations of the two **independent samples** identified via **by**. **sigma.by** can be a single value indicating the same standard deviation in the two **by**-groups, or a vector with two values, specifying the standard deviations in the two **by**-groups. To avoid errors, in the latter case the vector should be **named**, with names coinciding with the two levels of **by**.
 - **sigma.d** is an (optional) argument to specify the known standard deviation of the **difference** in **paired** samples.
- **var.test** indicates whether a test on the equality of variance in two (**independent**) samples is required (**var.test** = TRUE) or not (**var.test** = FALSE, default)

- **digits**: number of decimals used to round statistics (default = 2 decimals). If the chosen rounding formats some non-zero values as zero, the number of decimals is increased so that all values have at least one significant digit, unless the argument `force.digits` (see below) is set to `TRUE`.
- **force.digits, use.scientific**: indicate whether reported values should be forcedly rounded to the number of decimals specified in `digits` even if non-zero values are rounded to zero (default = `FALSE`), and/or whether too large or too small numbers should be displayed using scientific notation (default = `FALSE`).

In function `CI.diffmean` the additional argument `conf.level` specifies the confidence level (default is 0.95).

In function `TEST.diffmean` the argument `mdiff0` specifies the null hypothesis (default is 0); if `x` and `y` are given as inputs, `mdiff0` is the difference $\mu_x - \mu_y$; if `x` and `by` are given as inputs, the two levels of the variable specified in `by` are considered in their standard order (alphabetical or numeric, or order of the levels for factors), say `by1` and `by2`, and `mdiff0` is $\mu_{by1} - \mu_{by2}$. The argument `alternative` specifies the direction of the alternative hypothesis, which can be `"two.sided"` (difference between populations' means is different from `mdiff0`, default), or `"less"` (difference between populations' means lower than `mdiff0`), or `"greater"` (difference between populations' means is higher than `mdiff0`).

6.3.1 Confidence intervals for independent samples

We show how to build intervals using both arguments `x`, `y` (data on the two samples are in two separate vectors¹) and arguments `x`, `by` (the two samples are obtained by splitting data in `x` based on an existing or appositely created vector taking two values). Note that if `x` and `y` are given, the confidence interval is built for $\mu_x - \mu_y$; if `x` and `by` are used, the two levels of the variable specified in `by` are ordered (alphabetical or numerical order, or order of the levels for factors), say `by1` and `by2`, and the confidence interval is built for $\mu_{by1} - \mu_{by2}$.

The function returns information on the sample statistics used to build the interval (samples' sizes, `n_x` and `n_y`; samples' means, `xbar` and `ybar`, populations' standard deviations `sigma_X` and `sigma_Y` – if known – or samples' standard deviations, `s_X` and `s_Y`, estimated standard error of the means' difference, `se`), and the lower and upper bounds of the intervals.

6.3.1.1 Unknown variances. In the typical case when variances are **unknown** confidence intervals are built both under the assumption of equality and under the assumption of inequality of variances.

Below we refer to two vectors containing data on the variable AOV for females and males:

¹We will consider the case when `x` and `y` are two vectors in the environment; of course, in some applications the vectors with data on the two samples might be columns of the same dataframe. In this case, `x` and `y` will be the names of the two columns.


```
# based on x,y: build vectors with data on males and females
AOV_F <- MktDATA$AOV[MktDATA$Gender == "F"]
AOV_M <- MktDATA$AOV[MktDATA$Gender == "M"]
CI.diffmean(x = AOV_M, y = AOV_F, type = "independent") # vectors in environment, no data specified
> Confidence interval for  $\mu_x - \mu_y$ 
> Samples: independent
> Confidence level: 0.95
> Variances: unknown
>   x = AOV_M
>   y = AOV_F
>
> Warning: 2 obs with NA removed from 'x' & 2 obs with NA removed from 'y'
>
> Unknown variances assumed to be equal
>      n_x  n_y  xbar  ybar xbar-ybar   s_X   s_Y   se Lower Upper
> Normal.Approx 1176 1046 49.86 16.74    33.12 29.88 14.04 1.01 31.14  35.1
> Student-t    1176 1046 49.86 16.74    33.12 29.88 14.04 1.01 31.14  35.1
>
> Unknown variances assumed to be different
>      n_x  n_y  xbar  ybar xbar-ybar   s_X   s_Y   se Lower Upper
> Normal.Approx 1176 1046 49.86 16.74    33.12 29.88 14.04 0.97 31.21  35.03
> Student-t    1176 1046 49.86 16.74    33.12 29.88 14.04 0.97 31.21  35.03
```

Note that in both cases **two** confidence intervals are reported based on the normal approximation and on the Student's *t* distribution. The decision concerning the interval to take into account depends on the assumptions of the analyst, even if one could clearly be interested to **test** whether the unknown populations' variances are equal or not (see the end of the section).

The intervals can also be built using the entire set of data on AOV split into two samples based on the levels of Gender, noting however that the levels of Gender would be ordered alphabetically (F and M), and the interval for $\mu_F - \mu_M$ rather than that for $\mu_M - \mu_F$ would be obtained. Clearly this is not an issue, (the two intervals only have reversed signs) but if one wants to control the parameter strictly, a splitting factor must be created with the desired order (**output printed at the end of this section**). This is why the syntax based on *x,y* is *sometimes* easier to control and use.

```
# based on x,by (type = "independent" can be omitted - default):
# based on Gender, F-M
CI.diffmean(x = AOV, by = Gender, data = MktDATA) # AOV in dataframe, data specified
# based on new splitting factor
Gender.R <- factor(MktDATA$Gender, levels = c("M","F"))
CI.diffmean(x = AOV, by = Gender.R, data = MktDATA) # M-F
```

Below is another example, with groups defined based on whether the clients have children or not:

```
# based on x,y: build vectors with data on the clients with and without children
AOV_no_child <- MktDATA$AOV[MktDATA$Children == 0]
AOV_w_child <- MktDATA$AOV[MktDATA$Children > 0]
CI.diffmean(x = AOV_no_child, y = AOV_w_child, type = "independent",
  conf.level = 0.90) # no data specified
> Confidence interval for  $\mu_x - \mu_y$ 
> Samples: independent
> Confidence level: 0.9
> Variances: unknown
>   x = AOV_no_child
>   y = AOV_w_child
>
> Unknown variances assumed to be equal
>      n_x  n_y  xbar  ybar xbar-ybar   s_X   s_Y   se Lower Upper
```

```
> Normal.Approx 631 1593 58.21 24.75      33.46 32.97 20.56 1.16 31.55 35.37
> Student-t     631 1593 58.21 24.75      33.46 32.97 20.56 1.16 31.55 35.37
>
> Unknown variances assumed to be different
>           n_x  n_y  xbar  ybar xbar-ybar   s_X   s_Y   se Lower Upper
> Normal.Approx 631 1593 58.21 24.75      33.46 32.97 20.56 1.41 31.14 35.78
> Student-t     631 1593 58.21 24.75      33.46 32.97 20.56 1.41 31.14 35.78
```

Also in this case, the syntax based on `x,by` can be used, by preliminarily building a vector whose levels define the splitting criterion:

```
# same result as before using:
# based on x,by: build the grouping vector and then the interval
HasChildren <- MktDATA$Children > 0
CI.diffmean(x = AOV, by = HasChildren, type = "independent",
            conf.level = 0.90, data = MktDATA) # AOV in dataframe, data specified
```

Verifying hypothesis on the equality of variance. To build – besides the confidence intervals – also a test on the equality of variances, it is possible to specify `var.test = TRUE`, as shown in the example below, where we build the same confidence intervals obtained before based on the entire set of data on AOV split into two samples based on the two levels of the factor Gender.R:

```
# CI for the difference between means in groups of males and
# based on x,by: x split based on the by's (2) levels
CI.diffmean(x = AOV, by = Gender.R, type = "independent",
            var.test = TRUE, data = MktDATA) # AOV in dataframe, data specified
> Confidence interval for  $\mu_x - \mu_y$ 
> Samples: independent
> Confidence level: 0.95
> Variances: unknown
>   x = AOV|Gender.R=M
>   y = AOV|Gender.R=F
>
> Warning: 2 obs with NA removed from 'x' & 2 obs with NA removed from 'y'
>
> Unknown variances assumed to be equal
>           n_x  n_y  xbar  ybar xbar-ybar   s_X   s_Y   se Lower Upper
> Normal.Approx 1176 1046 49.86 16.74      33.12 29.88 14.04 1.01 31.14 35.1
> Student-t     1176 1046 49.86 16.74      33.12 29.88 14.04 1.01 31.14 35.1
>
> Unknown variances assumed to be different
>           n_x  n_y  xbar  ybar xbar-ybar   s_X   s_Y   se Lower Upper
> Normal.Approx 1176 1046 49.86 16.74      33.12 29.88 14.04 0.97 31.21 35.03
> Student-t     1176 1046 49.86 16.74      33.12 29.88 14.04 0.97 31.21 35.03
>
> Levene test for homogeneity of variance
> Null hypothesis      H0:  $\sigma^2_x = \sigma^2_y$ 
> Alternative hypothesis H1:  $\sigma^2_x \neq \sigma^2_y$ 
>   s2_x   s2_y F-stat df1  df2 p-value
> 892.57 197.23 425.72   1 2220 <0.0001
```

Additional Material

6.3.1.2 Known variances. As mentioned, the case when the variances in the two populations are known is very rare (particularly in social sciences). Even so, for the sake of completeness, below are some examples.

```
# CI for the difference between means in groups of males and females
# based on x,y: build vectors with data on the two groups
AOV_F <- MktDATA$AOV[MktDATA$Gender == "F"]
AOV_M <- MktDATA$AOV[MktDATA$Gender == "M"]
CI.diffmean(x = AOV_M, y = AOV_F, type = "independent",
            sigma.x = 30, sigma.y = 25, conf.level = 0.99)
> Confidence interval for  $\mu_x - \mu_y$ 
> Samples: independent
> Confidence level: 0.99
> Variances: known
>   x = AOV_M
>   y = AOV_F
>
> Warning: 2 obs with NA removed from 'x' & 2 obs with NA removed from 'y'
>   n_x  n_y  xbar  ybar xbar-ybar sigma_X sigma_Y  SE Lower Upper
> 1176 1046 49.86 16.74   33.12     30     25 1.17 30.11 36.13
```

Similarly to what was illustrated in the previous section, the same result (not printed) can be obtained using x, by:

```
# same result as before (not printed) using:
# based on x,by: x split based on the by's (2) levels
CI.diffmean(x = AOV, by = Gender.R, type = "independent",
            sigma.by = c("F"=25,"M"=30), conf.level = 0.99, data = MktDATA)
```

Note importantly that when by is used (and the known variances differ), sigma.by should be a **named** vector (guaranteeing the correct assignment of the variance to the two by-subsets), otherwise the function returns an error. This is the reason why we definitely suggest to prefer the approach based on x, y in this case.

In the particular case when the variances are assumed to be **known** and **equal**, a single value can be specified for the common variance; in this case, if by is used, sigma.by does not need to be named, because the assignment of variance to the groups is clear.

```
# based on x,y: vectors with data on the two groups
# note that only one between sigma.x and sigma.y needs to be specified
CI.diffmean(x = AOV_M, y = AOV_F, type = "independent", sigma.x = 28, conf.level = 0.99)
> Confidence interval for  $\mu_x - \mu_y$ 
> Samples: independent
> Confidence level: 0.99
> Variances: known
>   x = AOV_M
>   y = AOV_F
>
> Warning: 2 obs with NA removed from 'x' & 2 obs with NA removed from 'y'
>   n_x  n_y  xbar  ybar xbar-ybar sigma_X sigma_Y  SE Lower Upper
> 1176 1046 49.86 16.74   33.12     28     28 1.19 30.05 36.19
```

The same result (not printed) is obtained using:

```
# same as (not printed):
# based on x,by: x split based on the by's (2) levels
# note that a single value is specified in sigma.by, and no names are needed
CI.diffmean(x = AOV, by = Gender.R, type = "independent", sigma.by = 28,
            conf.level = 0.99, data = MktDATA) # data specified
```

6.3.2 Hypotheses testing for independent samples

Hypotheses on the means of independent samples can be tested using inputs and syntax identical to those needed to build confidence intervals; the unique difference is that instead of specifying the confidence level, it is necessary to specify the null value hypothesized for the difference between the means and the direction of the alternative hypothesis.

In the examples below we report only the syntax based on the `x` and `y` inputs for the same examples considered in the previous section; the syntax based on the `x` and `by` is reported but not printed.

6.3.2.1 Unknown variances. When variances are unknown, tests on hypotheses on the difference between means are built both under the assumption of equality and under the assumption of inequality of variances. Also in this case it is possible to test the hypothesis of equality between variances, by specifying `var.test = TRUE`, as shown below.

```
# Unilateral test for the difference between means in groups of males and females
# based on x,y: build vectors with data on the two groups
AOV_F <- MktDATA$AOV[MktDATA$Gender == "F"]
AOV_M <- MktDATA$AOV[MktDATA$Gender == "M"]
TEST.diffmean(x = AOV_M, y = AOV_F, type = "independent", mdiff0 = 30,
              alternative = "two.sided", var.test = TRUE)
> Test hypotheses on  $\mu_x - \mu_y$ 
> Samples: independent
> Variances: unknown
>   x = AOV_M
>   y = AOV_F
>
> Warning: 2 obs with NA removed from 'x' & 2 obs with NA removed from 'y'
> Null hypothesis      H0: ( $\mu_x - \mu_y$ ) = 30
> Alternative hypothesis H1: ( $\mu_x - \mu_y$ )  $\neq$  30
>
> Unknown variances assumed to be equal
>
>      n_x  n_y  xbar  ybar  xbar-ybar   s_X   s_Y   se stat p-value
> Normal.Approx 1176 1046 49.86 16.74    33.12 29.88 14.04 1.01 3.09  0.002
> Student-t     1176 1046 49.86 16.74    33.12 29.88 14.04 1.01 3.09  0.002
>
> Unknown variances assumed to be different
>
>      n_x  n_y  xbar  ybar  xbar-ybar   s_X   s_Y   se stat p-value
> Normal.Approx 1176 1046 49.86 16.74    33.12 29.88 14.04 0.97  3.2  0.001
> Student-t     1176 1046 49.86 16.74    33.12 29.88 14.04 0.97  3.2  0.001
>
> Levene test for homogeneity of variance
> Null hypothesis      H0:  $\sigma^2_x = \sigma^2_y$ 
> Alternative hypothesis H1:  $\sigma^2_x \neq \sigma^2_y$ 
>   s2_x  s2_y F-stat df1  df2 p-value
> 892.57 197.23 425.72   1 2220 <0.0001

# same as (not printed):
# based on x,by: x split based on the by's (2) levels
TEST.diffmean(x = AOV, by = Gender.R, type = "independent", mdiff0 = 30,
              alternative = "two.sided",
              var.test = TRUE, data = MktDATA) # data specified
```

Below is an example of groups identified based on a condition defined on the variable `Children`:

```
# Test for the difference between means in groups of respondents
# with and without children
# based on x,y: build vectors with data on the two groups
AOV_no_child <- MktDATA$AOV[MktDATA$Children == 0]
AOV_w_child <- MktDATA$AOV[MktDATA$Children > 0]
```

```

TEST.diffmean(x = AOV_no_child, y = AOV_w_child, type = "independent",
              mdiff0 = 35, alternative = "less", var.test = TRUE) # no data specified
> Test hypotheses on  $\mu_x - \mu_y$ 
> Samples: independent
> Variances: unknown
>   x = AOV_no_child
>   y = AOV_w_child
> Null hypothesis      H0:  $(\mu_x - \mu_y) = 35$  or  $(\mu_x - \mu_y) \geq 35$ 
> Alternative hypothesis H1:  $(\mu_x - \mu_y) < 35$ 
>
> Unknown variances assumed to be equal
>
>      n_x  n_y  xbar  ybar  xbar-ybar   s_X   s_Y   se  stat p-value
> Normal.Approx 631 1593 58.21 24.75    33.46 32.97 20.56 1.16 -1.33  0.09
> Student-t    631 1593 58.21 24.75    33.46 32.97 20.56 1.16 -1.33  0.09
>
> Unknown variances assumed to be different
>
>      n_x  n_y  xbar  ybar  xbar-ybar   s_X   s_Y   se  stat p-value
> Normal.Approx 631 1593 58.21 24.75    33.46 32.97 20.56 1.41 -1.09  0.14
> Student-t    631 1593 58.21 24.75    33.46 32.97 20.56 1.41 -1.09  0.14
>
> Levene test for homogeneity of variance
> Null hypothesis      H0:  $\sigma^2_x = \sigma^2_y$ 
> Alternative hypothesis H1:  $\sigma^2_x \neq \sigma^2_y$ 
>      s2_x  s2_y F-stat df1  df2 p-value
> 1086.99 422.69 205.51   1 2222 <0.0001

# same as (not printed):
# based on x,by: build the grouping vector and then the interval
HasChildren <- MktDATA$Children > 0
TEST.diffmean(x = AOV, by = HasChildren, type = "independent",
              mdiff0 = 35, alternative = "less", var.test = TRUE,
              data = MktDATA) # data specified for x

```

Additional Material

6.3.2.2 Known variances. We present below two examples relative to the (rather uncommon) case when variances are known. In the first case we assume that variances are known and different:

```

# Unilateral test for the difference between means in groups of males and females
# based on x,y: build vectors with data on the two groups
# different variances
AOV_F <- MktDATA$AOV[MktDATA$Gender == "F"]
AOV_M <- MktDATA$AOV[MktDATA$Gender == "M"]
TEST.diffmean(x = AOV_M, y = AOV_F, type = "independent",
              sigma.x = 30, sigma.y = 25, mdiff0 = 30,
              alternative="greater") # vectors in environment, no data specified

> Test hypotheses on  $\mu_x - \mu_y$ 
> Samples: independent
> Variances: known
>   x = AOV_M
>   y = AOV_F
>

```

```
> Warning: 2 obs with NA removed from 'x' & 2 obs with NA removed from 'y'
> Null hypothesis      H0: ( $\mu_x - \mu_y$ ) = 30 or ( $\mu_x - \mu_y$ )  $\leq$  30
> Alternative hypothesis H1: ( $\mu_x - \mu_y$ ) > 30
>   n_x  n_y  xbar  ybar xbar-ybar sigma_X sigma_Y   SE stat p-value
> 1176 1046 49.86 16.74    33.12    30    25 1.17 2.67  0.004

# same result (not printed) using:
# based on x,by: x split based on the by's (2) levels
TEST.diffmean(x = AOV, by = Gender.R, type = "independent",
              sigma.by = c("F"=25, "M"=30),
              mdiff0 = 30, alternative = "greater",
              data = MktDATA) # columns in dataframe, data specified
```

In this case, instead, we assume that variances are known and equal:

```
# Test for the difference between means in groups of respondents
# with and without children
# based on x,y: build vectors with data on the two groups
# equal variances (only sigma.x specified)
AOV_no_child <- MktDATA$AOV[MktDATA$Children == 0]
AOV_w_child <- MktDATA$AOV[MktDATA$Children > 0]
TEST.diffmean(x = AOV_no_child, y = AOV_w_child, type = "independent",
              sigma.x = 25, mdiff0 = 35, alternative = "less") # no data specified

> Test hypotheses on  $\mu_x - \mu_y$ 
> Samples: independent
> Variances: known
>   x = AOV_no_child
>   y = AOV_w_child
> Null hypothesis      H0: ( $\mu_x - \mu_y$ ) = 35 or ( $\mu_x - \mu_y$ )  $\geq$  35
> Alternative hypothesis H1: ( $\mu_x - \mu_y$ ) < 35
>   n_x  n_y  xbar  ybar xbar-ybar sigma_X sigma_Y   SE stat p-value
>  631 1593 58.21 24.75    33.46    25    25 1.18 -1.31  0.09

# same as (not printed):
# based on x,by: build the grouping vector and then the interval
# equal variances: sigma.by is a single value
HasChildren<-MktDATA$Children>0
TEST.diffmean(x = AOV, by = HasChildren, type = "independent",
              sigma.by=25, mdiff0 = 35, alternative = "less",
              data = MktDATA) # data specified for x
```

6.3.3 Test on the equality of variance for independent samples

The function `TEST.diffvar()` allows testing the equality between the variances of two independent samples. The input arguments must be specified as described above. For example, to test the equality of variances as done in the last example without running the test on the difference between means:

```
# based on x,y: vectors with data on the two groups
TEST.diffvar(x = AOV_M, y = AOV_F)

# same as (not printed): x split based on the by's (2) levels
TEST.diffvar(x = AOV, by = Gender.R, data = MktDATA) # data specified
```

6.3.4 Confidence intervals for paired samples

Paired samples can only be identified using the arguments `x` and `y` in the function `CI.diffmean` (thus, `by` is not allowed). We consider both the case when the vectors with data on the two paired samples are two columns of the same dataframe, and the

case when vectors with data on the two paired samples need to be built.

6.3.4.1 Variance of the difference unknown. Below is the syntax to build the confidence interval for the difference between the means of the number of purchases online and in-store, when no information on the variance is available:

```
# paired samples, variance of the difference assumed UNKNOWN
CI.diffmean(x = NStore_Purch, y = NWeb_Purch, type = "paired", conf.level = 0.90, data = MktDATA)
> Confidence interval for  $\mu_x - \mu_y$ 
> Samples: paired
> Confidence level: 0.9
> Variance: unknown
>   x = NStore_Purch
>   y = NWeb_Purch
>
>      n xbar ybar dbar=xbar-ybar  s_D  se Lower Upper
> Normal.Approx 2224 5.82 4.07      1.75 2.88 0.06  1.65  1.85
> Student-t     2224 5.82 4.07      1.75 2.88 0.06  1.65  1.85
```

Also in this case, two intervals are reported, based on the normal approximation and on the Student's t distribution. Note that the function does not perform any control about the reliability of the assumptions on the underlying populations' distributions.

We now consider the case when two paired vectors are built; specifically, we obtain the confidence interval for the difference between the means of the two variables considered above limiting attention to female respondents; note that the samples are paired because data are extracted from the same dataframe using the same indexing vector (indicating whether a respondent is a female)

```
# build the paired samples vectors
NWeb_Purch.F <- MktDATA$NWeb_Purch[MktDATA$Gender == "F"]
NStore_Purch.F <- MktDATA$NStore_Purch[MktDATA$Gender == "F"]
CI.diffmean(x = NStore_Purch.F, y = NWeb_Purch.F, type = "paired",
            conf.level = 0.90)
> Confidence interval for  $\mu_x - \mu_y$ 
> Samples: paired
> Confidence level: 0.9
> Variance: unknown
>   x = NStore_Purch.F
>   y = NWeb_Purch.F
>
> Warning: 2 obs with NA on 'x,y' removed
>
>      n xbar ybar dbar=xbar-ybar  s_D  se Lower Upper
> Normal.Approx 1046 4.06 2.99      1.07 1.92 0.06  0.97  1.17
> Student-t     1046 4.06 2.99      1.07 1.92 0.06  0.97  1.17
```

Additional Material

6.3.4.2 Variance of the difference known. We briefly acknowledge this particular case with reference to the previous example:

```
# paired samples, variance of the difference assumed KNOWN
CI.diffmean(x = NStore_Purch, y = NWeb_Purch, type = "paired",
            sigma.d = 3, conf.level = 0.90, data = MktDATA)
> Confidence interval for  $\mu_x - \mu_y$ 
> Samples: paired
> Confidence level: 0.9
> Variance: known
```



```
> x = NStore_Purch
> y = NWeb_Purch
> n xbar ybar dbar=xbar-ybar sigma_D SE Lower Upper
> 2224 5.82 4.07 1.75 3 0.06 1.64 1.85
```

As shown in the previous section, it is also possible to build confidence intervals based on two properly defined vectors.

6.3.5 Hypotheses testing for paired samples

Hypotheses on the means of independent samples can be tested using inputs and syntax identical to those needed to build confidence intervals, specifying the null value hypothesized for the difference and the direction of the alternative hypothesis instead of the confidence level. Below we report examples based on the `x` and `y` used to build confidence intervals.

6.3.5.1 Variance of the difference unknown.

```
TEST.diffmean(x = NStore_Purch, y = NWeb_Purch, type = "paired", mdiff0 = 2,
              alternative = "less", data = MktDATA)
> Test hypotheses on  $\mu_x - \mu_y$ 
> Samples: paired
> Variance: unknown
> x = NStore_Purch
> y = NWeb_Purch
> Null hypothesis H0:  $(\mu_x - \mu_y) = 2$  or  $(\mu_x - \mu_y) \geq 2$ 
> Alternative hypothesis H1:  $(\mu_x - \mu_y) < 2$ 
> n xbar ybar dbar=xbar-ybar s_D se stat p-value
> Normal.Approx 2224 5.82 4.07 1.75 2.88 0.06 -4.11 <0.0001
> Student-t 2224 5.82 4.07 1.75 2.88 0.06 -4.11 <0.0001
```

For two paired vectors (not columns in a dataframe):

```
NWeb_Purch.F <- MktDATA$NWeb_Purch[MktDATA$Gender == "F"]
NStore_Purch.F <- MktDATA$NStore_Purch[MktDATA$Gender == "F"]
TEST.diffmean(x = NStore_Purch.F, y = NWeb_Purch.F, type = "paired", mdiff0 = 1,
              alternative = "greater")
> Test hypotheses on  $\mu_x - \mu_y$ 
> Samples: paired
> Variance: unknown
> x = NStore_Purch.F
> y = NWeb_Purch.F
> Warning: 2 obs with NA on 'x,y' removed
> Null hypothesis H0:  $(\mu_x - \mu_y) = 1$  or  $(\mu_x - \mu_y) \leq 1$ 
> Alternative hypothesis H1:  $(\mu_x - \mu_y) > 1$ 
> n xbar ybar dbar=xbar-ybar s_D se stat p-value
> Normal.Approx 1046 4.06 2.99 1.07 1.92 0.06 1.21 0.11
> Student-t 1046 4.06 2.99 1.07 1.92 0.06 1.21 0.11
```

Additional Material

6.3.5.2 Variance of the difference known. We briefly acknowledge this particular case with reference to the previous example:

```
# paired samples, variance of the difference assumed KNOWN
TEST.diffmean(x = NStore_Purch, y = NWeb_Purch, type = "paired", sigma.d = 3,
              mdiff0 = 2, data = MktDATA)
```



```

> Test hypotheses on  $\mu_x - \mu_y$ 
> Samples: paired
> Variance: known
> x = NStore_Purch
> y = NWeb_Purch
> Null hypothesis      H0: ( $\mu_x - \mu_y$ ) = 2
> Alternative hypothesis H1: ( $\mu_x - \mu_y$ )  $\neq$  2
> n xbar ybar dbar=xbar-ybar sigma_D SE stat p-value
> 2224 5.82 4.07 1.75 3 0.06 -3.95 <0.0001

```

6.4 Inference on the difference between proportions

The functions `CI.diffprop()` and `TEST.diffprop()` allow building confidence intervals for the difference between the proportions in two populations, and to test hypotheses on it. In both the functions:

- **x, y, by:** arguments used to identify the two samples whose proportions have to be compared. **x** must be specified, and can be combined with **y** or with **by** (only **one** between **y** and **by** can be specified).
 - **x, y:** should be specified when data on the two sub-samples are stored in two distinct vectors. **x, y** can be the names of two vectors or factors **as well as** the names of two columns in the dataframe specified in **data**. It is possible to use a mixed specification (e.g, one vector and one column in data).
 - **x, by:** should be specified when data on the two sub-samples should be obtained by *splitting* **x** into two groups based on the **two values** of the **by** vector; in this case, **x** and **by** must have the same length, and **by** can take only two values; **x, by** can be the names of two vectors, **or** the names of two columns in the dataframe specified in **data** (it is possible to use a mixed specification (e.g, one vector and one column in data)).
- **success.x:** if **x** is a factor, a character vector, or a numeric non-binary vector, **success.x** **must** be used to indicate the category/value corresponding to success. The argument can be omitted if **x** is a binary numeric vector (takes values 0 or 1 only; in this case **success.x** is assumed to be 1) or a logical vector (in these cases **success.x** is assumed to be TRUE).
- **success.y:** same explanation as for **success.x** (needed when **y** is used). Note that if the value/level coded as success is the same for **x** and **y**, **success.y** can be omitted.
- **digits:** number of decimals used to round statistics (default = 2 decimals). If the chosen rounding formats some non-zero values as zero, the number of decimals is increased so that all values have at least one significant digit, unless the argument **force.digits** (see below) is set to TRUE.
- **force.digits, use.scientific:** indicate whether reported values should be forcedly rounded to the number of decimals specified in **digits** even if non-zero values are rounded to zero (default = FALSE), and/or whether too large or too small numbers should be displayed using scientific notation (default = FALSE).

As seen in the previous sections, the two functions have additional specific parameters. In `CI.diffprop`, **conf.level** is used to specify the confidence level (default is 0.95). For function `TEST.diffprop`, **pdiff0** specifies the null hypothesis (the default is 0); if **x** and **y** are given as inputs, **pdiff0** is the value of the difference $p_x - p_y$; if **x** and **by** are given as inputs, the two levels of the variable specified in **by** are considered in their standard order (alphabetical or numeric, or order of the levels for factors), say **by1** and **by2** and **pdiff0** is the value of the difference $p_{by1} - p_{by2}$.

The argument **alternative** specifies the direction of the alternative hypothesis ("two.sided", "less", or "greater" if the difference between populations' proportions is different from **pdiff0**, lower than **pdiff0**, or higher than **pdiff0** respectively).

6.4.1 Confidence intervals for the difference between proportions

We illustrate how to build intervals using both arguments **x, y** (data on the two samples are in two separate vectors²) and arguments **x, by** (the two samples are obtained by splitting data in **x** based on an existing or appositely created vector taking two values). Note that if **x** and **y** are given as inputs, the confidence interval is built for $p_x - p_y$; if **x** and **by** are given as inputs, the two levels of the variable specified in **by** are considered in their standard order (alphabetical or numeric, or order of the levels for factors), say **by1** and **by2**, and the confidence interval is built for $p_{by1} - p_{by2}$.

We first consider input vectors that are **not binary or logical**, so that the value coded as success must be indicated.

²We will consider the case when **x** and **y** are two vectors in the environment; of course, in some applications the vectors with data on the two samples might be columns of the same dataframe. In this case, **x** and **y** will be the names of the two columns.

```
# based on x,y: build vectors with data on the two groups
WouldSuggest_F <- MktDATA$WouldSuggest[MktDATA$Gender == "F"]
WouldSuggest_M <- MktDATA$WouldSuggest[MktDATA$Gender == "M"]
CI.diffprop(x = WouldSuggest_M, y = WouldSuggest_F,
            success.x = "Yes", # success.x = success.y
            conf.level = 0.99, digits = 2) # no data specified
> Confidence interval for p_x - p_y
> Confidence level: 0.99
> x:WouldSuggest_M = Yes
> y:WouldSuggest_F = Yes
>
> Warning: 2 obs with NA removed from 'x' & 2 obs with NA removed from 'y'
> n_x n_y phat_x phat_y phat_x-phat_y s_X s_Y se Lower Upper
> 1176 1046 0.69 0.67 0.02 0.46 0.47 0.02 -0.03 0.07

# same result (not printed) using x,by: x split based on the by's (2) levels
CI.diffprop(x = WouldSuggest, by = Gender.R, success.x = "Yes", conf.level = 0.99, data = MktDATA)
```

The same procedures can be applied for **numeric** input vectors; for example below we consider the difference in the proportions of respondents who **did not join** any of the past campaigns (thus, success is 0) among females and males:

```
PastCampaigns_F <- MktDATA$PastCampaigns[MktDATA$Gender == "F"]
PastCampaigns_M <- MktDATA$PastCampaigns[MktDATA$Gender == "M"]
CI.diffprop(x = PastCampaigns_M, y = PastCampaigns_F,
            success.x = 0, conf.level = 0.99) # success.x = success.y
> Confidence interval for p_x - p_y
> Confidence level: 0.99
> x:PastCampaigns_M = 0
> y:PastCampaigns_F = 0
>
> Warning: 2 obs with NA removed from 'x' & 2 obs with NA removed from 'y'
> n_x n_y phat_x phat_y phat_x-phat_y s_X s_Y se Lower Upper
> 1176 1046 0.71 0.89 -0.17 0.45 0.32 0.02 -0.22 -0.13

# same result (not printed) with x split based on the by's (2) levels
CI.diffprop(x = PastCampaigns, by = Gender.R, success.x = 0, conf.level = 0.99, data = MktDATA)
```

Instead for the **binary** variable LastCampaign, indicating whether respondents joined (1) or not (0) the last marketing campaign), the arguments success.x and success.y can be omitted, provided that the value 1 corresponds to success. We compare once again the proportions of successes among males and females.

```
LastCampaign_F <- MktDATA$LastCampaign[MktDATA$Gender == "F"]
LastCampaign_M <- MktDATA$LastCampaign[MktDATA$Gender == "M"]
CI.diffprop(x = LastCampaign_M, y = LastCampaign_F, conf.level = 0.99, digits = 3)
> Confidence interval for p_x - p_y
> Confidence level: 0.99
> x:LastCampaign_M = 1
> y:LastCampaign_F = 1
>
> Warning: 2 obs with NA removed from 'x' & 2 obs with NA removed from 'y'
> n_x n_y phat_x phat_y phat_x-phat_y s_X s_Y se Lower Upper
> 1176 1046 0.179 0.117 0.063 0.384 0.321 0.015 0.024 0.101

# same as (not printed)
CI.diffprop(x = LastCampaign, by = Gender.R, conf.level = 0.99, digits = 3, data = MktDATA)
```

The same holds for the **logical** variable `Deals.ge50`, indicating whether or not (TRUE/FALSE) the share of products purchases with discount is higher than or equal to 0.5. For this variable we compare the proportions of successes among respondents with or without children, building a two-levels vector identifying the groups; note that `success.x` and `success.y` are omitted and therefore it is assumed that the success event is TRUE:

```
NoChildren <- MktDATA$Children == 0 # logical vector
Deals_w_child <- MktDATA$Deals.ge50[NoChildren == FALSE]
Deals_no_child <- MktDATA$Deals.ge50[NoChildren == TRUE]
CI.diffprop(x = Deals_w_child, y = Deals_no_child, conf.level = 0.9, digits = 3)
> Confidence interval for p_x - p_y
> Confidence level: 0.9
> x:Deals_w_child = TRUE
> y:Deals_no_child = TRUE
> n_x n_y phat_x phat_y phat_x-phat_y s_X s_Y se Lower Upper
> 1593 631 0.189 0.005 0.184 0.391 0.069 0.01 0.167 0.201

# same as (not printed)
CI.diffprop(x = Deals.ge50, by = NoChildren, conf.level = 0.9, digits = 3, data = MktDATA)
```

6.4.2 Hypotheses testing on the difference between proportions

Hypotheses on the proportions of independent samples can be tested using inputs and syntax identical to those needed to build confidence intervals, specifying the null value hypothesized for the difference and the direction of the alternative hypothesis instead of the confidence level. Note further that the standard error of the difference between sample proportions is estimated based on the sample proportions, except in the case when `pdiff0=0`, when it is based on the proportion of successes observed in the pooled samples.

In the examples below we report only the syntax based on the `x` and `y` inputs for the same examples considered in the previous section.

Character (or factor) input vectors:

```
# based on x,y: build vectors with data on the two groups
WouldSuggest_F <- MktDATA$WouldSuggest[MktDATA$Gender == "F"]
WouldSuggest_M <- MktDATA$WouldSuggest[MktDATA$Gender == "M"]
TEST.diffprop(x = WouldSuggest_F, y = WouldSuggest_M,
  success.x = "Yes", pdiff0 = 0, alternative = "two.sided")
> Test hypotheses on p_x - p_y
> x:WouldSuggest_F = Yes
> y:WouldSuggest_M = Yes
>
> Warning: 2 obs with NA removed from 'x' & 2 obs with NA removed from 'y'
> Null hypothesis H0: (p_x - p_y) = 0
> Alternative hypothesis H1: (p_x - p_y) ≠ 0
> n_x n_y phat_x phat_y phat_x-phat_y s_X s_Y se_0 stat p-value
> 1046 1176 0.67 0.69 -0.02 0.47 0.46 0.02 -1.08 0.28
```

Numeric input vectors

```
PastCampaigns_F <- MktDATA$PastCampaigns[MktDATA$Gender == "F"]
PastCampaigns_M <- MktDATA$PastCampaigns[MktDATA$Gender == "M"]
TEST.diffprop(x = PastCampaigns_F, y = PastCampaigns_M,
  success.x = 0, pdiff0 = 0.1, alternative = "greater")
> Test hypotheses on p_x - p_y
> x:PastCampaigns_F = 0
> y:PastCampaigns_M = 0
>
> Warning: 2 obs with NA removed from 'x' & 2 obs with NA removed from 'y'
> Null hypothesis H0: (p_x - p_y) = 0.1 or (p_x - p_y) ≤ 0.1
```

```
> Alternative hypothesis H1: (p_x - p_y) > 0.1
>   n_x  n_y phat_x phat_y phat_x-phat_y  s_X  s_Y  se stat p-value
> 1046 1176   0.89   0.71         0.17 0.32 0.45 0.02 4.51 <0.0001
```

Binary input vectors:

```
LastCampaign_F <- MktDATA$LastCampaign[MktDATA$Gender == "F"]
LastCampaign_M <- MktDATA$LastCampaign[MktDATA$Gender == "M"]
TEST.diffprop(x = LastCampaign_M, y = LastCampaign_F,
              pdiff0 = 0, alternative = "greater", digits = 3)
> Test hypotheses on p_x - p_y
>   x>LastCampaign_M = 1
>   y>LastCampaign_F = 1
>
> Warning: 2 obs with NA removed from 'x' & 2 obs with NA removed from 'y'
> Null hypothesis      H0: (p_x - p_y) = 0 or (p_x - p_y) ≤ 0
> Alternative hypothesis H1: (p_x - p_y) > 0
>   n_x  n_y phat_x phat_y phat_x-phat_y  s_X  s_Y se_0 stat p-value
> 1176 1046  0.179  0.117         0.063 0.384 0.321 0.015 4.139 <0.0001
```

Logical input vectors:

```
NoChildren <- MktDATA$Children == 0 # logical vector
Deals_w_child <- MktDATA$Deals.ge50[NoChildren == FALSE]
Deals_no_child <- MktDATA$Deals.ge50[NoChildren == TRUE]
TEST.diffprop(x = Deals_w_child, y = Deals_no_child,
              pdiff0 = 0.2, alternative = "less", digits = 3)
> Test hypotheses on p_x - p_y
>   x:Deals_w_child = TRUE
>   y:Deals_no_child = TRUE
> Null hypothesis      H0: (p_x - p_y) = 0.2 or (p_x - p_y) ≥ 0.2
> Alternative hypothesis H1: (p_x - p_y) < 0.2
>   n_x n_y phat_x phat_y phat_x-phat_y  s_X  s_Y  se  stat p-value
> 1593 631  0.189  0.005         0.184 0.391 0.069 0.01 -1.552  0.06
```

6.5 Goodness-of-fit and independence tests

We describe tests to verify if a univariate or a bivariate population distribution, relative to variables taking a **limited** number of values, differs from a theoretical probability distribution. In this case, we rely on the function `chisq.test()` available in base R, whose arguments will be discussed with reference to the specific tests.

6.5.1 Goodness-of-fit (fully specified probabilities)

The goodness-of-fit test for given probabilities verifies whether the distribution of a categorical or discrete variable (or factor) differs from a theoretical probability distribution. The null hypothesis that the population distribution coincides with the theoretical one is contrasted with the alternative hypothesis that the two distributions are different. The test can be performed using the function `chisq.test(x,p)`, where:

- **x**: is the **sample frequency distribution** (*counts*) of a variable (taking a limited number of values). Note therefore that the function **cannot be directly applied to a vector of data**.
- **p**: is a vector whose elements are the probabilities of the variable's levels theorized by the null hypothesis.

The function tests the hypothesis that the population probabilities equal those specified in **p**, or the hypothesis that they are all **equal** if **p** is not specified.

Below we distinguish between the case when the test is applied to raw data and the case when only aggregated data (thus, only the sample distribution) is available.

6.5.1.1 Test based on raw data. Consider the distribution of the variable `CustClass`, describing the classification of customers based on their relevance.

```
# tabulate the distribution using table() or distr.table.x()
table(MktDATA$CustClass)
>
>   Bronze   Silver   Gold Platinum
>   1500     324     232     168
distr.table.x(MktDATA$CustClass, freq = "prop")
> MktDATA$CustClass Prop
>   Bronze 0.67
>   Silver 0.15
>   Gold 0.10
>   Platinum 0.08
>   TOTAL 1.00
```

According to some scholars, the proportions of cases in the four (ordered) classes is $p = (0.8, 0.15, 0.4, 0.01)$. We can proceed with the test to assess whether the evident difference between the empirical and the theoretical distribution is significant:

```
chisq.test(x = table(MktDATA$CustClass), p = c(0.8,0.15,0.04,0.01))
>
>   Chi-squared test for given probabilities
>
> data:  table(MktDATA$CustClass)
> X-squared = 1229.4, df = 3, p-value < 2.2e-16
```

6.5.1.2 Test based on aggregated data. If only the sample frequency distribution (and no data) is available, argument **x** should be a vector whose elements are the observed sample counts. In the example below we test whether based on a given sample distribution it is possible to conclude that at the population level the probabilities of the distinct values are all equal (default value for **p** in function `chisq.test`):

```
# observed distribution
p.obs <- c(150,149,165,135,114)
chisq.test(x = p.obs) # same as chisq.test(x = p.obs, p = c(0.2,0.2,0.2,0.2,0.2))
>
>   Chi-squared test for given probabilities
>
```

```
> data: p.obs
> X-squared = 10.331, df = 4, p-value = 0.03521
```

Otherwise, we can test whether the first three levels occur with a double probability in the population compared to the last two:

```
# observed distribution
p.obs <- c(150,149,165,135,114)
chisq.test(x = p.obs, p = c(0.25,0.25,0.25,0.125,0.125))
>
> Chi-squared test for given probabilities
>
> data: p.obs
> X-squared = 40.818, df = 4, p-value = 2.932e-08
```

6.5.2 Test of independence

The test of independence contrasts the null hypothesis that two variables (with a limited number of values) are independent with the alternative hypothesis that they are instead associated. To perform the test, the function `chisq.test(x,y)` can be used, where `x` and `y` are two vectors (note that they cannot be the names of columns in a dataframe) with the same length. Note that `p` is not needed, because the function determines both the sample joint distribution and the joint distribution under the assumption of independence (given the marginal distributions), and then calculates the chi-square test statistic and its p-value. In the following we illustrate how to apply the test to raw data and directly to a cross-table.

6.5.2.1 Test based on raw data. Below we verify whether the variable `CustClass` is associated with `Education`.

```
chisq.test(x = MktDATA$Education, y = MktDATA$CustClass)
>
> Pearson's Chi-squared test
>
> data: MktDATA$Education and MktDATA$CustClass
> X-squared = 16.75, df = 9, p-value = 0.05278
```

Note that when testing the association between `CustClass` and `Children`:

```
chisq.test(x = MktDATA$Children, y = MktDATA$CustClass)
> Warning in chisq.test(x = MktDATA$Children, y = MktDATA$CustClass): Chi-squared
> approximation may be incorrect
>
> Pearson's Chi-squared test
>
> data: MktDATA$Children and MktDATA$CustClass
> X-squared = 678.94, df = 9, p-value < 2.2e-16
```

a warning is printed out. Actually, the test is based on an approximation, and it is not reliable when one or more of the *expected* frequencies are small (5 or less). Observe that:

```
distr.table.xy(x = Children, y = CustClass, data = MktDATA)
> Joint counts
>      CustClass
> Children Bronze Silver Gold Platinum TOTAL
> 0          193    142   157      139    631
> 1          867    160    68       26   1121
> 2          393     20     4        2    419
> 3           47      2      3         1     53
> TOTAL     1500    324   232     168   2224
```

Note for example that the expected count for the pair (2, "Platinum"), $53 \cdot 168 / 2224 = 4.003$, is lower than 5.

Additional Material

6.5.2.2 Test based on aggregated data. When only the contingency table characterizing two variables is available, it is necessary to store the table in a matrix, and to apply the function `chisq.test` to the matrix:

```
obs.joint <- matrix(c(15,20,5,7, 33,10,5,15, 20,5,10,20),
                    byrow = TRUE, nrow = 3)

obs.joint
>      [,1] [,2] [,3] [,4]
> [1,]  15  20   5   7
> [2,]  33  10   5  15
> [3,]  20   5  10  20
chisq.test(x = obs.joint)
>
> Pearson's Chi-squared test
>
> data:  obs.joint
> X-squared = 25.314, df = 6, p-value = 0.0002987
```

6.6 Test on the correlation coefficient

The function `cor.test()` in base R test the hypothesis of null correlation against bilateral or unilateral alternative hypotheses, and also builds the confidence interval for the coefficient. Arguments `x` and `y` are the names of two **numerical** vectors (they cannot be the names of columns in a dataframe) and `alternative` specifies the direction of the alternative hypothesis ("two.sided", "less", and "greater"). A call of the function based on default values performs a bilateral test:

```
cor.test(MktDATA$Baseline, MktDATA$TotVal)
>
> Pearson's product-moment correlation
>
> data: MktDATA$Baseline and MktDATA$TotVal
> t = 77.468, df = 2194, p-value < 2.2e-16
> alternative hypothesis: true correlation is not equal to 0
> 95 percent confidence interval:
>  0.8441216 0.8665468
> sample estimates:
>      cor
> 0.8557355
```

When the alternative hypothesis is unilateral the obtained interval has lower endpoint equal to -1 (if `alternative="less"`) or upper endpoint equal to 1 (if `alternative="greater"`).

```
cor.test(x= MktDATA$Baseline, y= MktDATA$TotVal, alternative = "less", conf.level = 0.99)
>
> Pearson's product-moment correlation
>
> data: MktDATA$Baseline and MktDATA$TotVal
> t = 77.468, df = 2194, p-value = 1
> alternative hypothesis: true correlation is less than 0
> 99 percent confidence interval:
> -1.0000000 0.8684825
> sample estimates:
>      cor
> 0.8557355
```


7 The linear regression model

In this chapter we present one of the most important tools in data analysis, the linear regression model. We first illustrate how to estimate a regression model. Then, we show how to perform a basic analysis of residuals, and how to obtain predictions from a fitted model.

In this specific case, we prefer to rely on R functions, which are typically employed in more advanced courses; even so we still refer to the dataframe `MktDATA` available in package `UBStats`. Therefore, to obtain results illustrated in this chapter it is necessary to load the package and the dataframe:

```
library(UBStats)
data("MktDATA")
```

7.1 Estimating a regression model

To estimate a regression model, the function `lm()` is available in R. The function has different arguments, but we will use default values for most of them. Actually, for our purposes the generic call of the function will be `lm(formula, data)`, where `formula` is a symbolic description of the model to be fitted. A typical model has the form `response ~ terms` where `response` is the (**numeric**) response vector and `terms` is a series of terms which specifies a linear predictor for response. For example `response ~ first` indicates a linear model having only vector (or factor) `first` as explanatory variable (simple linear regression model), whereas `response ~ first + second + third` specifies a (multiple) linear model with three explanatory variables. Importantly, both `response` and the explanatory variables specified in `terms` can be objects in the environment as well as names of the columns in the dataframe specified in `data`.

7.1.1 Simple linear regression

To illustrate how to perform a regression analysis, we start estimating a simple regression model, relating variable `TotVal` (total amount spent by client in the last 2 years) to variable `NDeals` (number of products purchases with discount) in the dataframe `MktDATA`.

The following calls estimate the same model:

```
# three equivalent formulations:
lm(formula = TotVal ~ NDeals, data = MktDATA) # named formula
lm(TotVal ~ NDeals, data = MktDATA) # first argument unnamed
lm(MktDATA$TotVal ~ MktDATA$NDeals) # use vectors instead of columns in data
```

We will discuss later (see sec. 7.1.3) about some relevant differences between the two calls. For now note that the first call returns:

```
lm(formula = TotVal ~ NDeals, data = MktDATA) # named formula
>
> Call:
> lm(formula = TotVal ~ NDeals, data = MktDATA)
>
> Coefficients:
> (Intercept)      NDeals
>    613.26       -21.41
```

Thus, the simple call of function `lm` returns only point estimates of the coefficients. Indeed, `lm` stores in a **list** a large number of results related to the estimated model, that are available only if the function's output is **assigned** to an object:

```
Mod1 <- lm(formula = TotVal ~ NDeals, data = MktDATA)
```

We will illustrate in more details such output in Section 7.1.3, but at this point we focus on the model's estimated coefficients (and their significance) and goodness of fit. The function `summary()` applied to the output of function `lm()` returns a summary of the estimated model:

```
summary(Mod1)
>
> Call:
> lm(formula = TotVal ~ NDeals, data = MktDATA)
```

```

>
> Residuals:
>   Min       1Q   Median       3Q      Max
> -586.3 -508.0 -196.8  404.2 1899.2
>
> Coefficients:
>               Estimate Std. Error t value Pr(>|t|)
> (Intercept)   613.260     19.386   31.634 < 2e-16 ***
> NDeals        -21.414       6.504   -3.293  0.00101 **
> ---
> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
>
> Residual standard error: 575.1 on 2222 degrees of freedom
> Multiple R-squared:  0.004855,    Adjusted R-squared:  0.004407
> F-statistic: 10.84 on 1 and 2222 DF,  p-value: 0.001008

```

The function prints out the call, the five numbers summary for residuals, the estimated coefficients with their standard error estimates and their p-value, as well as the standard error of residuals, the R-squared, and the Adjusted R-squared and the F-statistic (not relevant in the case of simple linear regression).

It is also possible to obtain **confidence intervals** for all or selected regression coefficients by applying the function `confint(object, parm, level)`. In the function, `object` is the output of `lm`, `parm` can be used to possibly select the coefficients of the variables of interest, and can be a vector of numbers indicating the positions of the variables in the formula as well as a vector with their names (if `parm` is not specified all the coefficients are considered); finally `level` allows specifying the confidence level (default = 0.95).

```

confint(Mod1)
>               2.5 %       97.5 %
> (Intercept) 575.24328 651.276882
> NDeals      -34.16882  -8.659991

```

7.1.2 Multiple linear regression

Turning now to a **multiple** linear regression, we build a model explaining `TotVal` based on a more articulated set of variables of different **types**. As shown below, the function `lm()` allows including among the explanatory variables binary vectors (here `LastCampaign`) as well as character vectors or factors (here `CustClass`). In the latter case, a proper set of dummy variables (binary variables) is automatically created:

```

Mod2 <- lm(TotVal ~ Kids + NDeals + LastCampaign + CustClass, data = MktDATA)
summary(Mod2)
>
> Call:
> lm(formula = TotVal ~ Kids + NDeals + LastCampaign + CustClass,
>     data = MktDATA)
>
> Residuals:
>   Min       1Q   Median       3Q      Max
> -704.59 -98.96  -44.23   95.17  704.60
>
> Coefficients:
>               Estimate Std. Error t value Pr(>|t|)
> (Intercept)    242.985     7.694   31.582 < 2e-16 ***
> Kids           -196.277     7.826  -25.079 < 2e-16 ***
> NDeals          35.989     1.996   18.029 < 2e-16 ***
> LastCampaign    36.455    10.541    3.458 0.000553 ***
> CustClassSilver  649.477    11.271   57.625 < 2e-16 ***
> CustClassGold   1045.556    12.923   80.907 < 2e-16 ***

```

```
> CustClassPlatinum 1528.773      15.110 101.174 < 2e-16 ***
> ---
> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
>
> Residual standard error: 170.2 on 2217 degrees of freedom
> Multiple R-squared:  0.913,    Adjusted R-squared:  0.9128
> F-statistic: 3880 on 6 and 2217 DF,  p-value: < 2.2e-16
```

Note that the names of the coefficients of the dummy variables created for a factor are obtained as combinations of the name of the variable followed by the factor's levels; for example `CustClassGold` is the binary variable indicating whether the class of a client is "Gold" (`CustClass=="Gold"`); note also that the baseline category for the set of dummy variables is the first level of the factor, thus "Bronze" for `CustClass` (same holds for categorical variables whose levels are arranged in alphabetical order). Below are the 99% confidence intervals for the coefficients:

```
# not run: selected coefficients
confint(Mod2, parm = c("Kids", "CustClassGold"), level = 0.99)
# confidence intervals for all the coefficients
confint(Mod2, level = 0.99)
```

```
>
>          2.5 %    97.5 %
> (Intercept)    227.89677 258.0727
> Kids          -211.62489 -180.9291
> NDeals         32.07403  39.9030
> LastCampaign   15.78427  57.1255
> CustClassSilver 627.37474 671.5794
> CustClassGold  1020.21359 1070.8985
> CustClassPlatinum 1499.14152 1558.4051
```

7.1.3 A closer look at the regression output

The content of the list returned by the function `lm()` is very articulated:

```
names(Mod2) # names of the elements in lm's output list
> [1] "coefficients" "residuals"      "effects"      "rank"
> [5] "fitted.values" "assign"          "qr"          "df.residual"
> [9] "contrasts"     "xlevels"        "call"        "terms"
> [13] "model"
```

The list contains all the information needed to make inference on the parameters of the model, to obtain predictions, and to evaluate residuals and other diagnostics. We will typically use specifically-designed functions to obtain such results, nonetheless it is worth noting that it is possible to easily access some focal output, such as coefficients, the residuals and the fitted values of the model:

```
Mod2$coefficients # point estimates
>      (Intercept)      Kids      NDeals      LastCampaign
>      242.98471      -196.27700      35.98851      36.45488
>      CustClassSilver      CustClassGold      CustClassPlatinum
>      649.47709      1045.55604      1528.77329
head(Mod2$fitted.values) # fitted values for the first cases
>      1      2      3      4      5      6
> 928.4503 315.4281 190.6618 314.9617 1036.4159 278.9732
head(Mod2$residuals) # residuals of the first cases
>      1      2      3      4      5      6
> 215.54969 224.57190 -106.66177 -290.96174 -84.41585 -156.97322
```

The component named `model` in the list is a dataframe containing data on the variables involved in regression:

```

head(Mod2$model)
>   TotVal Kids NDeals LastCampaign CustClass
> 1   1144    0      1           0    Silver
> 2    540    0      1           1    Bronze
> 3     84    1      4           0    Bronze
> 4     24    0      2           0    Bronze
> 5    952    0      4           0    Silver
> 6    122    0      1           0    Bronze
dim(Mod2$model)
> [1] 2224    5
dim(MktDATA)
> [1] 2224   26

```

Note that `Mod2$model` and `MktDATA` include the same number of cases, because there are **no cases** in `MktDATA` with **missing values** on the **variables involved in the regression**.

Nonetheless, consider the following model:

```

Mod3 <- lm(TotVal ~ Baseline + Kids + Age + LastCampaign + CustClass, data = MktDATA)
summary(Mod3)
>
> Call:
> lm(formula = TotVal ~ Baseline + Kids + Age + LastCampaign +
>   CustClass, data = MktDATA)
>
> Residuals:
>   Min       1Q   Median       3Q      Max
> -853.49 -101.04  -25.07   85.07  564.55
>
> Coefficients:
>               Estimate Std. Error t value Pr(>|t|)
> (Intercept)    -67.4798    20.4396  -3.301  0.000977 ***
> Baseline     1039.1380    36.3568  28.582 < 2e-16 ***
> Kids          -43.2938     8.4354  -5.132 3.11e-07 ***
> Age             0.9342     0.2927   3.192 0.001434 **
> LastCampaign   -23.1334     9.9470  -2.326 0.020127 *
> CustClassSilver  506.0214    11.5851  43.679 < 2e-16 ***
> CustClassGold   819.1765    14.0020  58.504 < 2e-16 ***
> CustClassPlatinum 1219.4654    16.9984  71.740 < 2e-16 ***
> ---
> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
>
> Residual standard error: 155 on 2188 degrees of freedom
> (28 observations deleted due to missingness)
> Multiple R-squared:  0.928,    Adjusted R-squared:  0.9278
> F-statistic: 4029 on 7 and 2188 DF,  p-value: < 2.2e-16

```

As reported in the output, in this case 28 observations were deleted because of missingness on one or more variables in the model. Indeed:

```

head(Mod3$model)
>   TotVal Baseline Kids Age LastCampaign CustClass
> 1   1144 0.5298829    0  35           0    Silver
> 2    540 0.4398487    0  59           1    Bronze
> 3     84 0.2483518    1  45           0    Bronze

```

```
> 4      24 0.1847814      0 73      0      Bronze
> 5     952 0.3896539      0 49      0      Silver
> 6     122 0.2727245      0 55      0      Bronze
dim(Mod3$model)
> [1] 2196      6
dim(MktDATA)
> [1] 2224     26
```

Note that now `Mod3$model` has less cases compared to `MktDATA`, because it does not include cases with missing values; the same holds for `Mod3$residuals` and `Mod3$fitted.values`. Specifically, the element `na.action` in `Mod3` lists the position (row) of the excluded cases:

```
print(Mod3$na.action)
> 33 256 323 392 477 565 577 627 755 870 930 1025 1067 1136 1181 1449
> 33 256 323 392 477 565 577 627 755 870 930 1025 1067 1136 1181 1449
> 1460 1470 1505 1642 1693 1778 1811 1881 1936 2030 2062 2101
> 1460 1470 1505 1642 1693 1778 1811 1881 1936 2030 2062 2101
> attr(,"class")
> [1] "omit"
```

This has to be taken into consideration in order to avoid a wrong combination of information in the input data and in the output objects.

There is another relevant aspect that deserves attention, particularly with reference to predictions based on the model (see next section). Indeed, note that the names of the columns of the dataframe `Mod3$model`:

```
colnames(Mod3$model)
> [1] "TotVal"      "Baseline"      "Kids"          "Age"          "LastCampaign"
> [6] "CustClass"
```

match with the names of the columns `MktDATA` involved in the regression.

Consider that, as mentioned when introducing the simple linear regression model, the function `lm()` can be applied also by referring directly to the columns of a dataframe, for example:

```
Mod4 <- lm(MktDATA$TotVal ~ MktDATA$Baseline + MktDATA$Kids + MktDATA$Age +
           MktDATA$LastCampaign + MktDATA$CustClass)
```

This call estimates exactly the same model as `Mod3`. Nonetheless the column names of the dataframe created by the function `lm()` in this case are:

```
colnames(Mod4$model)
> [1] "MktDATA$TotVal"      "MktDATA$Baseline"      "MktDATA$Kids"
> [4] "MktDATA$Age"        "MktDATA$LastCampaign" "MktDATA$CustClass"
```

Thus, even if `Mod4` was estimated based on the exact same variables as `Mod3`, the names of the columns in `Mod4$model` **do not coincide** with those of the columns in `MktDATA`, because even if the variables are columns in `MktDATA`, R will treat them as vectors and will not connect them to the dataframe. This is something to be kept in mind when obtaining predictions, as discussed in the next section.

7.2 Predictions

Predictions on new data based on an estimated model can be obtained using the function `predict(object, newdata, interval = c("none", "confidence", "prediction"), level = 0.95)` (attention limited to selected arguments). The function allows obtaining the point estimate and the confidence interval for the prediction corresponding to a specific set of values for the independent variables. In the function:

- **object** is the output of the `lm()` function, containing all the needed information on the estimated model
- **newdata** is the name of a **new** dataframe containing the set of values of the independent variables with respect to which the prediction had to be obtained. If this argument is missing, the predictions are obtained for all the cases used to estimate the

regression model.

- **interval** specifies the type of required prediction interval, if any; it can take only three values: `confidence` for the confidence interval for the prediction of the mean value, `prediction` for the prediction interval for the individual value, and `none` if no interval is requested (in this case – the default – only the point prediction will be returned)
- **level** specifies the confidence level for the confidence or prediction interval (default is 0.95)

The most important point of attention in the usage of the function concerns the definition of the dataframe specified in the argument `newdata`. Such dataframe should contain a row for each combination of the independent variables, and **importantly** it must contain the same **variables** included in the model, with their exact same names, in order to guarantee the correct match between variables in the model and variables in the dataframe (see sec. 7.1.3 for details on this very important point).

To illustrate how to use the function, we start considering the simple linear model relating `TotVal` to `NDeals`; assume one wants to predict the `TotVal` for a client with `NDeals=6`:

```
# create a dataframe:
topredict1 <- data.frame(NDeals = 6)
topredict1
>   NDeals
> 1      6
# apply function predict:
predict(Mod1, topredict1, interval = "none") # only point estimate
> 1
> 484.7737
predict(Mod1, topredict1, interval = "confidence") # 95% interval for the mean
>   fit      lwr      upr
> 1 484.7737 432.0615 537.4858
predict(Mod1, topredict1, interval = "prediction", level = 0.99) # 99% interval for individual value
>   fit      lwr      upr
> 1 484.7737 -999.5541 1969.101
```

Note that the function reports the point fit and the lower and upper bounds of the required interval.

It is also possible to obtain predictions corresponding to more than one value:

```
# create a dataframe:
topredict1 <- data.frame(NDeals = c(2,4,6))
topredict1
>   NDeals
> 1      2
> 2      4
> 3      6
# obtain the 99% intervals for individual values:
predict(Mod1, topredict1, interval = "prediction", level = 0.99)
>   fit      lwr      upr
> 1 570.4313 -912.6208 2053.483
> 2 527.6025 -955.7086 2010.914
> 3 484.7737 -999.5541 1969.101
```

The same exact procedure can be used to obtain predictions based on a multiple regression model, by building a dataframe containing values on all the explanatory variables in the model. For example:

```
# the model:
Mod3 <- lm(TotVal ~ Baseline + Kids + Age + LastCampaign + CustClass, data = MktDATA)

# dataframe with data on variables with which to predict:
topredict3 <- data.frame(Baseline = 0.75, Kids = 0, Age = 35,
                        LastCampaign = 1, CustClass = "Gold")
topredict3
```

```

> Baseline Kids Age LastCampaign CustClass
> 1      0.75    0  35                1      Gold

# apply function predict:
predict(Mod3, topredict3) # only point estimate
>      1
> 1540.612
predict(Mod3, topredict3, interval = "confidence", level = 0.99) # 99% interval for the mean
>      fit      lwr      upr
> 1 1540.612 1503.059 1578.166
predict(Mod3, topredict3, interval = "prediction", level = 0.99) # 99% interval for individual value
>      fit      lwr      upr
> 1 1540.612 1139.328 1941.897

```

Note importantly that, as remarked in section 7.1.3, if the same model is estimated using the call:

```

Mod4 <- lm(MktDATA$TotVal ~ MktDATA$Kids + MktDATA$NDeals + MktDATA$LastCampaign + MktDATA$CustClass)
colnames(Mod4$model)
> [1] "MktDATA$TotVal"      "MktDATA$Kids"      "MktDATA$NDeals"
> [4] "MktDATA$LastCampaign" "MktDATA$CustClass"

```

the names of the columns in the `Mod4$model` do not coincide with those in `MktDATA`. Therefore obtaining predictions based on `Mod4` using:

```

predict(Mod4, topredict3, interval = "confidence")

> Warning: 'newdata' had 1 row but variables found have 2224 rows
>      fit      lwr      upr
> 1  928.4503  908.9867  947.9139
> 2  315.4281  292.1681  338.6881
> 3  190.6618  179.0424  202.2812
> 4  314.9617  302.0315  327.8920
> 5 1036.4159 1016.4617 1056.3701
> 6  278.9732  265.4789  292.4675

```

does not return the expected output. Indeed, in this case the function `predict` is not able to **match** the columns in `Mod4$model` with those in `topredict3`, and predictions **are obtained for all the cases in** `Mod4$model`!! Therefore, it is recommendable to specify the argument `data` in function `lm` to easily obtain predictions and to avoid errors.

7.3 Analysis of residuals and of influential observations

In this section, we briefly describe how to analyse a model's residuals and other diagnostics using graphical tools.

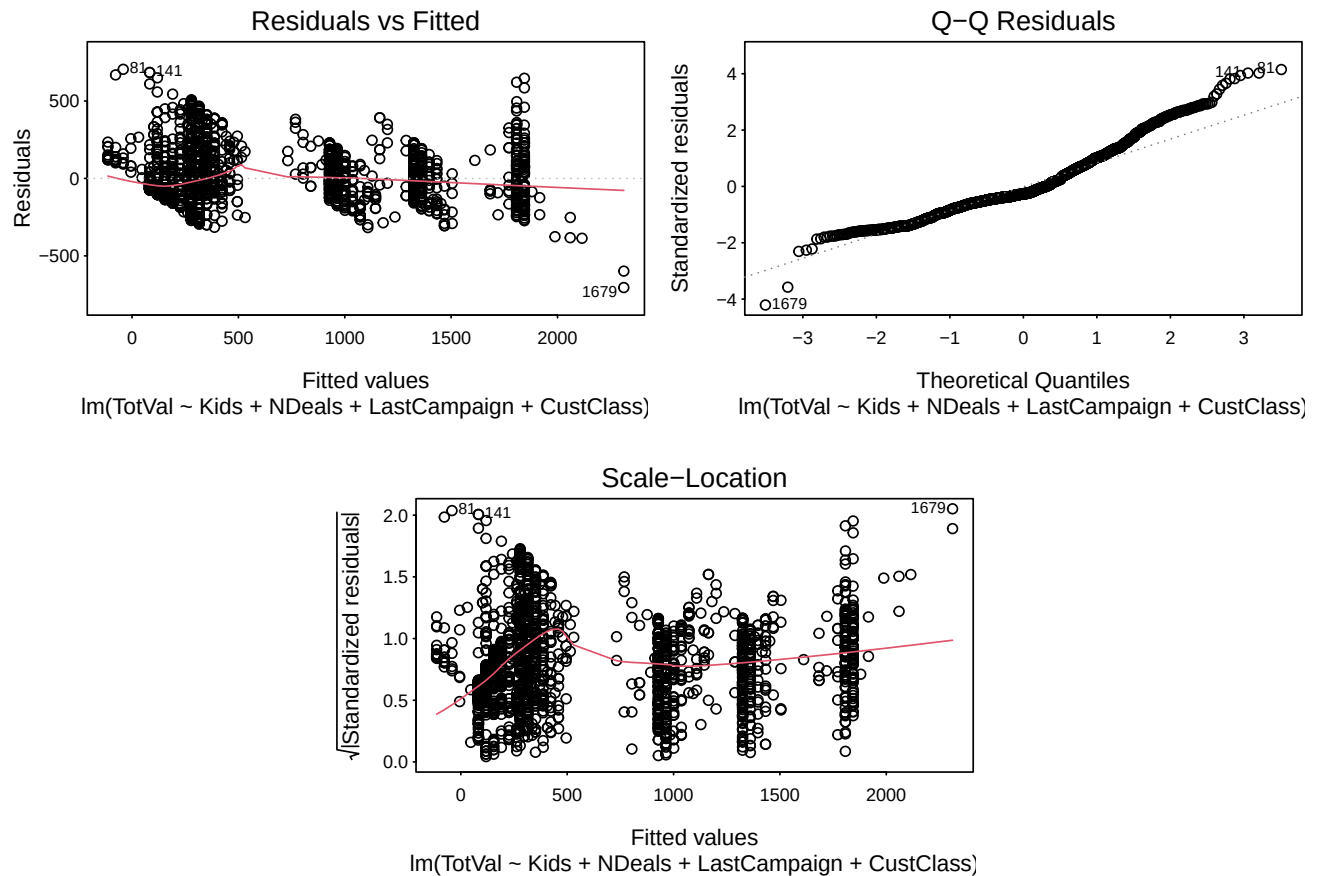
A first type of plots focuses on **regression's raw residuals**, obtained as the difference between the actual and the predicted response values, to assess whether they exhibit a pattern. Typically, also **standardized residuals** are analyzed; indeed residuals have a unit of measurement and can have very high or very low variance and range, and standardization allows to identify cases whose deviation from the model is relevant (standardized residuals higher than 3 in absolute value), which are potential outliers (even if specific diagnostics have been developed at this aim, see below).

Below are some interesting plots of a model's residuals that can be easily obtained using the basic R function `plot(object, which)`, where `object` is the output of function `lm`, allows selecting one out of 6 types of plots (specified in the argument `which`).

```

plot(Mod3, which = 1)
plot(Mod3, which = 3)
plot(Mod3, which = 2)

```

In the top-left figure (`which=1`) raw residuals (difference between the actual and the predicted response values) are plotted against fitted values, whereas in the top-right plot (`which=3`) the square root of standardized residuals are plotted against the fitted values. Such plots are very useful to assess whether residuals are patterned and (top-right plot) whether their variance increases (or decreases) depending on the fitted values.

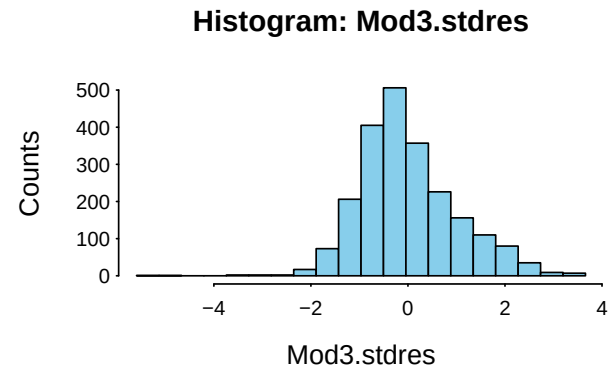
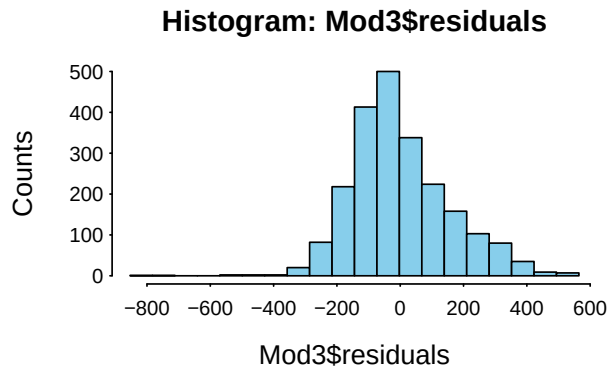
The bottom-left plot (`which=2`) reports the q-q plot of the **standardized** residuals. This is a plot where the quantiles of the distribution of the standardized residuals are plotted against the quantiles of the normal distribution; the 45-degrees line in the plot allows evaluating the closeness of the empirical distribution of residuals to the normal distribution.

Notably, fitted values and raw residuals can be easily extracted from the output of function `lm()`, as seen before, and standardized residuals can be easily obtained by applying the function `rstandard()` to the output of function `lm()`. Focusing on the model `Mod3` estimated in the previous sections:

```
head(Mod3$fitted.values)
>      1      2      3      4      5      6
> 1021.8587 421.5657 189.3353 192.7272 889.2196 267.2974
head(Mod3$residuals)
>      1      2      3      4      5      6
> 122.14133 118.43426 -105.33533 -168.72723 62.78039 -145.29737
Mod3.stdres <- rstandard(Mod3)
head(Mod3.stdres)
>      1      2      3      4      5      6
> 0.7900366 0.7661396 -0.6801140 -1.0908917 0.4058481 -0.9383647
```

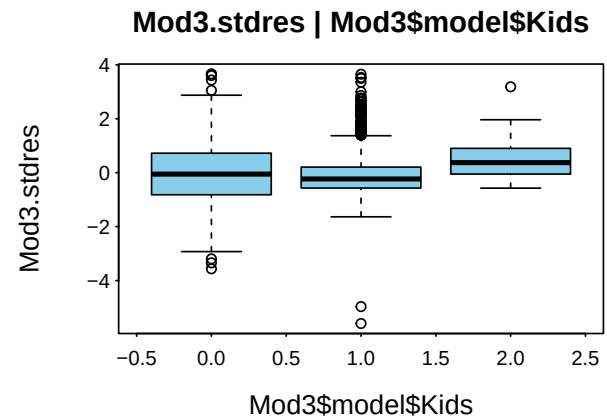
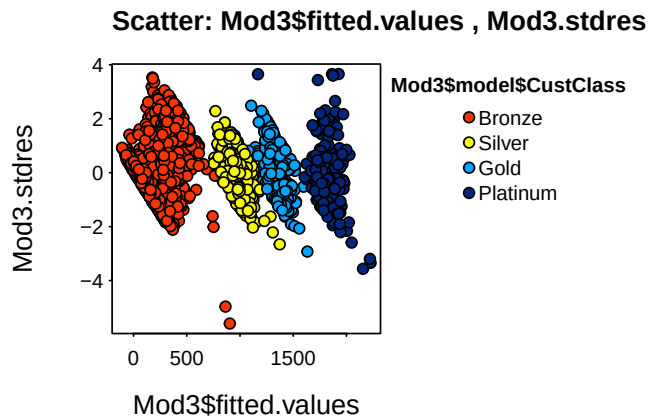
Such residuals and fitted values can be analyzed using other types of plots. For example, one might be interested to explore the distribution of residuals using histograms:

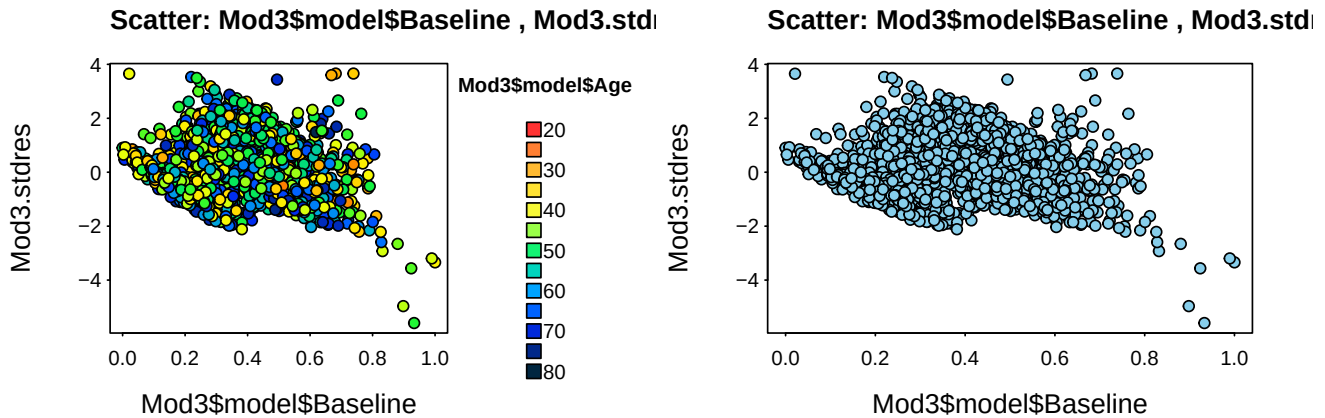
```
distr.plot.x(x = Mod3$residuals, breaks = 20, plot.type = "histogram")
distr.plot.x(x = Mod3.stdres, breaks = 20, plot.type = "histogram")
```



It can also be useful to plot residuals against one or more explanatory variables, in order to understand whether there is a pattern and/or higher or lower residuals are observed corresponding to specific values of the explanatory variables. Selected plots are reported below; note that we refer to data in the dataframe `Mod3$model` to be sure to refer to the cases actually used in computations:

```
# residuals against fitted values colored by the values of Kids
distr.plot.xy(x = Mod3$fitted.values, y = Mod3.stdres, plot.type = "scatter", var.c = Mod3$model$CustClass)
distr.plot.xy(x = Mod3$model$Kids, y = Mod3.stdres, plot.type = "boxplot")
distr.plot.xy(x = Mod3$model$Baseline, y = Mod3.stdres, plot.type = "scatter", var.c = Mod3$model$Age)
distr.plot.xy(x = Mod3$model$Baseline, y = Mod3.stdres, plot.type = "scatter")
```

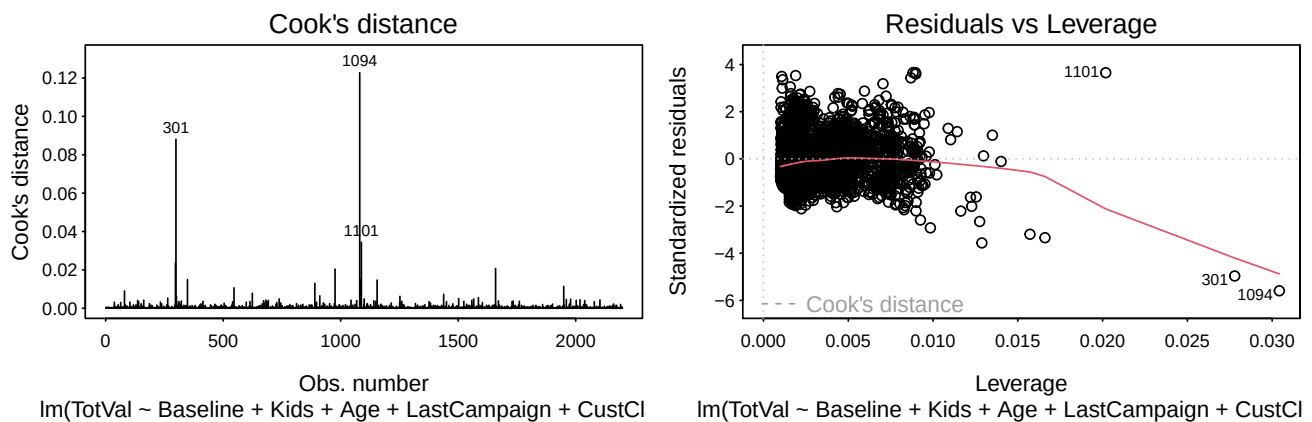




7.3.1 Other regression diagnostics

Other graphical tools can be used to assess the possible presence of **influential** cases. Cases that are extreme with respect to the explanatory variables might attract the regression model and unduly influence the results of regression. To identify possibly influential cases, the distance of cases from the others with respect to the explanatory variables is measured by the so-called **leverage**. Another useful measure is the so-called **Cook's distance** that measures the effect of deleting a given observation from the model. Points with a large Cook's distance and/or with a high leverage are potentially influential. Below are plots related to measures of influence that can be obtained using the function `plot()` introduced before:

```
plot(Mod3, which = 4)
plot(Mod3, which = 5)
```



Note that in all the plots the observations at rows 301, 1094 and 1101 (in the **original dataframe**) are indicated as critical, being characterized by high residual and by high values of the influential statistics. To printout these cases and to compare them with the others:

```
summary(Mod3$model)
> TotVal      Baseline      Kids      Age
> Min.       : 6.0      Min.    :0.0000   Min.    :0.000   Min.    :25.00
> 1st Qu.    : 56.0     1st Qu.:0.2033   1st Qu.:0.000   1st Qu.:43.00
> Median     : 345.0     Median :0.3161   Median :0.000   Median :50.00
> Mean       : 564.7     Mean    :0.3420   Mean     :0.444   Mean    :51.08
> 3rd Qu.    : 964.2     3rd Qu.:0.4650   3rd Qu.:1.000   3rd Qu.:61.00
> Max.       :2491.0     Max.    :1.0000   Max.     :2.000   Max.    :80.00
> LastCampaign CustClass
> Min.         :0.0000   Bronze  :1479
```

```

> 1st Qu.:0.0000 Silver : 322
> Median :0.0000 Gold   : 229
> Mean   :0.1507 Platinum: 166
> 3rd Qu.:0.0000
> Max.   :1.0000
print(MktDATA[c(301,1094,1101), c("TotVal","Baseline","Kids","Age","LastCampaign","CustClass")])
>      TotVal  Baseline Kids Age LastCampaign CustClass
> 301      105 0.89823359  1  44              0   Bronze
> 1094      51 0.93297136  1  49              0   Bronze
> 1101     1729 0.02101121  1  41              0  Platinum

```

Note that the first two cases are characterized by very high values of Baseline while having a very low value of TotVal, which explains both the high values of influential statistics and the high values of residuals. In the same way, the last case is characterized by an extremely low level of Baseline while having a very high value of TotVal.